

A Survey on Virtual Machine Migration: Challenges, Techniques, and Open Issues

Fei Zhang¹, Student Member, IEEE, Guangming Liu, Xiaoming Fu, Senior Member, IEEE, and Ramin Yahyapour

Abstract—When users flood in cloud data centers, how to efficiently manage hardware resources and virtual machines (VMs) in a data center to both lower economical cost and ensure a high service quality becomes an inevitable work for cloud providers. VM migration is a cornerstone technology for the majority of cloud management tasks. It frees a VM from the underlying hardware. This feature brings a plenty of benefits to cloud providers and users. Many researchers are focusing on pushing its cutting edge. In this paper, we first give an overview of VM migration and discuss both its benefits and challenges. VM migration schemes are classified from three perspectives: 1) manner; 2) distance; and 3) granularity. The studies on non-live migration are simply reviewed, and then those on live migration are comprehensively surveyed based on the three main challenges it faces: 1) memory data migration; 2) storage data migration; and 3) network connection continuity. The works on quantitative analysis of VM migration performance are also elaborated. With the development and evolution of cloud computing, user mobility becomes an important motivation for live VM migration in some scenarios (e.g., fog computing). Thus, the studies regarding linking VM migration to user mobility are summarized as well. At last, we list the open issues which are waiting for solutions or further optimizations on live VM migration.

Index Terms—Cloud computing, data center, virtual machine migration, pre-copy, post-copy, hybrid-copy, network connection, user mobility, performance analysis.

I. INTRODUCTION

VIRTUALIZATION technology divides a physical server into several isolated execution environments by deploying a layer (i.e., Virtual Machine Manager (VMM) or hypervisor) on top of hardware resources or operating system (OS). Each execution environment, i.e., Virtual Machine (VM), independently runs with an OS and applications without mutual interruption on each other. At the beginning, virtualization technology was not widely used due to a variety of reasons. For example, it will occupy a portion of hardware resources (CPU and memory) [1]. Furthermore, the

poor network bandwidth also hindered vendors to lease their idle physical servers to clients. As the related technologies evolve, such as the utilization of Fibre Channel (FC) [2], the improvement of hardware performance [3], the development of security technology [4], etc., a new service model—cloud computing [5], [6] emerges with the foundation of virtualization technology [7]. In cloud computing, big companies can piecemeal their spare hardware resources and rent them to customers in a pay-as-you-go manner, and users can quickly start to work on a VM without the big expense of hardware purchase and maintenance.

Because an increasing number of users choose cloud data centers to hold their applications [8], how to efficiently manage the VMs in a data center becomes a big issue. For example, some servers may be overloaded, while others are idling; if a server fails, all VMs on it will be impacted; etc. All these problems (how to evenly distribute tasks to servers, how to protect VMs from hardware failure, etc.) are solved along with the advent of a critical technology—VM migration. VM migration originates from process migration [9], [10]. However, process migration suffers from residential dependency problem which prevents it from being used for cloud management. VM migration makes a VM not fixed on the server where it is created. We can move a VM from one server to another, even from one data center to another data center. The majority of cloud management operations are supported by VM migration, such as server consolidation [11], zero-downtime hardware maintenance [5], [12], energy management [13], [14], and traffic management [15].

VM migration is not an operation only bringing benefits. It also introduces overheads to all the involved roles (the migrated VM, the source host, the destination host, the co-located VMs on these two hosts) [16]–[18]. Therefore, VM migration must be carefully applied for cloud management. Many studies [17], [19]–[21] on improving its performance and lowering its side effects were proposed over the past years.

There already are some previous works [16], [22]–[24] trying to summarize the achievements in the area of VM migration. Xu *et al.* [16] focus on the works about VM performance overhead, not migration techniques. They regard VM migration as one factor lowering VM performance since VM migration not only interferes on the migrated VM, but also the others located on the source and the destination hosts. Medina and García [22] survey VM migration mechanisms based on projects, but do not extract the common technologies between them. In addition, there is a lack of detailed analysis and comparison between different

Manuscript received June 6, 2017; revised October 27, 2017 and December 19, 2017; accepted January 15, 2018. Date of publication January 17, 2018; date of current version May 22, 2018. This work was supported by EU FP7 Marie Curie ITN CleanSky Project under Contract 607584. (Corresponding author: Fei Zhang.)

F. Zhang and X. Fu are with the Institute of Computer Science, University of Göttingen, 37077 Göttingen, Germany (e-mail: fei.zhang@gwdg.de; fu@cs.uni-goettingen.de).

G. Liu is with the National Supercomputer Center of Tianjin, Tianjin 300457, China (e-mail: liugm@nscc-tj.gov.cn).

R. Yahyapour is with Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen, 37077 Göttingen, Germany (e-mail: ramin.yahyapour@gwdg.de).

Digital Object Identifier 10.1109/COMST.2018.2794881

TABLE I
COMPARISON BETWEEN THE REVIEWED TOPICS OF
EXISTING SURVEYS AND OUR PAPER

Pp \ Tp	T	N	M	S	Ne	MM	P	U	SC	O
[16]							✓			
[22]		✓	✓	✓						✓
[23]	✓	✓	✓						✓	✓
[24]					✓					
Ours	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

The abbreviation list: *Pp*: paper. *Tp*: covered topics. *T*: taxonomy. *N*: non-live migration. *M*: memory data migration. *S*: storage data migration. *Ne*: network connection continuity. *MM*: multiple migration. *P*: migration performance analysis. *U*: user mobility-induced migration. *SC*: Summary and comparison between migration technologies. *O*: open research topics.

mechanisms. Ahmad *et al.* [23] make a detailed taxonomy for VM migration schemes, and then review migration technologies from the aspects of pre-copy, post-copy, hybrid-copy, and non-live migration. However, they only mentioned a few results regarding VM migration across data centers and multiple migration which are hot topics at present. Kokkinos *et al.* [24] summarize the technologies of live migration and disaster recovery for long-distance networks. Both of them face transferring a big amount of data over a slow network link. Live migration is one-time replication, while disaster recovery is a continuous operation. But they review only the works regarding VM migration across data centers from the perspective of network performance optimization. Some other surveys regarding VM migration [25]–[28] are either a simple elaboration or covered by the above mentioned works.

In this paper, we comprehensively review all technologies on VM migration, from non-live migration to live migration, from Local Area Network (LAN) migration to Wide Area Network (WAN) migration, from single migration to multiple migration, from user mobility-induced migration to Network Function Virtualization (NFV) Instance migration, etc. Our paper not only cover all contents of previously listed surveys, but also expand them with the new achievements in the area of live VM migration (such as the migration technologies in the scenario of mobile edge computing), comparisons between different migration mechanisms, and a discussion of outstanding research topics, etc., as shown in TABLE I. The contributions of this paper are summarized as follows:

- 1) The three challenges (memory data migration, storage data migration and network connection continuity) encountered by live VM migration in conventional cloud computing and the compatibility between the technologies of solving different challenges are discussed.
- 2) A new taxonomy of VM migration is designed. Migration schemes are categorized by migration manner (non-live and live), migration distance (LAN and WAN) and migration granularity (single and multiple).
- 3) The technologies for improving VM migration performance are classified and reviewed according to their targeting challenges.
- 4) VM migration is accompanied with overheads and migration performance is also impacted by a variety of factors. The studies on analyzing the procedure of

VM migration to understand the relationship between the influence factors and migration performance are described as well.

- 5) With the evolution of cloud computing (e.g., fog computing [29]) and the increasing application areas of virtualization technology (e.g., NFV), live VM migration encounters some new challenges. The migration mechanisms special for these areas are summarized.
- 6) All migration technologies reviewed in this paper are summarized and compared with the metrics extracted from the literature.
- 7) Finally, the outstanding issues on VM migration which need further optimizations or solutions are discussed.

The remainder of this paper is structured as follows. Section II introduces all the basic knowledge about VM migration. The works on non-live migration are described in Section III. The technologies for solving the three challenges of VM migration are depicted in Sections IV–VI, respectively. In Section VII, the works of understanding the relationship between migration performance and influence factors are described. The solutions for user mobility-induced VM migration are reviewed in Section VIII. The open research topics on live VM migration are discussed in Section IX. Finally, we conclude our work in Section X.

II. BASIC KNOWLEDGE

A. The Cornerstone of Cloud Management

Most of cloud management operations are implemented with the support of VM migration. In this section, we summarize all advantages and use cases of VM migration for intra-cloud and inter-cloud managements.

- *Zero-downtime hardware maintenance* [30], [31]: The servers in a data center may have a high possibility of failure after a long-time running or already failed several times. These servers can be replaced with new ones by moving out all VMs located on them and moving back after replacement. This is applicable for hardware upgrade as well.
- *Load balancing* [12], [32], [33]: Overloaded state not only shortens the lifespan of a server, but also degrades the quality of service (QoS). Meanwhile, servers running with underloaded state result in a waste of energy. Live VM migration ensures that all servers in a data center run evenly under the premise of without QoS decrease. The loads even can be balanced between several geodistributed data centers when VM migration over the Internet is enabled.
- *Server consolidation* [34], [35]: VMs are persistently created and destroyed in a data center. In addition, some of them may be suspended or idle. The VMs will be in a mess if the servers in a data center are not properly consolidated. In server consolidation, VMs are live migrated for either energy purpose (using as fewer servers as possible) or communication purpose (locating the VMs communicating heavily with each other on the same server to reduce network traffic).

- *Across-site management* [36]: For the cloud providers with multiple sites, there are more management options to improve the QoS and lower economical cost. For example, *Follow the sun* [30], [37] is a new IT operation strategy which moves compute resources close to the clients to minimize the network latency. Conversely, in order to reduce the cooling cost, Google and Microsoft propose the concept of *free cooling*, also called *follow the moon* [38], [39], which moves VMs and applications to the data center where it is in nighttime or with low temperature.
- *Hybrid cloud* [40], [41]: Hybrid cloud is featured with a couple of advantages [42]: (1) highly cost-effective and elastic; (2) better aligned with the business demands; (3) keeping private services in local. In hybrid cloud, users can offload some tasks to a public cloud when encountering peak workload, i.e., *cloud bursting* [43].
- *Cloud federation* [44]–[46]: The inherent deficiencies of an individual data center are driving cloud providers to cooperate with each other [47], i.e., *cloud federation* [48]. There still is a long way to go for cloud federation in industry, but it is already making benefits in academia. For example, EGI Federated Cloud [49] connects more than 22 computing sites across Europe and the world to provide services to academic researchers. With cloud federation, when the volume of the processed data is huge, we can choose to move computation environment to where data are located to avoid transferring a big amount of data.
- *Breaking vendor lock-in issue* [50], [51]: The worry of being locked in by one vendor is the main obstacle preventing users from choosing cloud computing. The investigation shows that 62% IT organizations regard moving VMs between data centers as an invaluable feature when choosing a cloud vendor [43].
- *Reaction to user mobility*: To avoid violating Service Level Agreements (SLA), sometimes cloud providers have to migrate VMs to a cloud data center which is close to users. This is also strongly required in the fog computing alike scenarios where each edge cloud data center only covers a small region of users. When a user moves out from the coverage area of an edge cloud data center, the corresponding VMs have to be migrated to the edge cloud data center where the user is currently at.

B. VM Migration

In this section, we firstly summarize the challenges encountered by VM migration, and then design a taxonomy for existing migration schemes. VM migration should move a part or all of the data of a VM from one place to another. Live migration is carried out under the precondition of no interruption to the services running in the migrated VM, while non-live migration does not have this limitation. In total, VM migration faces the following three challenges.

- 1) *Memory data migration*: If the migrated VM wants to continue to run from the suspend point after migration, all running states must be transferred to the target site.

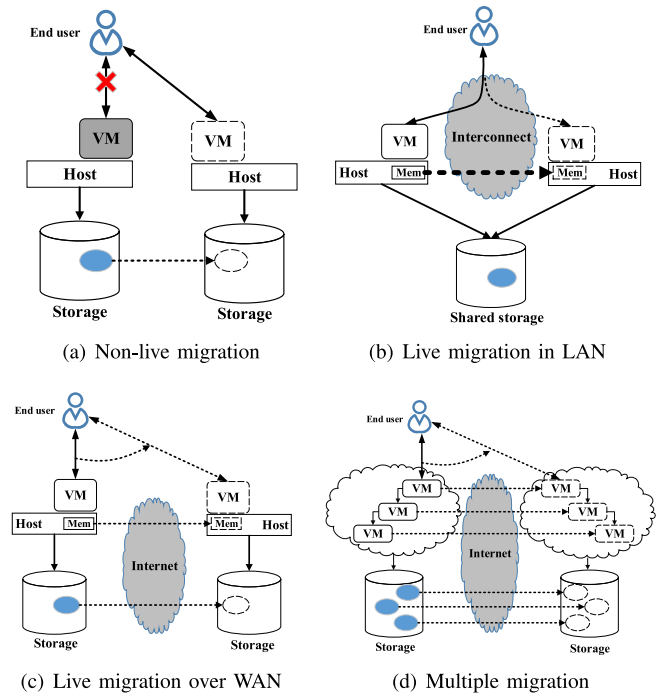


Fig. 1. Classification of migration schemes. The grey box in (a) is to denote that the VM is suspended or shut down on the source host. The bigger line width for memory data migration in (b) is to indicate the bigger network bandwidth of LAN in comparison with WAN. The VMs in a multiple migration can be correlated or independent. (d) only shows the migration for correlated VMs.

The state data contain CPU states, memory data, device states, etc. In general, the transfer of the running states is shorted as *memory data migration*.

- 2) *Storage data migration*: If the storage system of the source data center is not reachable for the target server of VM migration, the virtual disks of the migrated VM will be transferred to the destination site as well, because remotely accessing disk data not only results in a big disk I/O latency, but also may violate SLA.
- 3) *Network connection continuity*: After a VM moves to a new location, some strategies are required to make it reachable to its users. For live migration, the open connections should be also kept alive during migration.

The migrations with different conditions confront different aforementioned challenges. We classify all migration techniques from three perspectives: migration manner, migration distance, and migration granularity.

VM migration can be conducted in two manners: *non-live migration* and *live migration*. With non-live migration, the VM is firstly suspended or shut down before migration depending on whether it wants to continue the running services after migration or not. If the VM is suspended, the running states will be encapsulated and transferred to the target site. During the migration, no open network connection is kept, and all connections are rebuilt after VM resumption, as shown in Fig. 1(a). In live migration, memory data migration and network connection continuity are the two problems which must be solved to avoid service interruption. If the source and the destination sites do not share storage system, storage data

migration must be carried out as well. Obviously, non-live migration has a significant interruption to the service running in the migrated VM. This dramatically restricts its application field since many applications in a cloud data center are running in 7×24 manner. Hence, the majority of studies are focusing on live migration.

According to migration distance, VM migration is divided into two categories: *migration in LAN* and *migration over WAN*. Migrating a VM in LAN means the source and the destination servers are located in the same data center. With the development of network technologies, the difference and boundary between Metropolitan Area Network (MAN) and WAN disappear [24]. *Migration over WAN* in this paper refers to any migration across data centers. The migration mechanisms for LAN environments normally has two basic assumptions. (1) Shared storage system, such as Storage Area Network (SAN) and Network Attached Storage (NAS), is used in the data center. It is accessible from both servers in the migration, which indicates that storage data migration is unnecessary. (2) The source and the destination servers are in the same subnet. The migrated VM will keep its network configurations during the whole migration. Therefore, only an unsolicited ARP can redirect network connections to the new position [19]. Based on these two premises, migrating a VM in LAN only needs to solve the task of memory data migration, as shown in Fig. 1(b). However, migrating a VM in WAN environments does not have these advantages. There is no shared storage system, and different data centers have different network configurations as well. Furthermore, the network conditions (such as bandwidth, latency) between data centers are much worse than those of LAN, as shown in Fig. 1(c). Therefore, migrating a VM over WAN not only needs to solve all of the three challenges, they also become much harder in comparison with LAN migration.

To provide high quality of services to mobile devices, some new computing paradigms were proposed, such as fog computing [29], mobile edge computing (MEC) [52], cloudlet [53], etc. All of them show the same structure, i.e., cloud resources (compute and storage) are deployed at the network edge to provide low-latency services for either user equipments (UEs), such as smartphone and tablet, or the devices in the Internet of Thing (IoT). We use MEC to denote all these paradigms in this paper. Obviously, the migration in MEC belongs to WAN migration since they face the same migration challenges. However, the proximity of cloud resources to users in MEC introduces new requirements to VM migration (more details in Section VIII). For example, an edge cloud data center in MEC only serves the users in its coverage area [54]. When a user roams between different edge cloud data centers, the corresponding VM must be migrated to meet the low-latency requirement of mobile applications. We call this type of VM migration as *user mobility-induced migration*.

Nowadays, many applications in a data center consist of a group of VMs [55], [56]. These VMs are closely related to each other and work together to provide a service. For example, the three-tier application is a typical deployment architecture. It normally is composed of a presentation tier, an application tier, and a database tier. The number of VMs

in each tier can be scaled up or down according to the change of workloads. It is impossible to only migrate one of the correlated VMs to another data center because the long network latency between data centers will severely degrade service performance, even destroy the service, as shown in Fig. 1(d). Within a data center, all of the VMs on a server also may be migrated to another server for hardware maintenance. Therefore, according to migration granularity, VM migration contains *single migration* and *multiple migration*. Single migration migrates one VM each time, while multiple migration simultaneously moves a bunch of VMs.

C. VM Migration vs. Container Migration

Container is an unavoidable topic whenever VM is involved due to many common points between them. Meanwhile, there are many differences between them which make them coexist in the “virtualization world” [57], [58]. In this section, we will differentiate them from the migration perspective.

Containers are implemented by OS virtualization, while VMs are implemented by hardware virtualization. The containers on a host are sharing the underlying OS kernel, but VMs are complete and totally isolated execution environments (each VM installed with an OS). This difference makes container migration more close to process migration. Actually, the commonly used migration technology for containers is checkpoint and restart (CR) [59] which saves the memory state of a process into files and resume the process at the destination host from the checkpoint. A project—CRIU [60], based on CR, has been implemented for container migration.

A container is much more lightweight than a VM, which inherently leads to a smaller challenge for migrating a container than a VM. For example, for the containers which are running stateless services (e.g., RESTful Web services), we can directly kill the containers on the source host and spawn a new one on the destination host. The duration of this operation is tolerable and only the currently running requests will be affected.

Container migration will consider some problems which do not bother VM migration. For example, containers are not only sharing the underlying OS, but also some libraries. Therefore, during container migration, the destination host must prepare these libraries for the migrated container. However, the hosts at the destination site are also running other containers. Therefore, the destination host selection should be an important issue for container migration. In contrast, a VM can run on any host once they are virtualized and managed by the same type of VMM.

D. Performance Metric and Overhead

A good migration strategy not only tries to move a VM from one place to another as fast as possible, but also needs to minimize its side-effects. In this section, we summarize the performance metrics of assessing a migration strategy. Some migration mechanisms only focus on optimizing one aspect, while others perform well on several metrics simultaneously.

- *Total migration time*: This refers to the duration between the time when the migration is initiated and the time when

the migrated VM is resumed on the destination server and no data remains at the source site.

- *Downtime*: It is the duration that the migrated VM is out of service. This metric determines how transparent the migration is to the users of the migrated VM. For non-live migration, total migration time equals to downtime.
- *Total network traffic*: This metric means the total data transferred during the migration. It is a critical measurement when the migrated VM is running a network-intensive service because it will contend for network bandwidth with the migration process.
- *Service degradation*: It indicates how the service running in the migrated VM is affected by the migration. It can be measured by the changes of throughput, response time, etc.

The migration performance also can be evaluated by network bandwidth utilization. This metric can be gained by combining total migration time with total network traffic. The smaller the total migration time and the less the total network traffic are for a specific migration, the higher the network utilization is. We list all benefits we can make from VM migration in Section II-A. However, VM migration is not an only-gain-no-pain operation. It may bring interferences to all involved roles in the migration. They can be divided into three categories: computation overhead, network overhead and space overhead.

Computation overhead: Normally, the migration daemons are running in the VMMs of the source and destination hosts. The migration process will occupy some CPU cycles and memory spaces. This will lead to interferences to all VMs on these two hosts. If the migration daemons are running in the migrated VM [19], [61], some computation resources of this VM will be occupied. Besides, some migration optimization technologies also introduce computation overheads, such as data deduplication, data compression.

Network overhead: VM migration is a network-intensive workload. It will compete for network resources with the VMs running on the source and destination hosts. In addition, migration process reads data from the storage system of the source site and writes them to that of the destination site, which consumes a portion of I/O bandwidth as well.

Space overhead: Compared to other resources (such as CPU cycles, network bandwidth), storage space is less valuable. Inevitably, some migration technologies will implement migration or improve migration performances at the expense of storage space, such as snapshotting [62].

During discussing each migration strategy in this paper, both its advantages and disadvantages will be summarized and compared as well, i.e., which benefits we can make from it and which overheads it introduces.

E. Review Framework

According to the classification in Section II-B, we show VM migration schemes in Fig. 2. Based on it, we review migration technologies with the following framework.

- 1) Firstly, we describe the mechanisms regarding non-live migration (Section III). These mechanisms were

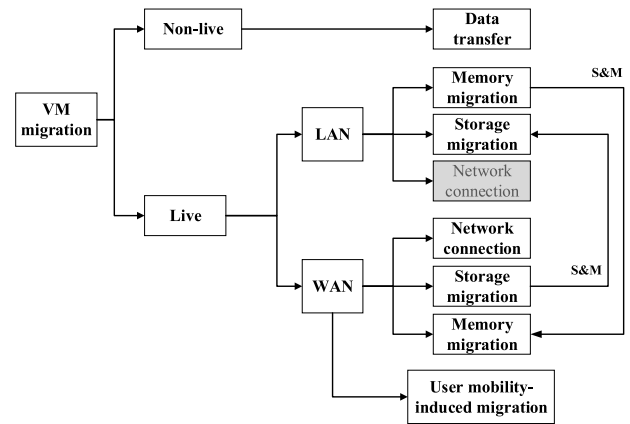


Fig. 2. The review structure on VM migration technologies. The arrows for memory and storage data migration technologies mean that they are mutually compatible in LAN and WAN environments. For both memory and storage data migrations, the mechanisms for single migration and multiple migration are covered. S&M refers to single and multiple migration, respectively.

proposed at the beginning of the advent of VM migration technology. Even though they are not used at present, they pave the way for the following development of live migration technologies and are helpful for us to better understand VM migration in essence.

- 2) Memory data migration is the main task for live VM migration in LAN environments. With the optimization over the past years, its performance almost touches the ceiling. Furthermore, memory data migration technologies for LAN environments are also suitable for WAN environments, only facing a slower migration bandwidth, and vice versa. Therefore, we review all memory data migration technologies together (Section IV).
- 3) Storage data migration is the bottleneck of the migration in WAN environments, and attracts many attentions from researchers. Even though shared storage systems (NAS and SAN) are pervasive in data centers, some data centers still are utilizing local disk to hold VM storage data [63]. With this deployment structure, storage data migration is also needed in LAN environments. Like the technologies for memory data migration, storage data migration technologies for WAN environments are also applicable for LAN environments. We summarize all of them in Section V.
- 4) It is easy to solve network connection problem for the migration in LAN, and we do not cover this part in this survey. The technologies for keeping network connection during and after the migration in WAN environments are reviewed in Section VI.
- 5) VM migration is not an operation which only brings benefits. It also introduces overheads to all involved roles, such as the co-located VMs on the same host and the migrated VM. Meanwhile, a variety of factors will influence on migration performance. To these ends, many researchers try to analyze the procedure of VM migration to better guide the design of optimization strategies. These studies are concluded in Section VII.

- 6) In the new paradigms of cloud computing (e.g., MEC), VM migration is highly related to user mobility. This type of VM migration will not only solve the same challenges as WAN migration, but also faces some new issues, so we review the state-of-the-art of this topic in an individual section (Section VIII).
- 7) At last, the outstanding research issues in the area of VM migration are discussed in Section IX.

III. NON-LIVE VM MIGRATION

The researches on non-live migration only appear at the beginning of cloud computing popularity. These works are mainly for the purpose of user or application mobility rather than data center management. User mobility indicates that a user can stop a work by suspending it on one computer and continue it by resuming the execution environment on another computer, and application mobility denotes that an application is not bound with the underlying OS. However, with the development of cloud computing, non-live migration can not meet the strict SLA requirements any more due to its long service downtime. In this section, we still review some typical non-live migration mechanisms, because they are an important part of the history of VM migration.

Kozuch and Satyanarayanan [64] design a migration system—*ISR*, which aims at the problem of user mobility. *ISR* implements user mobility by combining virtualization technology with distributed file system. It takes advantage of virtualization to encapsulate all user data (running state and disk data) and utilizes a central Network File System (NFS) [65] to provide a persistent storage for resuming VM at any place. *ISR* works in two steps: *suspend event* and *resume event*. During the suspend event, VM states are saved and copied to the central NFS, while the resume event copies the state data from the central NFS to the new location and resumes the VM. When the migration happens between two fixed places, incremental copying can be used to improve migration speed. Whitaker *et al.* [66] try to implement user mobility by designing an extensible and programmable VMM—*μDenali*. It provides the functionality like *ISR* [64] by integrating a checkpoint-restart mechanism with a NFS file system. VM migration is carried out by three phases: checkpointing VM states, transferring the state file to NFS, and unpacking it to a newly created VM on the target server.

Towards the same problem in [64], Sapuntzakis *et al.* [67] design a capsule-based system architecture—*Collective*. They call all the states (disk, memory, CPU registers, and I/O devices) of a running computer as a *capsule*. In order to reduce the data transferred over the network, capsules are stored in a tree hierarchy by adopting Copy-On-Write (COW) technology. A new capsule is created based on an existing capsule, and all writes from the new capsule are stored in a separated file. In further, several other optimizations are designed to speed up the migration of a capsule, such as memory ballooning, on-demand fetching disk blocks, data deduplication and compression.

Different from the VMM-based implementations in [64], [66], and [67], Osman *et al.* [9] focus on migrating a set of

TABLE II
THE SUMMARY OF NON-LIVE MIGRATION TECHNOLOGIES

Reference	Mechanism	Advantage	Drawback
[64], [66]	S/R(C/R),NFS,V	simple	slow
[67]	Capsule,COW MB,ODF DD,DC	fast	high
[9]	POD	fast	complex

The abbreviation list: *S/R*: suspend and resume. *C/R*: checkpoint and restart. *V*: virtualization. *Capsule*: tree-like hierarchy of capsule. *MB*: memory ballooning. *ODF*: on-demand fetching. *DD*: data deduplication. *DC*: data compression. *POD*: process domain abstraction.

processes instead of an entire machine by inserting a layer on an OS, called *Zap* virtualization. *Zap* introduces a Process Domain (pod) abstraction which isolates a set of processes into a private namespace. A pod is a self-contained utility and independent of the underlying OS, so it can be migrated between machines. *Zap* also employs a checkpoint-restart mechanism to implement pod migration. It realizes application mobility on a smaller granularity—process level, which is accompanied by a higher implementation complexity.

We simply summarize the non-live migration technologies and their advantages and drawbacks in TABLE II. From the table, we can see that virtualization also is an important technology for non-live migration. It encapsulates an execution environment or a bunch of processes into a movable package. Then, different mechanisms are utilized to reduce the transferred data (such as using shared storage system (NFS)) and accelerate the transfer speed (such as data deduplication and compression). Overall, these studies open the window to efficient cloud management and are replaced by live VM migration. The combination of suspend/resume and NFS is simple, but it faces a slow migration speed. The capsule-based solution and the POD-based solution can fast migrate an execution environment. However, the tree-like capsule hierarchy will incur a high management cost and process domain abstraction also has a high implementation complexity.

IV. MEMORY DATA MIGRATION

A. Migration Pattern

According to the VM handover time of a migration process, memory data can be migrated in three patterns: *pre-copy*, *post-copy*, and *hybrid-copy*. Pre-copy [19], [68] firstly copies the original memory data to the destination server, and then iteratively transfers newly dirtied pages in rounds. When the iteration meets the predefined thresholds, the VM is shut down on the source server. The remaining data are copied to resume the VM on the destination server. For example, the thresholds for stopping the iteration phase (called *termination conditions*) can be set as: (1) the transfer round reaches a predefined value; (2) the remaining data size is smaller than a predefined value; (3) the ratio between the size of transferred data and the allocated size of memory space is bigger than a predefined value, etc. Because VMs running different workloads have different memory dirtying features, no termination condition is suitable for all VMs. Therefore, several termination conditions are often combined to design a comprehensive strategy to adapt to

as many VMs as possible. Pre-copy is widely used in the mainstream VMMs/hypervisors, such as VMware [68], Xen [19], and KVM [69], because of its robustness. During migration, at least one site (the source site) has all of the data of the migrated VM. When the migration breaks up at halfway due to some reasons (such as network outage) or the VM fails to resume on the destination server, the VM can continue to run on the source server without data loss. The source server releases the data of the migrated VM until it is successfully resumed on the destination server. However, it also faces two main problems. (1) The iterative copying operation not only results in a big network overhead, but also leads to a long migration time. (2) When the memory dirtying rate is bigger than the available network bandwidth to VM migration, the migration process cannot be convergent. It means that the iterative copying makes no sense to reducing the remaining data on the source host. This issue is called *migration convergence problem*, which will result in a big migration downtime or a big network traffic.

Contrary to pre-copy, post-copy [70], [71] firstly stops the execution of a VM on the source host. Boot data are scratched and copied to the destination host to resume the VM. The rest of memory data can be transferred by different manners, such as on-demand fetching, active pushing, and prepaging [70]. All memory pages are copied only once in post-copy, so the total migration time is predictable. However, post-copy has a fatal issue. The latest data of the migrated VM are separated to the source host and the destination host, therefore, it may lose data or even destroy the migrated VM when the migration fails at halfway.

Hybrid-copy [72], [73] is a combination of pre-copy and post-copy. It firstly copies memory data with a limited round of iterations, and then hands over the execution of the VM to the destination server. The remaining memory pages are copied in a post-copy manner. Hybrid-copy inherits the strong points from both pre-copy and post-copy. The limited pre-copying iteration reduces network traffic. The majority of memory pages are transferred in the pre-copy phase, which decreases the pages remotely accessed by the destination host, i.e., minimizes the possibility of page faults at the destination site. This is helpful for lowering service degradation. However, it also succeeds the main disadvantage of post-copy, i.e., weak robustness.

For better understanding memory migration patterns, Clark *et al.* [19] divides VM migration into three basic phases:

- *Push phase*: The migrated VM runs on the source server. Its memory data are iteratively transferred to the destination server.
- *Stop-and-Copy phase*: The execution of the VM is halted on the source host. Related data are copied to the destination site to resume the VM.
- *Pull phase*: When the VM is running on the destination server and page faults happen, it remotely fetches these pages from the source server. At the same time, the remaining pages are proactively sent to the destination host to lower the possibility of page fault.

Different migration patterns are achieved by taking one or several phases of this division, as shown in Fig. 3. Pre-copy takes

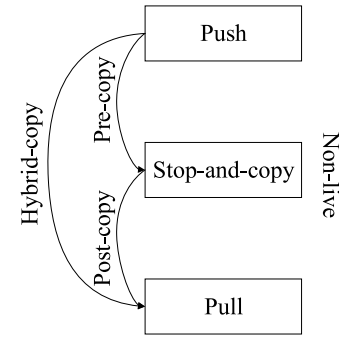


Fig. 3. The classification of memory data migration patterns.

TABLE III
THE COMPARISON BETWEEN PRE-COPY, POST-COPY,
AND HYBRID-COPY

Pattern	TT	DT	TN	PT	R
Pre-copy	big	big	big	N	strong
Post-copy	small	small	small	Y	weak
Hybrid-copy	medium	medium	medium	Y	weak

The abbreviation list: *TT*: total migration time. *DT*: downtime. *TN*: total network traffic. *PT*: the predictability of migration time. *R*: robustness.

the push and stop-and-copy phases, while post-copy takes the stop-and-copy and pull phases. But they transfer different data during the stop-and-copy phase. Pre-copy transfers all the remaining data at the source host, while post-copy only transfers the data needed for VM resumption. Hybrid-copy experiences all of these phases. If only the stop-and-copy phase is taken, it is non-live migration.

The comparison between the three migration patterns is shown in TABLE III. We only compare them from all aspects except service degradation which depends on many factors (such as, VM workloads, migration bandwidth). Even though post-copy and hybrid-copy are better than pre-copy in most of the metrics, they are still rarely used by cloud management platforms, which illustrates that migration robustness overweighs all other migration performance metrics, such as total migration time, total network traffic.

B. Pre-Copy

Pre-copy migration mechanism can be divided into the following six stages [19]:

- 1) *Stage 0: Pre-Migration*. A target server with enough resources is selected to receive the migrated VM.
- 2) *Stage 1: Reservation*. A VM position with the same size as the source VM is reserved on the target server.
- 3) *Stage 2: Iterative Pre-Copy*. Memory pages are iteratively transferred to the target server.
- 4) *Stage 3: Stop-and-Copy*. When the iteration meets one of termination conditions, the VM is suspended on the source server. Then all remaining data are copied to the target server. Now, both the source server and target server have identical copies of the migrated VM. Whenever a migration failure happens, the VM continues to run on the source server.
- 5) *Stage 4: Commitment*. The destination server informs the source server that it receives all data of the migrated

VM. Now the source server can reclaim the resources occupied by the migrated VM.

- 6) *Stage 5: Activation.* The VM is resumed on the target server. The external devices and network connections are recovered.

Clark *et al.* [19] further implement the migration mechanism in two manners: *managed migration* and *self migration*. Managed migration runs the migration daemon in the management VM (*Dom0*), while self migration is conducted by the migrated VM itself. Self migration is more complex than managed migration in terms of implementation. Because the migration process is running with other processes on the migrated VM in self migration, it must solve the migration consistency issue which is out of consideration for managed migration. They design a two-stage stop-and-copy phase to guarantee migration consistency. The first stage stops all processes except the migration process, and scans the dirtied pages. The second stage transfers all dirtied pages in the final scan. Due to the implementation complexity and intrusive deployment for each VM, self migration is rarely used for cloud management.

Pre-copy is also used by VMware to carry out its migration system—*VMotion* [68], and integrates it into the VirtualCenter management platform. It sets termination conditions for the iterative copying as: (1) less than 16MB modified pages left; (2) 1MB size decrease of modified pages between two neighbor rounds. They find that scanning dirtied memory pages during the iterative copying phase takes 20% network throughput drops to end users. Their results also show that reserving 30% CPU resource for migrating an idle 512MB Windows 2000 Server VM over a gigabit link minimizes the total migration time.

As discussed in Section IV-A, pre-copy mechanism faces many problems, such as migration convergence. In the rest of this section, we will summarize all of the technologies aiming at optimizing the performances of pre-copy.

1) *Compression:* Memory pages have strong regularities and contain many zero bytes as well [74], [75]. To reduce the total network traffic during migration, compression technology is the first one comes to mind. As shown in Fig. 4, there are two types of compression manners: *normal compression* and *delta compression*. Normal compression takes advantage of data regularities to encode information with fewer bits. The ratio between the size of the representation information and the original data is called compression ratio. Normal compression contains two interleaved phases: modeling and encoding [76]. The modeling phase is to find data regularities, and the encoding phase constructs the representation information. Delta compression reduces data transfer by only sending the difference between current version and its previous version. The difference is called *delta*, while the previous version is called *reference*. In decompression phase, the original data are gained by applying a delta to its reference. After each compression and decompression, the reference is replaced with the new version of data for the next round of delta compression. It is obvious that delta compression needs space to store the references and introduces additional management efforts for it. Although normal compression does not encounter these disadvantages, it is compute-intensive.

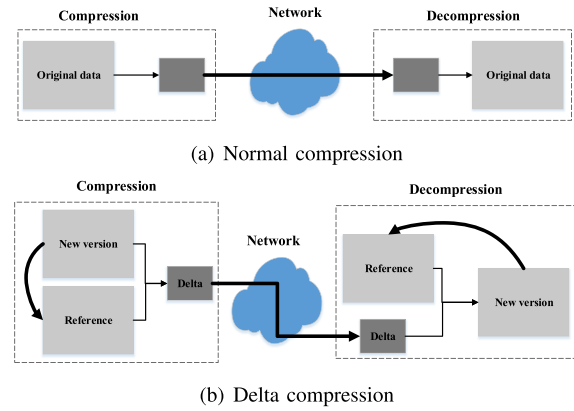


Fig. 4. The illustration of normal compression and delta compression.

However, compression has to make a trade-off between computational cost and migration performance benefits. A compression algorithm with a higher compression ratio will lead to bigger computational cost and compression time. To this end, Jin *et al.* [77], [78] propose a *Characteristic-Based Compression* (CBC) algorithm which adaptively chooses compression methods according to data regularities. Memory pages are classified into three categories according to their regularity characteristics: 1) pages with many zero bytes; 2) pages with high similarity; 3) pages with low similarity. The pages with high similarity are compressed by an algorithm with fast compression speed, such as WKdm [74], while the pages with low similarity are compressed by an algorithm with high compression ratio, such as LZ0 [79]. For the first type of pages, only the offsets of non-zero bytes are recorded. The threshold between low similarity and high similarity is also adjustable to adapt to a variety of VMs with different memory dirtying rates. Multi-threaded techniques are employed to accelerate the speed of compression and decompression as well.

Hacking and Hudzia [80] propose to use delta compression to shorten the migration duration of a VM running large enterprise applications which are featured by large memory size (can reach tens of GB) and fast memory dirtying rate. They introduce a warmup phase before migration to transfer the original memory data to the destination server in the background. During migration, delta compression is conducted to reduce the network traffic of migration. VMM keeps the previous version of recently dirtied pages in an Adaptive Replacement Cache (ARC) [81]. Before transferring a dirty page, if its previous version is stored in the ARC, only the difference (*delta*) between them is transferred to the target server, and at the same time the ARC is updated to the current version. Otherwise, the dirty page is directly sent to the target server. They also show the capability of their migration method in WAN environments.

Based on the work in [80], Svärd *et al.* [82] propose some new optimizations. They use a two-way associative caching scheme [83] instead of ARC to store the reference pages. This caching scheme is lightweight (small CPU occupation) and has a constant lookup time. XORed Binary Run Length Encoding (XBRLE) compression algorithm [84] is used to compress

pages. Their experimental results indicate that their system outperforms the default KVM migration algorithm regarding both total migration time and downtime. However, the performance improvement strongly depends on the cache size. A bigger cache leads to a high compression ratio, and also results in a higher management cost. For example, in their experiments, 512MB cache and 1GB cache are used for delta compression for a VM with 1GB RAM and a VM with 8GB RAM, respectively.

2) *Data Deduplication*: Many previous works [85]–[87] prove that a big amount of identical memory pages exist within a VM or between VMs. These duplicate pages can be eliminated during VM migration. They are partly zero pages and partly result from using the same libraries or applications. There are three types of similarities existing in VM memory pages: *intra-VM similarity*, *inter-VM similarity*, *inter-site similarity*. Intra-VM similarity denotes the duplicate pages within the migrated VM. Inter-VM similarity refers to the identical pages between different VMs at the same data center. This similarity can be used to transfer identical pages only once when multiple VMs are migrated concurrently. Inter-site similarity explores the identical pages between the migrated VM and the VMs located at the destination data center. SHA-1 [88] and SuperFastHash [89] are the two commonly used hashing algorithms to locate duplicate pages. In this section, we review the studies on exploiting intra-VM similarity and inter-site similarity for VM migration, and these on inter-VM similarity are described in Section IV-B7 on multiple migration.

Riteau *et al.* [90], [91] design a migration system—*Shrinker*, to improve the performance of migrating memory data over WAN. It utilizes distributed content-based addressing to avoid transferring duplicate pages between the migrated VM and the VMs running at the destination site (i.e., inter-site similarity). However, VM memory pages change over time, so a dynamic indexing approach is needed. Shrinker solves this problem with two subsystems: (1) a site-wide distributed hash table (DHT) and (2) a periodic memory indexer. The DHT is built on a peer-to-peer (P2P) network [92] which is formed by all hosts at the destination site. Each host is a node in the P2P network ring. The hash digests are kept in the DHT. VM hypervisors periodically scan their memory and update the items in DHT. When a VM will be migrated to another site, firstly the fingerprints of its memory pages are sent to the destination site. Then the duplicate pages are eliminated by comparing the fingerprints with DHT. At last, the source host only transfers the pages which are not located at the destination site. The intra-VM similarity feature is also exploited by them: only the first byte of zero pages is sent to the destination site.

Nowadays, cloud providers pre-deploy template VM images for fast VM creation. Zhang *et al.* [93] find that many redundant memory blocks are located between the VMs which are cloned from the same VM template by an extensive experiment. To utilize this feature and decrease the footprint size of the VMs on the same host, Content Based Page Share (CBPS) [87] is widely used in virtualization platforms (such as VMware ESX [94], Xen [95], [96]) to make the VMs on the same physical server share memory pages [97]. Based

on these observations, Zhang *et al.* design a metadata based migration system—*Mvmotion*, which makes the migrated VMs share some redundant memory pages with the VMs running on the destination host (i.e., inter-site similarity) by utilizing CBPS technology. The metadata of a VM contain the hashing values and the block numbers of memory pages. During migration, the metadata of the migrated VM is sent to the destination data center to find the pages already existing at the destination host. However, because the memory pages are dynamically dirtied, the metadata maintenance and the data consistency issue in [90], [91], and [93] will introduce a big management cost to the involved VMMs and data centers.

Jo *et al.* [98] accelerate VM migration by utilizing shared disk blocks rather than memory pages. They observe that many memory pages of a VM are replicas of disk blocks [99]. They propose to only transfer unique memory pages from the source server to the destination server. The information of the memory pages which are the replicas of disk blocks is logged into a list. The destination server gets these memory pages from the shared storage system instead of from the source server. However, it is infeasible to find the duplicate pages during migration because the big size of storage data will result in a long comparison time. Therefore, they have to maintain the list of duplicated pages all the time for future potential migration. This will introduce overheads to both the hypervisor and the guest VMs.

Li *et al.* [100] and Zheng and Hu [101] propose a template-based migration mechanism. If a page appears n times in a data center which is bigger than the preset threshold, it is called a *template page*. Similar with [90] and [91], the fingerprints of the destination data center's template pages are stored in a Distributed Hash Table (DHT). They classify memory pages into three categories: *uniform pages*, *normal pages*, and *duplicate pages*. All bytes of uniform pages are identical, while a duplicate page is same with one of the template pages in DHT. Others are normal pages. In their migration system, uniform and normal pages are transferred by using the default migration interface of VMM. Duplicate pages are constructed at the destination data center by copying its identical template pages. Multi-threading is utilized to accelerate the transfer of duplicate pages. However, they do not evaluate the impacts of different thresholds of defining a template page on migration performance.

There are also some other studies which utilize data deduplication to accelerate VM migration. Zhang *et al.* [86] observe that at least 30% of non-zero memory pages are identical or similar, and design a new migration strategy—*Migration with Data Deduplication (MDD)* which takes advantage of intra-VM similarity. Both data deduplication and delta compression are utilized by Wood *et al.* [37] to accelerate memory data and storage data migration over WAN. Data deduplication is used to explore the duplicate items for memory pages and disk blocks, and delta compression aims to reduce the transferred bites when a page has been copied before.

Many “free” pages (such as zero pages, cache pages) exist in a VM. These pages will not influence on the correctness after the migrated VM is handed over to the destination

server. Clark *et al.* [19] do not transfer these pages during the first full-transfer round by using memory ballooning mechanism [102]. This mechanism is also combined with QuickAssist Technology (QAT) data compression [103] by Zhang *et al.* [104] to accelerate VM migration in the scenario of the network function virtualization (NFV). Koto *et al.* [105] run a process in the migrated VM to record the pages which are unnecessary for VM correctness after migration. These pages will not be transferred during migration and are reproduced after VM resumption. However, even though these pages are not important for the correctness of VM execution, losing and reconstructing these pages will result in a big service degradation after migration.

3) *RDMA*: Many high-speed interconnect technologies, such as InfiniBand [106], Myrinet [107], provide the functionality of Remote Direct Memory Access (RDMA). RDMA allows memory data to be remotely accessed without the involvement of CPU and cache. Huang *et al.* [108] take advantage of this feature of InfiniBand to minimize the side effects of VM migration and improve migration performance. To fully exploit the benefits of RDMA for VM migration, they design the migration mechanism from the following aspects.

Migration protocol: Memory pages are divided into normal pages and page table pages. Different from normal pages, page table pages must be pre-processed before sending to the destination server. Therefore, only normal pages can be transferred by RDMA, and page table pages are copied with normal send/receive model. Because the page table size is small, the majority of data still pass through RDMA. In addition, both RDMA read and RDMA write can be employed for VM migration. RDMA read moves the burden to the destination server, while RDMA write keeps that on the source server. Towards this end, the server which has fewer workloads is selected to carry out memory data migration.

Memory registration: There are two options for registering the buffers required by RDMA [109]: *copy-based approach* and *zero-copy approach*. Copy-based approach uses pre-registered buffers for data transfer, while zero-copy approach registers the buffers on the fly. Neither of these approaches can be directly used for VM migration. Since InfiniBand can transfer data by directly using hardware Direct Memory Access (DMA) addresses in kernel space, they export this functionality to the user space for Xen migration process to solve the registration problem.

Non-contiguous transfer: Normal memory migration manners transfer memory data in page granularity. This is tolerable for TCP transfer, but it will result in a low bandwidth utilization for InfiniBand RDMA. Thereby, page clustering is employed by rearranging the mapping table of memory pages to improve the possibility of transferring more contiguous pages in each RDMA operation.

Network QoS: To efficiently exploit the available bandwidth for migration, they revise the dynamic rate-limiting in Xen [19]. They start the network bandwidth for RDMA at the maximum bandwidth limit, and decrease it when detecting network traffic from other applications by controlling the issuance of RDMA operations.

According to their experimental results, service degradation during migration is reduced by up to 70%. It also gets 80% and 77% migration performance improvement regarding total migration time and downtime, respectively.

Ibrahim *et al.* [110] also utilize InfiniBand RDMA to migrate the VMs with High Performance Computing (HPC) applications. They comprehensively investigate the performance relationships between VM migration and the applications running in the migrated VM. A series of findings are observed in their experimental results. (1) The monitoring mechanism of migration process introduces a considerable interruption to the workloads running in the migrated VM. The more cores the VM runs with, the bigger the interruption is. They observe that parallelism of the monitoring process is beneficial to migration performance. (2) The memory pages of HPC applications are easily dirtied faster than the available migration bandwidth. Hence, normal migration termination conditions (predefined target downtime and iteration limit) will result in a sub-optimal migration performance. (3) The Writable Working Set (WWS) (the set of frequently dirtied memory pages) varies significantly when a VM is running different workloads. They further evaluate the performance of VM migration when a dedicated network path (InfiniBand RDMA) is employed for migration. They find that: (1) Migration downtime depends on the start time of the migration over the application lifetime. A bigger application dataset or more processors used by the migrated VM leads to a longer downtime. (2) Even though a dedicated migration path is provided, the migration still severely impacts on the quality of service. (3) Multiple migration will experience a longer downtime than single migration. Based on these observations, they propose an optimized migration termination strategy for HPC VMs (see Section IV-B6).

4) *Checkpointing/Recovery and Trace/Replay*: Normal pre-copy scheme is sensitive to memory dirtying rate and also results in a big network traffic. Considering this issue, Liu *et al.* [111], [112] design and implement a novel migration system—*CR/TR-Motion*, by utilizing checkpointing/recovery and trace/replay (CR/TR) technology. It is based on a full system trace and replay system—*Revirt,dunlap2002revirt*. They record the execution trace of the migrated VM into log files, and iteratively transfer log files rather than dirty pages to the destination server where log files are replayed to recover VM states. This can improve migration performance due to the fact that log file size (growth rate around 0.04GB to 1.2GB per day) is much smaller than the size of dirty memory pages. Furthermore, migration downtime is also decreased because fewer data is left in the final stop-and-copy phase. However, CR/TR-Motion also faces two challenges: (1) I/O consistency. Due to using shared storage system between the source server and the destination server, during the replaying phase, the read operations may get a wrong data (Write-After-Read (WAR) hazard) and the write operations will change the disk blocks again which are already written by the source server (Write-After-Write (WAW) hazard). (2) Tracking the execution of the VM with multiple VCPUs is complicated. For the first issue, they record the data read from disk within the log files to prohibit the I/O operations of the replaying phase. For the second

problem, VCPU *hotplug* is adopted to configure the migrated VM to use only one VCPU during migration and reconfigure the number back after migration. Obviously, this operation will incur a dramatic service degradation. Experimental results show that CR/TR-Motion improves migration performance by 72.4%, 31.5% and 95.9% on downtime, total migration time and total network traffic, respectively, compared to Xen's default migration approach.

Cully *et al.* [114] also utilize checkpointing to migrate VM memory data to another host by copying the whole system state rather than only replaying inputs deterministically. They repeat the final stage (the stop-and-copy phase) of pre-copy to transfer the latest memory states to the destination host. To increase the checkpointing speed, two optimizations are conducted on the default Xen live migration mechanism: reducing the number of inter-process requests required to suspend and resume the guest domain and entirely removing *xenstore* from the suspend/resume process. With these optimizations, the checkpoint frequency can reach 40 times per second.

5) *Page Reordering*: The memory pages of a VM have different access characteristics. Some pages remain clear during the whole lifetime of the VM, while some are frequently modified. This characteristic can be used to improve migration performance by reordering the pages for transfer in each iteration. Svard *et al.* [115] design a mechanism—*dynamic page transfer reordering*, to lower page retransmission during the iterative copying phase. They assign a weight for each page according to its update frequency. Pages are transferred in the order of increasing weight. The most frequently updated pages are postponed to the final stop-and-copy phase.

Similarly, Checconi *et al.* [116] propose two page-reordering mechanisms: a Least Recently Used (LRU) based approach and a frequency-based approach. LRU-based approach prioritizes the transfer of the pages which are least recently used, while frequency-based approach transfers pages in the order of increasing access frequency.

6) *Migration Convergence*: The main risk for pre-copy is the migration process cannot converge to an optimal point for the final stop-and-copy phase. This situation happens when the VM dirties its memory pages faster than the migration bandwidth. A plenty of optimization strategies are designed to migrate the VM which has a fast memory dirtying rate. They solve the migration convergence problem from two aspects: *tuning memory dirtying rate* and *changing migration termination conditions*.

Clark *et al.* [19] find that some memory pages are dirtied very frequently, i.e., WWS. It is unnecessary to iteratively copy WWS to the target site during migration. Therefore, during the iterative copying phase, the dirtied pages transferred in each iteration are selected like this: only those dirtied in the previous round and not dirtied in the current round again. The paravirtualization feature of Xen also provides some optimization opportunities. A monitoring thread is awakened in the migrated VM when migration begins to stun the rogue processes for migration convergence. It can record the WWS of each process in the migrated VM, and limits the maximum page faults for each process. In further, to make a

trade-off between migration convergence and network bandwidth saving, a dynamic rate-limiting approach is designed by them to control migration bandwidth. Because memory dirtying rate changes over time, setting a static network bandwidth for migration is not always optimal. A minimum bandwidth limit and a maximum bandwidth limit are predefined in their approach. Migration starts with the minimum bandwidth, and increases it with a constant increment each round. When the bandwidth reaches the maximum value or the remaining data size is smaller than 256KB, the migration enters the stop-and-copy phase. The upper bandwidth limit is used for the stop-and-copy phase to lower migration downtime.

Jin *et al.* [117] find that memory dirtying rate approximately has a linear relationship with VM execution speed, based on several practical experiments. Assuming a VM runs a specific program, the faster the VCPU frequency is, the faster the I/O (especially write operation) speed is. Therefore, they propose to tune memory dirtying rate to a desirable value to lower migration downtime by tuning VCPU execution frequency. Nevertheless, this approach will severely interrupt the performance of the applications running in the VM. Hence their main target applications are those which can tolerate moderate performance degradation. For example, reducing rendering frame rate of game applications will not lead to a big service interruption. Liu *et al.* [118] also control the CPU resources assigned to the migrated VM to improve migration performance by using the Credit algorithm of Linux kernel. Mashtizadeh *et al.* [20] design a similar mechanism—*Stun During Page Send (SDPS)*. It does not tune the frequency of a whole VCPU, and only injects delays to page writes to lower page dirtying rate.

Ibrahim *et al.* [110] aim to migrate the VMs with HPC applications (such as MPI and OpenMP applications). They propose to switch the iterative copying to the final stop-and-copy phase when no further reduction in downtime is achievable. They define three memory update patterns: (1) iterative copying does not reduce the amount of dirtied pages; (2) the number of dirtied page decreases for a short duration (such as, synchronization and barrier operations) so that a small downtime can be achieved; (3) most of transmitted pages are duplicate pages. For the first pattern, when a stable memory modification rate is detected, the migration will step into the stop-and-copy phase. For the second pattern, the iterative copying is stopped when the dirtied pages are transferred within a preset interval. For the third pattern, the retransmission rate is monitored. When it exceeds 90%, the migration process moves to the stop-and-copy phase. Atif and Strazdins [119] propose a more rigid termination strategy. According to their statistics on Xen's default migration mechanism for HPC applications, iteratively copying the memory data of HPC applications is only a waste of time and CPU cycles. Therefore, they only iterate the pre-copy phase twice. The first iteration copies all memory pages, and the second iteration directly enters into the final stop-and-copy phase. This is to decrease total migration time and total network traffic at the expense of service degradation. In CloudNet [37], it firstly detects the point where the amount of transferred pages is equal or bigger than the dirtied pages. After this point, it

stops the execution of the VM when the number of dirtied pages is smaller than any previous iteration.

7) *Multiple Migration*: Multiple migration faces some new challenges in comparison with single migration. For example, a migration strategy has to take the influence on the communication between the migrated VMs into consideration if they are correlated. In this section, we review the works on optimizing the performances of multiple migration. Deshpande *et al.* [120] name the migration of multiple co-located active VMs (on a same server) as *live gang migration*. They employ both data deduplication and delta compression to eliminate the duplicate memory pages exist between the co-located VMs. At the beginning of migration, all memory pages are transferred to the destination site. Only one copy of identical pages are transferred. The iterative copying phase only sends the page identifiers of newly-found duplicate pages to the destination server. They find that even though two pages are different, they still may be partially identical. Therefore, they continue to use delta compression to reduce total network traffic. All duplicate pages are acting as reference pages, and unique pages (no duplicate) are compared with them to generate *deltas*. When the delta size of a page is smaller than a threshold, the delta is transferred to the destination site, otherwise, the entire page is transferred.

Deshpande *et al.* [121], [122] further propose a new migration mechanism—*gang migration using global (cluster-wide) deduplication* (GMDG), by expanding the migration approach for the VMs on a single host [120] to a server rack which holds many hosts. The GMDG works as: (1) all duplicate pages between the VMs on a rack are identified before migration, (2) only one copy of these duplicate pages are transferred to each target rack, (3) at the target racks, once a server received a duplicate page, it populates this page to other servers in the same rack which need this page instead of fetching it from the source rack. Data deduplication is also applied between the VMs on the source rack and the target rack.

Live VM migration not only imports interferences to both the source server and the destination server, but also to the VMs running on these two servers. Xu *et al.* [17] extensively analyze migration interference and co-location interference during and after migration. Migration interference refers to the service degradation of the VMs located on the source and the destination servers during migration, while co-location interference denotes the performance losses of the VMs on the destination server after new VMs are migrated in. They created performance models for these two interferences. Based on these models, they propose an interference-aware migration strategy—*iAware*, to minimize both migration interference and co-location interference. For each migration, *iAware* firstly chooses the migrated VM(s) with the least migration interference, and then chooses the target host(s) by estimating the co-location interference. *iAware* is lightweight and can be used as a complementary manner for other migration strategies. Even though *iAware* also is suitable for single migration, its benefits are thoroughly exploited when many VMs are simultaneously migrated, such as in the scenarios of load balancing and server consolidation.

Bari *et al.* [123] try to solve the similar problem but without the selection phase of the migrated VMs and the target servers. They aim at the migration sequence problem to minimize total migration time and downtime when the initial and target VM placements are given. The key insight of their algorithm is to separate the migrated VMs into Resource Independent Groups (RIGs). The VMs in the same RIG will be migrated between distinct machines. In other words, at any time a server is only running one migration process to reduce the network contention and service interruption to other VMs running on it. Therefore, the VMs in the same RIG can be migrated simultaneously, and RIGs are migrated sequentially. For each RIG, the VM which has the shortest migration time will be firstly migrated.

Multi-tier applications are ubiquitous in cloud data centers. The VMs in a multi-tier application are normally communication-dependent. Some works try to optimize the migration for multiple correlated VMs from different perspectives. Wang *et al.* [124] make efforts on choosing an optimal migration sequence and a proper migration bandwidth for multiple migration. The migration sequence is derived by collecting a variety of information, such as network topology and traffic matrix of the data center, memory sizes and memory dirtying rates of VMs. Sarker and Tang [125] design a dynamic bandwidth adaptation strategy to minimize the total migration time for a given number of VMs. The total migration time is controlled by adaptively choosing sequential and parallel migration and changing migration bandwidth.

Liu and He [126] design an adaptive network bandwidth allocation algorithm to reduce the service interruption of live migrating multi-tier application over WAN. They migrate the correlated VMs concurrently and design a synchronization algorithm to make different migration processes finish at an approximate time. Their migration system consists of a central arbitrator and a migration daemon running in the Dom0 of Xen. The arbitrator dynamically tunes the network bandwidth for each migration process by collecting information from the migration daemon. Moreover, a *wait-and-copy* phase is introduced to synchronize different migration processes to start the final stop-and-copy phase at the same time.

In order to fully utilize network bandwidth to transfer as many VMs as possible in a given period of time, such as for the disaster recovery scenario, Kang *et al.* [127], [128] propose a feedback-based migration system. It adaptively changes the number of VMs in a migration by drawing experiences from the TCP's congestion control algorithm [129]. A controller starts the migration from a slow start (SS) phase where a small number of VMs (called *VM window*) are migrated in parallel, and increases the size of VM window gradually. When network congestion is detected, the migration enters a congestion avoidance (CA) phase where the VM window is reduced accordingly.

Ye *et al.* [130] propose to reserve resources (CPU cycles and memory space) for migration. At the target host, several empty VMs with 100% CPU utilization and certain memory spaces are created before VM migration. When the migration starts, these reserved VMs are shut down to leave resources to the migrated VMs. They find that parallel migration is better

than sequential migration when enough resources are available for migration, otherwise, parallel migration is worse. Based on their experimental results, several optimizations are designed. For example, they firstly migrate the VM with small memory size to increase migration efficiency. However, the resource reservation operation increases the migration interference to the destination server.

8) *Others*: Besides the common problems discussed above, some migrations conditions require special migration strategies. Liu *et al.* [131] struggle for the VMM heterogeneity problem, i.e., implementing VM migration between different VMMs. To smooth the heterogeneity of different VMMs, they design a common migration protocol and a common virtual machine abstraction method. The commands and data sent by the source VMM are transformed into intermediate formats which are then intercepted into the formats of the VMM at the target site. Based on this proposal, they implement VM migration between KVM and Xen.

Nathan *et al.* [132] comprehensively compare the performances of non-adaptive migration and adaptive migration with different parameter values, such as VM size and page dirtying rate. Non-adaptive migration technique migrates a VM at the maximum available bandwidth, while adaptive migration technique changes the migration bandwidth according to memory dirtying rate. They find that non-adaptive migration is better than adaptive migration in most scenarios regarding migration performances (such as total migration time, downtime, total network traffic). However, adaptive migration utilizes fewer resources (CPU, network bandwidth) than non-adaptive migration. Based on these benefits of adaptive and non-adaptive migrations, they propose a novel migration technique—*Improved Live Migration (ILM)*. The key idea behind ILM is to use non-adaptive migration but with limited resources (network bandwidth and Dom0 CPU cycles) for migration.

Raghunath and Annappa [133] make efforts to lower the overhead of VM migration by choosing an appropriate migration triggering point. This is implemented by combining predicted future workloads and migration parameters. The migration-triggering system consists of two components: one is a centralized controller which runs as a resource usage collector, and one is running in the Dom0 of every physical machine. Whenever the central controller detects hotspot problem, it coordinates VM migration tasks according to the resource usage statistics (the utilizations of CPU, memory and network bandwidth) gathered from all individual servers. Baruchi *et al.* [134] also try to find an appropriate migration triggering point by exploring application resource consumption features. The execution history data of the applications on a VM are analysed with Fast Fourier Transformation (FFT) which is used to identify cyclic patterns in natural events to estimate the cycle size of the applications. Within each cycle, they then find the moment which is suitable for starting VM migration with the prediction of migration cost.

Mann *et al.* [135] firstly crate a migration cost model for pre-copy which is used to predict the bandwidth required to finish a migration within a specific time window. Based on this model, the available target hosts for VM migration are

ranked. The finally selected target host will satisfy both the constraint of migration time and the minimal interference to other flows in the network.

Xia *et al.* [136] firstly use linear programming to formulate the path selection and bandwidth assignment problem when multiple VMs will be migrated from different source hosts to different destination hosts in the NFV scenario. Two approaches are proposed to solve this problem: critical edge pre-allocating approach and backtracking approach. The critical edge pre-allocating approach assign bandwidth to each migration process according to the available bandwidth of the edge all migration will pass through. The backtracking approach is a greedy strategy which initially assigns the network bandwidth according to the memory size of the migrated VM and decreases it when network congestion happens.

C. Post-Copy

Post-copy firstly hands over the VM to the destination site. Therefore, the optimizations for it mainly focus on reducing the possibility of page fault after VM handover. In other words, they are to avoid remotely accessing memory pages from the source site when the VM is resumed on the destination host. Hines *et al.* [70], [137] holistically describe the process of post-copy. To reduce the page faults at the destination site and total migration time, they design four optimization mechanisms to accelerate the transfer of memory pages: demand-paging, active pushing, prepaging, and dynamic self-ballooning (DSB). With demand-paging, when page faults happen after the VM is running on the target server, it fetches these pages from the source server over the network. This access manner will result in a big service degradation. To reduce the possibility of page fault, memory pages are proactively copied by using active pushing and prepaging. Active pushing continuously copies memory pages to the target host in the background. Prepaging is based on the spatial and temporal localities of memory access. Every time when the source host receives a page fault from the destination host, it not only transfers this page, but also the pages surrounding it, to the destination site. DSB aims to avoid transferring free memory pages of the VM. To increase the robustness of post-copy, a periodic incremental checkpointing is suggested to synchronize the updated states back to the source host in case of migration failure. However, this not only neutralizes the advantages of post-copy (such as, transferring all pages only once) in comparison with pre-copy, but also introduces new overheads to the destination server.

Sahni and Varma [138] first iteratively scan the page table to identify the WWS of the migrated VM. The WWS is sent with the running states for VM resumption to the destination host to reduce the possibility of page faults. On-demand fetching, active pushing, prepaging and compression are combined to quickly cut down the dependency on the source host.

Resource overcommitment used to be an attractive point for cloud providers. However, Hirofuchi *et al.* [71], [139] find that it is rarely utilized by cloud providers in practice. One reason is that pre-copy is widely adopted by VMMs for VM

migration. To implement resource overcommitment, the idle and unused VMs in a data center must be consolidated on as fewer servers as possible to spare resources to accommodate more VMs. To achieve a high QoS, these VM must be quickly changed to a new location when they becomes active or are consuming more resources. However, pre-copy can not meet this requirement due to its long handover time. Hirofuchi *et al.* propose to utilize post-copy for the instantaneous relocation of VMs for the overcommitment purpose. Two optimizations are used to lower page failures at the destination site: pre-paging and active pushing. In pre-paging, the neighbor 128 pages are copied for a page fault. Their mechanism can relocate a heavily-loaded VM within one second.

To avoid the migration convergence problem of pre-copy, Shribman and Hudzia [140] propose to employ post-copy to migrate the VM with a high memory dirtying rate. They design several optimizations to lower the service degradation resulting from remotely accessing memory pages: RDMA, Pre-paging and Linux Memory Management Unit (MMU) Integration. MMU Integration uses OS management tool to only pause the threads in the destination host which are waiting for memory pages from the source server and continue to run other threads.

D. Hybrid Copy

Different hybrid-copy strategies can be gained by combining pre-copy with post-copy in different manners. Hu *et al.* [72] firstly transfer all memory data to the destination server. During this period, the newly dirtied pages are logged in a bitmap. Then the bitmap and the running states are sent to the destination server to resume the VM. The dirtied pages are on-demand fetched from the source server according to the bitmap. They further utilize delta compression to lower network traffic during the transfer of dirtied memory pages.

Kim *et al.* [73] list out the weaknesses of both pre-copy and post-copy migration schemes. To this end, they design a novel migration mechanism—*guide-copy*. It works like: firstly, the running states of the VM are copied to the destination server to resume the VM there; at the same time, the VM continues to run on the source host, and the memory access patterns are recorded; then, the memory pages are transferred to the destination server according to the memory access pattern to reduce page faults of the destination server; when the source server experiences a non-memory-intensive period, the execution of the VM on the source server is terminated; now the migration becomes a typical post-copy scheme. The VM context on the source host is called *guide context*, and that on the destination host is called *migrated context*. The guided context is used to guide the page transfer. However, their method has two problems. (1) When the execution on the source server is slower than that on the destination server due to some reasons or the network gets stuck, the memory access patterns will not alleviate the page faults of the destination server. (2) During migration, the migrated VM is simultaneously running on both the source server and the target server, which results in a big resource waste.

Deshpande and Keahey [141] propose a traffic-sensitive technique to migrate co-located VMs. They believe that when

the migration traffic has the same direction with the traffics of the applications running in the VM, they will contend for the network bandwidth. When their directions are opposite, there is no competition. Therefore, pre-copy will contend for network bandwidth with the applications on the source host which have outbound network traffic, while post-copy competes for network bandwidth with the applications on the destination host which have inbound network traffic. Based on these observations, their migration mechanism combines pre-copy with post-copy to lower the overall network contention for multiple migration. Some of the co-located VMs are migrated by pre-copy, whereas some are migrated by post-copy.

The WWS of a VM is much smaller than the full memory fingerprint. By utilizing this feature, Deshpande *et al.* [142] further propose to move non-WWS memory pages to a portable per-VM swap device before migration. Only the WWS pages are migrated through the direct TCP connection in pre-copy manner, while the non-WWS pages are remotely fetched from the swap device by the destination host on demand.

E. Summary of Memory Data Migration Technologies

The migration technologies for memory data are summarized in TABLE IV. The majority of optimizations are designed for pre-copy mechanism due to its robustness. Additionally, almost all optimization technologies are implemented on Xen or KVM because of the open-source feature. We also conclude the benefits and costs of different optimizations according to the criterion discussed in Section II-D. From the table, we can see that total migration time is the main consideration of migration performance. The improvements for other three metrics are evenly distributed. Different optimization strategies also introduce various overheads to the source or the destination hosts. As discussed in Section II-D, the costs contain computational overhead to the source and the destination host, network overhead and space overhead. Some optimizations introduce computational overheads, such as compression, data deduplication, CR/TR, page reordering, etc. Some bring in network overheads, such as distributed data deduplication, feedback-based migration, heterogeneous migration. Additional spaces are also required by some strategies, such as storing the reference data of delta compression and the hash tables of data deduplication.

As shown in Fig. 5, for a better summary of memory data migration optimization mechanisms, we classify them along different points of the migration path: the migrated VM(s), other co-located VMs, I/O operation, VMM, hardware, network link, storage system. The optimizations for the migrated VM(s) are to reduce the original data size, such as compression [77], [78], [80], data deduplication [86], [90], [91], [93], CR/TR [111], [112], ballooning [19], [70], [72], [137], free page elimination [19], [105], etc. For post-copy and hybrid-copy, the technology on this level is on-demand fetching. Some efforts aim to lower the impact of migration on the migrated VM and other VMs on the source

TABLE IV
THE SUMMARY OF MEMORY DATA MIGRATION TECHNOLOGIES

Pat.	Ref.	G	Tec.	VMM	Benefit				Cost	
					T	D	N	I	Co	Sp
Pre	[19]	S	BA/FE/PWWS	Xen	✓	✓	✓		✓	
	[68]	S	-	VMware					✓	
	[104]	S	FE/NC	KVM	✓	✓			✓	✓
	[77], [78]	S	NC	Xen	✓	✓	✓		✓	
	[80], [82]	S	DC	KVM	✓	✓	✓		✓	✓
	[90], [91], [100], [101]	S	DD	KVM	✓		✓		✓	✓
	[86], [93], [98]	S		Xen	✓		✓		✓	
	[121], [122]	M		KVM	✓		✓		✓	
	[105]	S	FE	Xen	✓		✓		✓	✓
	[37]	S	DD,DC	Xen	✓		✓		✓	✓
	[120]	M		KVM	✓		✓		✓	✓
	[108], [110]	S	RDMA	Xen	✓	✓		✓		
	[111], [112]	S	CR/TR	-	✓	✓	✓		✓	✓
	[114]	S	CRe	Xen	✓			✓		
	[115], [116]	S	PR	KVM	✓		✓	✓	✓	✓
	[19], [117]–[119]	S	MC	Xen	✓	✓	✓		✓	
	[20]	S		VMware	✓	✓			✓	
	[110]	S		KVM	✓	✓	✓	✓	✓	
	[17]	M	IR	Xen				✓	✓	
	[126]	M	IR, BA	Xen				✓	✓	✓
	[123]	M	MS	-		✓	✓		✓	
	[124], [125]	M	MS,BA	-		✓	✓		✓	
	[127], [128]	M	FB	KVM	✓				✓	
	[130]	M	RR	Xen	✓	✓			✓	✓
	[143]	M	AN	-			✓	✓	✓	
	[131]	S	HM	KVM, Xen					✓	✓
	[132]	S	RM	Xen	✓		✓	✓		
	[133]	S	TP	Xen	✓	✓	✓	✓		✓
	[134]	S		Xen	✓	✓			✓	✓
	[135]	S	TS, BA	VMware	✓			✓	✓	
	[136]	M	PS/BA	-	✓			✓	✓	✓
Post	[70], [137]	S	-	Xen	✓			✓	✓	✓
	[138]	S	PWWS	KVM	✓			✓		
	[71], [139]	S	RO	KVM	✓		✓	✓		
	[140]	S	MC	KVM	✓		✓		✓	
Hyb.	[72]	S	DC	Xen	✓		✓	✓	✓	✓
	[73]	S	GC	KVM	✓			✓	✓	
	[141]	M	NA	KVM	✓			✓	✓	
	[142]	S	PSD	KVM	✓		✓		✓	✓

The abbreviation list: *Pat.*: migration pattern. *Hyb.*: hybrid-copy. *Ref.*: references. *G*: migration granularity (*S*: single migration, *M*: multiple migration). *Tec.*: migration or optimization technology. *T*: total migration time. *D*: downtime. *N*: total network traffic. *I*: service interruption. *Co*: computational overhead. *Ne*: network overhead. *Sp*: space overhead. *FE*: free page elimination. *NC*: normal compression. *DC*: delta compression. *DD*: data deduplication. *CRe*: checkpointing and replication. *PR*: page reordering. *MC*: migration convergence. *IR*: interference reduction. *BA*: migration bandwidth allocation. *TS*: target selection. *PS*: path selection. *MS*: migration sequence. *FB*: feedback-based migration. *RR*: resource reservation. *AN*: application and network aware. *HM*: heterogeneous migration. *RM*: resource management. *TP*: migration triggering point. *PWWS*: transferring WWS with the running states to resume VM. *RO*: resource overcommitment. *GC*: guide-copy. *NA*: network contention alleviation. *PSD*: using portable swap devices to migrate non-WWS pages.

and the destination host [17], [77], [78]. To decrease the data transferred in the iterative copying phase and make migration convergent, the I/O features of the migrated VM are analyzed and utilized by some migration mechanisms, such as, application-aware migration [17], [110], [117], [119], WWS detection [19], [115], [138], different termination conditions [37], [144], [145], migration convergence strategies [110], [117], [119]. The improvements on the VMM level contain active pushing, prepagging, heterogeneous migration [131], migration triggering point [133], MMU integration [140], and page sharing [93], [100], [101]. The RDMA functionality of interconnects can be also used for VM migration [108]. Many efforts are made to increase the utilization of network bandwidth, such as bandwidth allocation strategies for multiple migration [17], [123], dynamic rate-limiting [19], [110], multi-threading [100], page reordering [115], [116], page clustering [108], network contention alleviation [141]. The destination server also can directly fetch the clear memory

pages (replicas of disk blocks) from the shared storage system to reduce the data transferred from the source site [98].

V. STORAGE DATA MIGRATION

Storage data migration has both similarities and differences with memory data migration. Both of them are to transfer data between two sites, so some memory data migration mechanisms are also suitable for storage data migration, such as data deduplication. Storage data migration also faces different challenges. (1) Low migration bandwidth. Storage data migration normally happens between two data centers where the network bandwidth is much smaller than the interconnect of a data center. (2) Big data size. The size of the virtual disk of a VM ranges from several to hundreds of gigabytes. (3) Storage data migration is conducted on block level while memory data migration is on page level. (4) Memory data have a closer relationship with the QoS of the migrated VM than

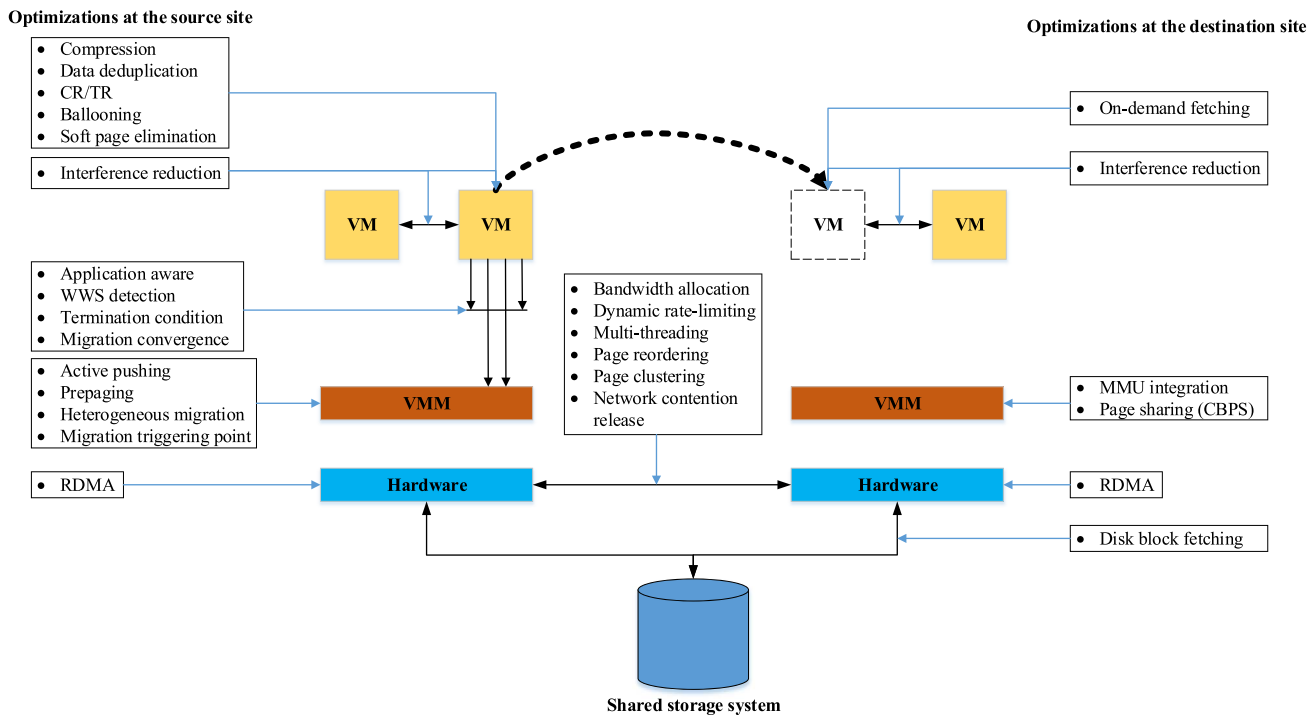


Fig. 5. The optimizations at different points of memory migration path.

storage data. With these conditions, some special optimization technologies different from those for memory data migration are proposed for storage data migration.

A. Migration Pattern

According to the migration sequence between memory data and storage data, storage data migration can be also classified into three patterns: *pre-copy*, *post-copy*, and *hybrid-copy*. Pre-copy migrates storage data before memory data, while post-copy transfers storage data after memory data. Hybrid-copy migrates storage data and memory data simultaneously. By combining different memory data and storage data migration patterns, nine migration patterns are available for live VM migration over WAN: *Pre-Pre*, *Pre-Post*, *Pre-Hybrid*, *Post-Pre*, *Post-Post*, *Post-Hybrid*, *Hybrid-Pre*, *Hybrid-Post* and *Hybrid-Hybrid*, as shown in Fig. 6.

The two patterns in each name denote memory data migration pattern and storage data migration pattern, respectively. For example, *Pre-Hybrid* migrates memory data in pre-copy manner and storage data in hybrid-copy manner. In other words, memory and storage data are concurrently transferred, namely hybrid-copy, and memory data are migrated with pre-copy pattern. If the VM is running on the source host during storage data migration, two additional mechanisms are required: (1) the dirtied blocks will be logged and retransferred to the destination host for data consistency, such as *Pre-Pre* and *Pre-Hybrid*; (2) a strategy is needed to coordinate the I/O operations from the migrated VM and the read operations from the migration process.

As discussed in Section IV-A, post-copy and hybrid-copy have a weak robustness for memory data migration, which is

also applicable for storage data migration patterns. Therefore, from migration pattern names, the pattern containing *Post* or *Hybrid* is weak regarding robustness. They may lose data or destroy the migrated VM if the migration fails at halfway. Only the *Pre-Pre* pattern can ensure the migrated VM correct under different situations, and does not need manual intervention even migration outage happens. From Fig. 6, the robustness of *Pre-Pre* pattern is guaranteed because it hands over the migrated VM only when the destination site receives all data of the migrated VM.

B. VMware Strategies

VMware Inc. [146] is the most productive enterprise about virtualization and cloud management technologies. Snapshotting, Dirty Block Tracking (DBT) and IO Mirroring are successively proposed for VMware ESX to migrate the storage data of a VM [62].

VM snapshotting works as: when a snapshot is taken for the storage data of a VM, the snapshot becomes read-only and all new writes are redirected to a new file. Based on this characteristic of VM snapshotting, snapshots are iteratively created and copied to the destination data center. This operation is repeated until the snapshot size is smaller than a threshold. Then the VM is suspended, and the last snapshot is copied to the destination site. DBT is similar with pre-copy memory data migration mechanism. Firstly the entire disk data are copied to the destination site by a full transfer phase, concurrently a bitmap is created to track the dirtied blocks. After finishing a transfer round, the dirtied blocks are transferred again. Until the amount of dirtied blocks becomes stable or a threshold is reached, the VM is suspended and the rest of dirtied

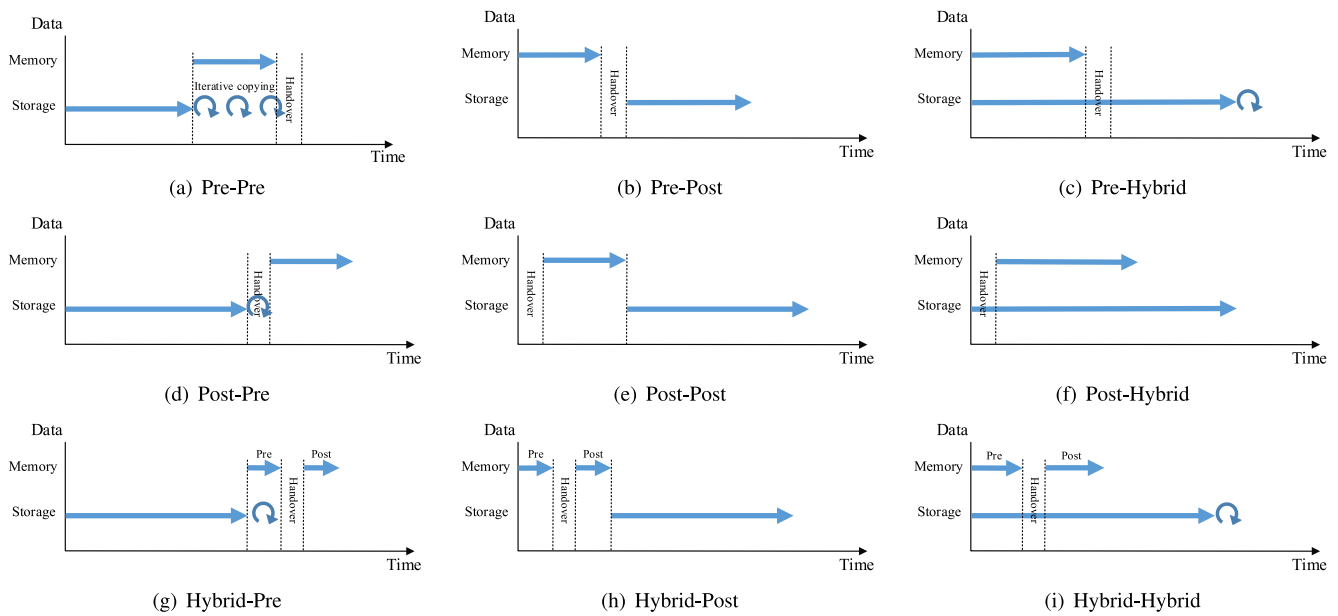


Fig. 6. Migration patterns of live VM migration over WAN. The retransmission of dirtied disk blocks can be implemented in different manners. For example, the dirtied blocks can be synchronized to the destination site during migration or transferred at the end of storage data migration in bulk. In these figures ((a), (c), (d), (g), (i)), we only show the second option.

blocks are copied to the destination site. Different from snapshotting, DBT operates in a smaller granularity, block level rather than snapshot level, which provides more optimization possibilities. For example, to lower the efforts of block tracking, only the blocks already copied to the destination site will be tracked, named *incremental DBT*. Also, the hot blocks are detected and transferred at the last copy phase to decrease the migration traffic. IO Mirroring mirrors all new writes from the migrated VM to the destination data center while the original storage data is copied in bulk. The copying process is based on VMKernel data mover (DM) [147]. Because DM reads and writes disk without the involvement of the migrated VM, a strategy is required to coordinate DM read and VM I/O operations.

These three migration mechanisms are characterized by different strengths and weaknesses. Snapshotting is simple and robust, and is resilient to performance disparity of the source and destination disk volumes. However, committing and storing intermediate snapshots introduces big computational and space overheads. DBT conducts VM migration in a smaller granularity, but it faces the same issue as pre-copy memory migration mechanism—migration convergence. When the disk blocks are dirtied faster than the transfer speed, a manual intervention is needed to stop the iteration. IO Mirroring is the latest migration technology for VMware ESX. It can guarantee migration convergence because the I/Os are continuously mirrored to the destination site. It is also resilient to different disk performances, since the mirrored I/Os are automatically tuned to the slower volume speed.

They further integrate IO Mirroring with pre-copy memory migration mechanism and implement a live migration system—*XvMotion* [20]. A variety of optimizations are designed to improve storage data migration performance. Multiple TCP connections are created to accelerate data

transfer, and a write barrier is used to ensure data consistency. To lower the impact of IO Mirroring on the running service, the writes are asynchronously instead of synchronously mirrored to the target site. XvMotion also supports to migrate a VM with multiple virtual disks which are separated on different volumes. To smooth the performance disparities of different volumes, it limits each disk to queue a maximum of 16 MB data into the shared transfer buffer. VMware also implements storage data migration by utilizing Cisco SAN extension technology [30]. They extend the storage system to be shared by the source and the destination data centers. The FCIP I/O Acceleration of Cisco switches is enabled to decrease the time of accessing storage system through data center interconnect.

C. Replication

Replication consists of two concurrent parts: bulk transfer and I/O redirection. It is similar with IO Mirroring. Bulk transfer moves the original disk of the migrated VM, while I/O redirection asynchronously or synchronously sends the new writes from the migrated VM to the destination site. Both synchronous and asynchronous replications face advantages and disadvantages [148], [149]. Synchronous replication guarantees data consistency between the source and the destination sites, without the risk of losing data. Therefore, it is applicable for migrating the VM which is running applications with a high-security requirement, such as financial system. However, it can not benefit from write coalescing to lower network traffic, even though a block is dirtied very frequently. Also, it leads to a bad service performance due to the long disk write latency. In contrast, asynchronous replication marks write operations as complete without the necessity of waiting for the responses from the destination site, therefore, it does not impact on

application performance. It provides more optimization spaces for the synchronization of newly-dirty blocks. For example, the frequently-dirty blocks can be sent to the destination site periodically to decrease the data transferred over network. Different write operations can be also batched and pipelined to shorten synchronization time. Nevertheless, asynchronous replication may lose data (these have not been synchronized to the destination site) when the source server fails during migration.

By balancing the benefits and weaknesses of asynchronous and synchronous replication, Ramakrishnan *et al.* [148] propose to asynchronously replicate new writes during transferring the original disk data and change to synchronous replication after finishing the transfer, to avoid long I/O latency due to the bandwidth contention between the copying and the synchronization processes. The replication functionality is also implemented in some software (such as DRBD [150], HAST [151]) for high availability. Directly using the software to migrate storage data is also an option. For example, Wood *et al.* [37] employ DRBD to implement storage data migration in a similar manner as [148].

Liu *et al.* [111], [112] utilize Copy-On-Write (COW) technology to store the disk data of a VM in two layers: base image and COW image. The root file system of a VM are stored in the base image which remains read-only, and a COW image is used to store new data from the running VM. They asynchronously copy the base image to the target server before an anticipated migration, and the COW image is transferred with other data (memory, VCPU states) during VM migration. However, they assume the size of COW image is considerably smaller than the basic image which is not the case in current data centers anymore. Therefore, a huge COW image will lead to a long migration time.

Bradford *et al.* [152] only utilize asynchronous replication to migrate storage data. The new writes during migration are intercepted into *deltas* which are recorded in a queue. After finishing the bulk transfer of the original disk, these deltas are copied and applied to the disk image at the destination site. When the growth speed of deltas is bigger than the network bandwidth, write throttling is taken to slow down the VM execution speed.

D. Data Deduplication

Similar with memory data, there are also many duplicate blocks between different VM images [63], [87], [153]. Therefore, storage data migration can be accelerated by data deduplication as well. Data deduplication firstly cuts a VM image into blocks, and calculates a fingerprint for each block. The fingerprints are utilized to locate duplicate blocks within an image or between different images. These duplicate blocks are eliminated or transferred only once to reduce data transfer time.

Each memory page is an identification unit in data deduplication for memory data migration. However, for storage data, data deduplication faces how to cut a big file into small pieces. There are two options: fixed-size chunking or variable-size chunking. Fixed-size chunking is simple and

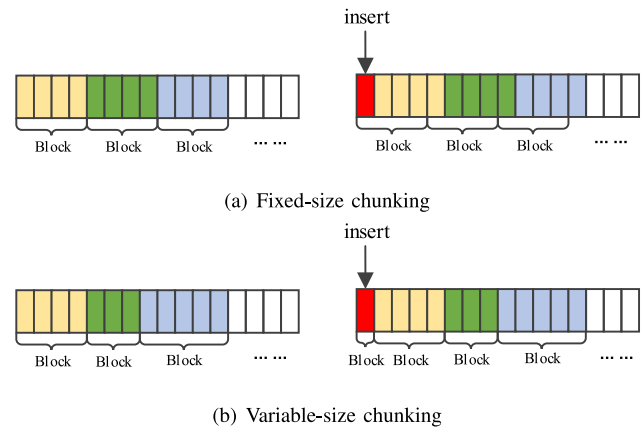


Fig. 7. The bit-shifting problem with file chunking.

has a small computational overhead, but it suffers from bit-shifting problem. As shown in Fig. 7(a), inserting one or several bits at the beginning of a block, all of the following blocks will be changed. Variable-size chunking [154] is not bothered by this problem. However, it is compute-intensive, which limits its benefits to migration performance improvement. Hence, fixed-size chunking is more popular for storage data migration [39], [153], [155]. Same as memory data migration, SHA-1 and SuperFastHash still are the two popular fingerprint-calculating algorithms [86], [93], [101], [156], [157].

As depicted in Section IV-B2, data deduplication for storage data migration also can utilize three types of similarities: *intra-image similarity*, *inter-image similarity*, and *inter-site similarity*. Intra-image similarity denotes the duplication situation within the image of the migrated VM. Inter-image similarity is the duplicate blocks located between the images within a data center, while inter-site similarity is the duplication condition between the images at the source site and these at the destination site.

Different migration mechanisms exploit one or several of the three similarities to facilitate VM storage data migration according to migration conditions. Bose *et al.* [158], [159] propose to decrease migration time at the expense of storage space. They keep several replicas of a VM image at different cloud data centers, and choose one replica as the primary copy. The changes from the primary copy are propagated to other replicas periodically. The selection of data center takes long-term average computation cost and end-user latency requirements into consideration. The Content Based Redundancy (CBR) technique with Rabin fingerprints [154] is employed to decrease the data transferred over the network during propagating changes. With this image distribution structure, they name the movement of VM storage data in their system (*CloudSpider*) as *hiberwake* (short for *hibernate-awake*) to differentiate from normal VM migration. In normal migration, all storage data must be transferred from the source site to the target site, while in the *hiberwake* only the new changes of storage data need to be copied to the target site. Therefore, it can dramatically lower the total migration time. However, it is also accompanied with a big management cost and a big space

consumption. The periodical change updates import additional network traffic as well. Riteau *et al.* [91] also expands their mechanism (distributed content-based addressing) for memory data migration [90] to storage data migration.

Zhang *et al.* [156] propose to deploy the disk image of a VM into three layers: an operating system (OS) layer, a working environment (WE) layer and a user data (UD) layer. Applications or software stacks are installed in the WE layer by using an OS image as backing file. Both OS and WE images are running as the base images of a UD image, and remain read-only during their whole lifetime. The modifications to base images are redirected to the UD image. They name this structure as *three-layer image structure*. With this structure, the data of a VM (OS and WE images) which have a high similarity possibility are kept unmodified, and the data (UD image) with a small similarity possibility are stored in the UD layer. After this separation, they only conduct data deduplication on the OS and WE layers to improve deduplication efficiency since data deduplication must make a trade-off between computational cost and migration benefits. The three-layer image structure improves data sharing between VMs, which is further beneficial to multiple migration. They [160] then propose to introduce a central repository to deploy and store base images (OS and WE images) for different data centers. With this structure, base images can be reused between different data centers, which can reduce the network traffic during storage data migration. Some further optimizations for data deduplication are also proposed by them. Celesti *et al.* [46] propose a similar migration mechanism, but they only separate a VM image into two layers: base image and user data. This separation does not thoroughly exploit the possible data sharing within a data center because many VMs are running with the same software stack.

Sometimes, we need to frequently migrate a VM between two or several fixed locations. For example, a personal VM system is commutatively used between home and office [161]. In such scenarios, many disk blocks are reusable. Takahashi *et al.* [39] combine data deduplication with DBT mechanism to speed up the migration between two fixed places. When a VM will be migrated back to a location where its previous version of disk data are located, only the newly dirtied blocks are transferred.

E. Software-Based Approach

Some works carry out storage data migration by directly utilizing existing solutions or a software implementation. Hirofuchi *et al.* [162], [163] migrate VM storage data based on a block-level I/O protocol—*Network Block Device (NBD)* [164]. Their migration system consists of two NBD storage servers through which the source and the destination hosts access storage data, respectively. Virtual disks of a VM are block device files (e.g., */dev/nbd0*) on a host OS. At the beginning of migration, the memory data of the migrated VM are firstly transferred to the destination server, and then the storage data are migrated in a post-copy manner through the NBD connection between the two storage servers. Disk blocks are directly fetched from the NBD storage server at

the source site, which lowers the interruption to the source host and other VMs on this host. On-demand fetching and background copying are combined to accelerate storage data transfer.

Tang [165] designs a new virtual machine image format—*Fast Virtual Disk (FVD)*, which supports a series of functionalities, such as Copy-on-Read (CoR), adaptive prefetching, Copy-on-Write (CoW), internal snapshot. Some of the functionalities are beneficial to VM migration. For example, CoR and adaptive prefetching can be combined to gradually copy the virtual disk in the background to the target host in a post-copy manner. CoR transfers data blocks on demand, and adaptive prefetching transfers data during resource idle time. Adaptive prefetching even can be paused and its transfer rate is adjustable as well.

F. I/O-Aware Migration

Storage data migration encounters the same problem as memory data migration. If storage data are migrated in post-copy pattern, the I/O feature of the migrated VM determines the frequency of remotely accessing disk blocks. If pre-copy is employed, the dirtying rate of disk blocks is critical for migration performance. Some migration mechanisms take the I/O features of the migrated VM into consideration for a better control of migration process. Zheng *et al.* [166] design a scheduling algorithm for storage data migration. Instead of transferring storage data from the beginning to the end of a disk, their algorithm considers the I/O characteristics of the workloads running in the migrated VM to arrange the migration sequence of disk blocks. It records a short history of the disk I/O operations to predict the future I/O characteristics in terms of temporal locality, spatial locality, and popularity (read/write frequency). According to their experiments, these I/O characteristics are predictable. The migration technology can be used to optimize different migration schemes (pre-copy, post-copy, and hybrid-copy) and has a strong adaptability for different workloads. It is beneficial to reduce the data iteratively transferred in the pre-copy migration pattern, to decrease the blocks remotely accessed in the post-copy migration pattern, and to improve both of these two aspects in the hybrid-copy migration pattern.

Similarly, Nicolae and Cappello [167] mainly aim at improving the storage data migration performance of I/O-intensive VMs. By utilizing the spatial and temporal localities of disk access, they propose a hybrid active push/prioritized prefetch strategy. They monitor and record how many times a block has been written, and only the blocks which have been written more than a preset threshold are marked as *dirty*. During VM migration, they avoid transferring these frequently written blocks and fetch them in decreasing order of access frequency from the destination site after VM handover.

G. Multiple Migration

Many VMs in a data center are correlated with several others rather than running independently [121], [168], [169]. Migrating one of them to another data center will lead to a severe service degradation due to the big network latency

between data centers which decreases the communication performance between these VMs. Therefore, besides the three challenges of VM migration, multiple migration faces some new problems, such as migration sequence and the interruption to the inter-VM communication.

Al-Kiswani *et al.* [157] call the VM images belonging to the same application as a *VMFlock*. They design a migration system—*VMFlockMS*, to migrate a *VMFlock*. *VMFlockMS* consists of three components: *VM Profiler*, *VM Migration Appliance*, and *VM Launch Pad*. *VM Profiler* logs the blocks which are necessary for VM boot. These blocks are prioritized to transfer to the destination site. *VM Migration Appliance* utilizes distributed data deduplication and transfer approach to migrate storage data. Several nodes are used to parallelize data deduplication and transfer. Each node deduplicates and transfers the blocks with hashes in a specified range. Both inter-image and inter-site similarities are exploited. *VM Launch Pad* resumes the VMs once all blocks logged by the *VM Profiler* are received. The remaining data is migrated in post-copy manner.

A lack of progress management brings many issues to multiple migration. For example, how long will each migration process take?, how is the trade-off between application performance degradation and migration time?, how do multiple migration processes avoid splitting application components across data centers?, etc. In order to lower the impacts of migrating multiple correlated VMs over WAN on application performance, Zheng *et al.* [170] design a migration progress management system—*Pacer*. They firstly design models to predict the dirtying rates of memory and storage data and total migration time. On the basis of these models, *Pacer* manages migration by controlling the migration time of each migration process and coordinating them to finish at a close time to alleviate the component-split issue.

For the same problem, Zheng *et al.* [171] propose a communication-impact-driven coordination algorithm to decrease the service interference of VM migration. They formulate multi-tier application migration as a problem of minimizing performance impact. They define the performance impact as:

$$impact = \sum_{i=1}^n \sum_{j>i}^n |t_i - t_j| * TM[i, j] \quad (1)$$

TM is the communication traffic matrix between the migrated VMs, and t_i is the migration finish time of VM i . They implement their migration system—*COMMA*, based on this performance impact model. *COMMA* consists of a central controller and a local process in each VM's hypervisor. It migrates VMs in two steps. In the first step, it coordinates the migration of all VM's storage data to make them finish at the same time. In the second step, they put VMs into different *valid groups*. The sum of the dirtying rates of the VMs in a valid group is smaller than the available network bandwidth. Then inter-group scheduling and intra-group scheduling are combined to lower the impact of migration on the communications between the migrated VMs and improve the network utilization, respectively. According to

experimental results, parallel migration is better than sequential migration, and *COMMA* is further better than parallel migration regarding service interruption. However, due to the coordination of reducing communication impact, it may result in a longer total migration time in comparison with parallel migration. Regarding the migration sequence problem, Cerroni [45], [172] compares sequential and parallel migration strategies when a group of VMs will be migrated in cloud federation. The results illustrate that sequential migration has less influence on network performance and parallel migration results in a smaller downtime.

Although shared storage systems (NAS or SAN) are popular in data centers, share-nothing storage architecture also is widely employed due to its high scalability. With share-nothing architecture, the storage data of a VM must be located where the VM is running. Furthermore, with the strict agree-upon SLAs, some management tasks (such as load balancing) should keep away from the high-load periods of the related VMs (the migrated VMs, the VMs located on the source and target host), to lower the interruption to the running services. Therefore, in practice each migration has to be finished within a time window, otherwise, a cost penalty will be followed due to the violation of SLA. Tsakalozos *et al.* [173], [174] design a migration system to improve VM migration performance when a share-nothing storage system is adopted and each migration task must finish within a time window. Their migration system is composed of three components: (1) a central migration scheduler which issues migration tasks; (2) a broker running in each hypervisor to monitor the resource consumption of VM migration; (3) a special-purpose file system—*MigrateFS*. In order to fulfill the time constraint of each migration, two resource consumptions are tuned: the network bandwidth for migration and the disk I/O bandwidth of the migrated VMs. They are controlled by the *MigrateFS* according to the information collected from brokers. Under this framework, the disk images are migrated in a similar manner as asynchronous replication. Their migration system prioritizes total migration time at the expense of service performance because of controlling disk I/O speed.

H. Others

Apart from the general issues with storage data migration, many researchers try to solve some problems in special situations. Luo *et al.* [175] propose a Three-Phase Migration (TPM) algorithm which is composed of pre-copy, freeze-and-copy, and post-copy. In the pre-copy phase, storage data and memory data are iteratively transferred to the destination site. The dirty information of storage data is recorded in a block-bitmap. In the freeze-and-copy phase, the VM is suspended and the dirtied memory data and the block-bitmap are sent to the destination server. In the post-copy phase, the VM is resumed on the target server, and the modified storage data blocks are moved in a pull and push manner according to the block-bitmap. They also use write throttling for the I/O intensive workloads to ensure migration convergence. After the VM is moved to the target server, a new block-bitmap is created to

record the new changes of disk data. Because in some scenarios (such as hardware maintenance) the VM will be migrated back to the original server, they further propose an Incremental Migration (IM) scheme. When the VM is being migrated back to the original server, only the blocks are dirtied in the destination server are synchronized to the source site according to the new block-bitmap.

Nowadays, solid-state drive (SSD) is widely used in data centers. They have faster I/O speed than mechanical hard disk drive (HDD). Zhou *et al.* [176] try to solve the I/O speed disparity problem when migrating VM images between SSD and HDD. They design three migration strategies for different situations: (1) Low Redundancy (LR) mechanism. Because all disk data of the migrated VM will be eventually moved to the destination site, LR mechanism directly writes data to the destination host during migration. Therefore, the disk of the migrated VM is divided into copied region and to-be-copied region. The read operations to the copied region will fetch the data from the destination site. (2) Source-based Low Redundancy (SLR) mechanism. It is based on LR mechanism, but it keeps the I/O operations to the to-be-copied region at the source site while issues write operations to the copied region to the destination site. (3) Asynchronous IO Mirroring (AIO) mechanism. It is derived from IO Mirroring [20]. The original IO Mirroring writes data to both the source and the destination sites, but AIO marks the write operation as complete when the faster disk accomplishes the write operation and the slower disk conducts the write operation in the background. The first and third strategies are for the migration from a slow disk (HDD) to a fast disk (SSD), while the second is for the migration from a fast disk (SSD) to a slow disk (HDD).

Normal migration approaches only transfer the memory and storage data of the migrated VM to the target site, but without moving the host-side cache data. Therefore, the VM will suffer from a big performance degradation after resumption on the destination server. There are three layers of caches in a virtualization system: VM direct cache (L1), host-side cache (L2), and storage-side cache (L3). Lu *et al.* [177] design a cache warm-up mechanism—*Successor*, to recover the cache data at the destination host during VM migration. They mainly focus on the recovery of the host-side cache (L2). A cache warm-up mechanisms is designed for both LAN migration and WAN migration. They parallelize the warm-up process with the migration process. *Page preloading* is utilized to warm up the cache of the destination host from the storage server when a VM is migrated within LAN. For WAN migration, a *piggyback warm-up on migration* method is adopted. When the storage data of the migrated VM are being transferred from the source site to the destination site, the cache of the destination host is filled with the hot pages on-the-fly.

Shen *et al.* [178] design a geo-replicated image file storage to support efficient migration of VM storage data based on Supercloud [179]. It cuts an image file into constant size blocks (4KB) and stores several replicas for each block in other cloud data centers. The primary replica (i.e., where the VM is running) propagates block updates to other replicas according to a transition probability table which is added into the meta-data of each image file. The transition probability

table records the probability that a VM is migrated from current cloud data center to another. It can be created by users or trained on the fly. They also assign a priority for each block according the read/write frequency. During VM migration, the blocks with high priority are firstly propagated to the destination site and then update other blocks. Besides, many blocks of a VM image file (e.g., the blocks of base image) remain unmodified during their whole lifetimes, which will be further beneficial to storage data migration.

Arif *et al.* [180] utilize a machine learning based method to guide the migration over WAN to minimize total migration time and downtime. They use a monitoring engine to continuously monitor and log the resource utilization in a data center and VM migration events. These log data and several thresholds are applied to guide the decision making for future VM migrations. Kumar and Schwan [181] extend the device virtualization module of VMM with a Mid-Point (MP) module which is special for monitoring the states and the pending I/O operations of virtual devices. During VM migration, the MP of the source host establishes a channel with that of the destination host to seamless transfer device states and pending I/O operations, called *device hot-swapping*.

I. Summary of Storage Data Migration Technologies

The migration technologies for storage data are summarized in TABLE V. From the table, we can find many common phenomena with memory data migration. (1) Same as memory data migration, data deduplication still is an important optimization technology for storage data migration. (2) Pre-copy pattern is also widely utilized for storage data migration due to its robustness, and the majority of optimizations are designed for it. Combining with pre-copy memory migration pattern, Pre-Pre becomes the most popular migration pattern for live VM migration over WAN. (3) KVM and Xen are the two popular experimental platforms. (4) Most of the studies are concentrating on single migration. Actually, optimizing the performances of multiple migration is strongly desirable as well. (5) The total migration time still is the main concern among all performance metrics. (6) Computational overheads to the source and the destination servers are the main side effect. Only a small part of optimization technologies bring in network and space overheads.

We also illustrate different migration technologies and optimizations through the migration path in Fig. 8. The majority of migration technologies take the synchronization-like manner, i.e., the original disk image is copied in bulk and the new disk writes are recorded and iteratively transferred to the destination site, such as snapshotting, DBT [62], IO mirroring [20], [62], replication [37], [111], [112], [148], [152]. Other available migration mechanisms include NBD [162], [163], central base image repository [160], etc. On the basis of these technologies, many optimization strategies are designed to further improve the migration performance, such as data deduplication [86], [93], [101], [156], [157], write throttling [175], layered image structure [46], [111], [112], [156], [160], new image format [165], special file system [173], [174], cache warm-up [177], bandwidth allocation [173], [174], etc.

TABLE V
THE SUMMARY OF STORAGE DATA MIGRATION TECHNOLOGIES

Main	Others	Ref.	Pat.	G	VMM	Benefit			Cost		
						T	N	I	Co	Ne	Sp
SNT	-	[62]	Pre-Pre	S	VMware				✓	✓	✓
DBT	iDBT					✓	✓		✓		✓
IOM	M-TCP,BC	[20], [62]	Pre-Pre	S	VMware	✓		✓		✓	
eSAN	CIOA	[30]	Pre-Pre		VMware	✓	✓	✓		✓	
Rep	A-S	[148]	Pre	S		✓	✓				
	BRDB	[37]	Pre-Pre	S	Xen		✓	✓			
	LIS	[111], [112]	Hybrid	S	-			✓			✓
	A,WT	[152]	Pre-Pre	S	Xen	✓	✓			✓	
DD	DCBA	[91]	-	S	KVM	✓			✓		✓
	HW	[158], [159]	-	S	-	✓				✓	✓
	LIS	[46], [156]	Pre-Pre	B	KVM	✓	✓		✓		
	LIS,CB	[160]	Pre-Pre	B	KVM	✓	✓	✓		✓	
	DBT	[39]	-	S	KVM	✓	✓		✓		✓
SA	DDD	[157]	Post	M	-	✓	✓		✓		
	NBD	[162], [163]	Pre-Post	S	Xen KVM	✓	✓				
	FVD	[165]	-	S	KVM	✓	✓				
IOA	BS	[166]	All	S	-	✓	✓	✓	✓		✓
	PPP	[167]	Hybrid	B	KVM	✓	✓	✓	✓		✓
MPM	PM	[170]	Pre-Pre	B	KVM			✓	✓	✓	
IR	MC,PIM	[171]	Pre	M	KVM			✓	✓		
RM	A	[173], [174]	Pre-Pre	M	Xen	✓	✓		✓		
TPM-IM	WT	[175]	Pre-Pre	S	Xen	✓	✓		✓		✓
HM	LC,SLR,AIO	[176]	-	S	Xen	✓		✓	✓		
CWU	PP,PWM	[177]	Pre-Pre	S	KVM			✓	✓	✓	
ML	PPr	[180]	All	B	-	✓	✓		✓		✓
GRFS	PBM	[178]	Hybrid	B	Xen	✓		✓	✓	✓	✓
DHS	-	[181]	All	B	Xen		✓	✓	✓		✓

The abbreviation list: *Main*: main migration strategy. *Ref.*: references. *Pat.*: pattern (one pattern denotes the literature only considers storage data migration and the pattern is storage data migration pattern, while a pattern pair means the literature considers both memory data and storage data migrations. *All* denotes the proposed optimization is suitable for all migration patterns.). *G*: granularity (*S*: single migration, *M*: multiple migration, *B*: single and multiple migrations). *Others*: other optimization technologies. *T*: total migration time. *N*: total network traffic. *I*: service interruption. *Co*: computational overhead. *Ne*: network overhead. *Sp*: space overhead. *SNT*: snapshotting. *DBT*: dirty block tracking. *iDBT*: incremental DBT. *IOM*: IO Mirroring. *eSAN*: extended SAN technology. *Rep*: replication. *DD*: data deduplication. *IOA*: I/O-Aware. *MPM*: migration process management. *SA*: software-based approach. *IR*: interference reduction. *RM*: resource management. *MS*: migration sequence. *TPM-IM*: Three-Phase Migration and Incremental Migration. *HM*: heterogeneous migration. *CWU*: cache warm-up. *M-TCP*: multiple TCP connections. *BC*: buffer congestion control. *CIOA*: Cisco FCIP I/O Acceleration. *A-S*: asynchronous and synchronous replication. *A*: asynchronous replication. *LIS*: layered image structure. *CB*: central repository. *WT*: write throttling. *DCBA*: distributed content-based addressing. *HW*: hiberwake. *DDD*: distributed data deduplication. *BS*: block scheduling. *PPP*: push/prioritized prefetch. *PM*: performance predict models. *MC*: migration coordination. *PIM*: performance impact model. *PP*: page preloading. *PWM*: Piggyback warm-up on migration. *ML*: machine learning. *PPr*: performance prediction. *GRFS*: geo-replicated file storage system. *PBM*: priority-based block migration. *DHS*: device hot-swapping.

VI. NETWORK CONNECTION CONTINUITY

Network connection continuity is a challenge for migrating a VM across data centers. When a VM arrives at a new data center, it normally will get new network parameters. Therefore, some strategies are needed to keep the network connection between the migrated VM and its users. There are many technologies available for this purpose. In this section, we review the studies on network connection continuity for VM migration from different network layers.

A. Layer-2 Solution

The solutions in Layer-2 for keeping network connection are similar. The core idea behind them is to extend LAN to multiple data centers. Then the migrated VM can preserve its network configuration during and after migration. Cisco Data Center Interconnect (DCI) solution [182] is utilized by VMware to solve the network connection issue during VM migration [30]. Cisco DCI is a LAN extension technology and can be based on different transport options, such as dark fiber, multiprotocol label switching (MPLS), IP. Cisco also provides another LAN extension technology called Overlay Transport

Virtualization (OTV) [183] which is available for preserving network connections for VM migration as well. OTV utilizes a control plane protocol rather than data plane learning to exchange MAC reachability information. It introduces the concept of “MAC in IP” which dynamically encapsulates Layer-2 flows.

CloudNet [37] groups the resources of several geo-distributed data centers into a Virtual Cloud Pool (VCP). It adopts Multi-Protocol Label Switching (MPLS) based VPNs to create a private network abstraction for multiple data centers. They further use Virtual Private LAN Services (VPLS) to connect several MPLS endpoints to a single LAN segment. When migrating a VM within a VCP, no special operation is needed. Otherwise, a new VPLS endpoint will be created at the destination data center to include it into the same VCP as the source data center. The VPN-based mechanism is also utilized by Hirofuchi *et al.* [162], [163].

Jiang and Xu [184] propose an application-level virtual network architecture which is to create isolated virtual networks on an overlay infrastructure (e.g., PlanetLab). A virtual network is called a VIOLIN which is a “virtual world” containing three important entities (vHost (VM),

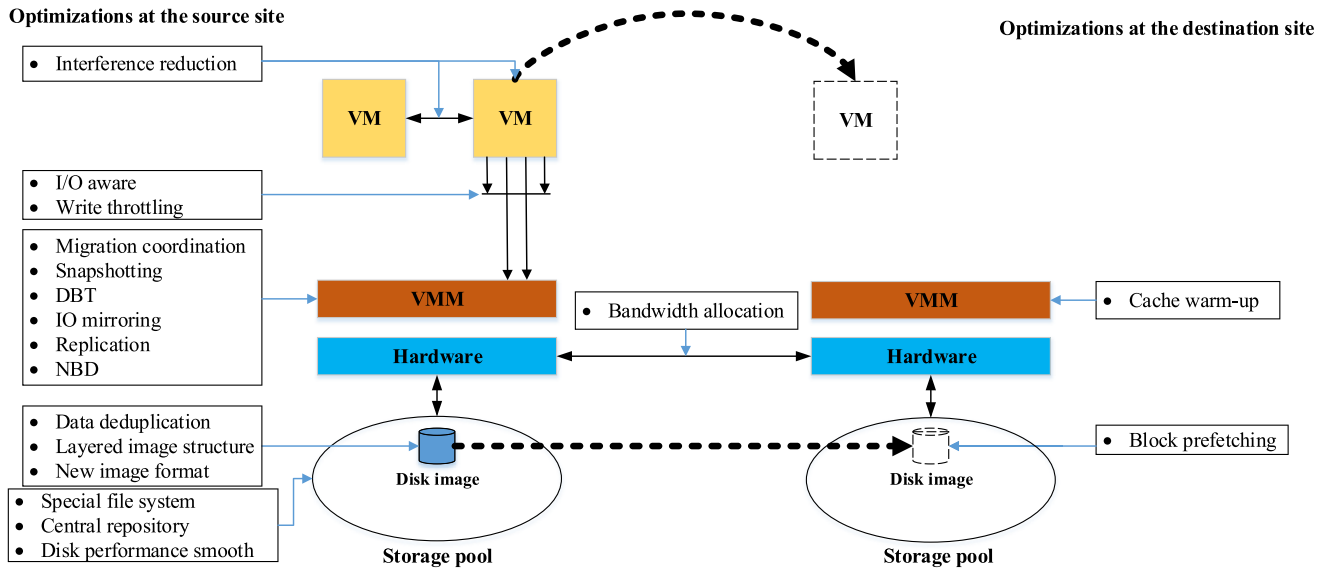


Fig. 8. The optimizations at different points of storage data migration path.

vLAN, vRouter) like in real network system (end-host, LAN, and router). All of the entities in a VIOLIN are software-based and a VIOLIN has its own IP address space, so it is easy to create, delete, and migrate them (including VMs). Ganguly *et al.* [185] propose a IP-over-P2P (IPOP [186]) virtual networking technique by using the Brunet P2P protocol [187]. This endows their system—WOW, a distributed and scalable wide-area network of virtual workstations, the ability of seamless VM mobility.

Nagin *et al.* [188] design Virtual Application Networks (VANs) to encapsulate the components of a complex application (such as three-tier application) into a virtual network. The VMs belonging to the same application can be located in different sites, and form a distributed virtual network. Therefore, the network connection can be maintained if a VM is migrated within a VAN. VAN also is utilized by Hadas *et al.* [189] in a cloud federation project—RESERVOIR [190], to improve intra- and inter-site migratability of VMs.

VLAN (a wider concept called *overlay Ethernet*) also can be implemented by different standard protocols, such as Provider Backbone Bridges (PBB, IEEE 802.1ah) [191], Shortest Path Bridging (SPB, IEEE 802.1aq) [192] and Transparent Interconnection of Lots of Links (TRILL, RFC 6325) [193]. However, there is a lack of literature which adopts these protocols to solve the network connection problem during VM migration. The key insight behind them is similar with the aforementioned layer-2 solutions, except that a special physical device—*Routing Bridge*, is needed to implement TRILL [194].

B. Layer-3 Solution

IP tunneling [195] and Dynamic DNS [196] are widely employed to redirect network connections during VM migration [38], [111], [148], [152], [197]. They divide the network

connection issue during live VM migration into two sub-problems: *preserving open connections* and *redirecting new connections*. Before the migrated VM is handed over, a tunnel is created between the source and the destination sites. When the VM is resumed on the destination host, it will get new IP and MAC addresses. The DNS entry of this VM is updated with its new network parameters. Then all new network connections will be redirected to the new location. Meanwhile, all old connections still communicate with the source site of the VM. All packets sent to the source site are forwarded to the destination site through the tunnel until all open connections are closed. This mechanism is simple and easy to deploy. However, its drawbacks are also obvious. It results in a long convergence time which in turn leads to a long residual dependency on the source site.

VMware Inc. cooperates with F5 Networks Inc. [198] and designs an IP tunneling alike solution [43]. Several F5 products and technologies are integrated into vMotion for data transfer during migration and network redirection after migration, such as BIG-IP Local Traffic Manager (LTM), BIG-IP Global Traffic Manager (GTM), BIG-IP integrated WAN optimization services and iSessions tunneling. Before migration, an iSessions tunnel is created by using BIG-IP LTM between the source and destination data centers. Then the VM is migrated through this tunnel. After migration, the BIG-IP LTM at the source site redirects the active connections to the target site through the iSessions tunnel. Meanwhile, the BIG-IP GTM sends new connections directly to the new place of the migrated VM. This solution is available for both planned and unplanned migration events.

Mobile IP [199], [200] is special for supporting seamless network connectivity when a node is moved from one place to another. In Mobile IP, each node has two addresses: a home address and a care-of address. The home address is permanent, while the care-of address is associated with the network where the node is currently located. Network connection is kept

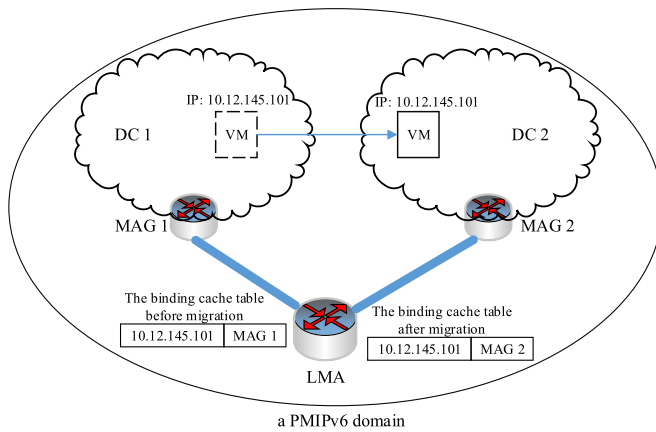


Fig. 9. The overview of PMIPv6 structure.

during VM migration by two components: Home Agent (HA) and Foreign Agent (FA). The HA maintains the permanent address of the node and acts as the router of this address, while the FA manages the care-of address. For live VM migration, the packets are forwarded from the HA to the FA after migration to keep the VM reachable. Pre-installing a Mobile IP stack is time-consuming. To this end, Li *et al.* [201] propose a Hypervisor controlled Mobile IP (HyperMIP) to reduce the overhead of network redirection during VM migration. HyperMIP is integrated with hypervisor which acts as the agents of Mobile IP. As the next generation of Internet technology, Mobile IPv6 has more advantages than Mobile IPv4. Harney *et al.* [202] design a migration system which utilizes Mobile IPv6 to keep network continuity.

The downtime of live VM migration not only includes the time t_1 for transferring the remaining data and resuming the VM, but also includes the time t_2 for recovering the network connection. Silvera *et al.* [197] find that many features of Mobile IP are unnecessary for live VM migration, such as movement detection, trust relations with FA. Therefore, they design a new network redirection framework which combines Mobile IP with IP tunneling to parallelize migration process with network configurations to reduce the time t_2 .

The Proxy Mobile IPv6 (PMIPv6) [203] designed by IETF is also capable of keeping the network connection for VM migration [204], [205]. PMIPv6 contains two main components: local mobility anchor (LMA) and mobile access gateway (MAG). A LMA is an anchor point of an access network and a MAG is an entity which is responsible for detecting and managing the attachment of a MN on its access link. As shown in Fig. 9, when a VM is migrated to another data center, MAG1 will detect and inform the LMA about the leave of this node from its domain and MAG2 will detect and inform the LMA about a new attachment of a node. LMA correspondingly changes the items in its binding cache table to redirect user's connections to the new location of the VM. When a VM is migrated within one PMIPv6 domain, it can keep its original IP address after migration. However, when the migration happens between different domains, a Mobile IP-like solution is required to maintain the IP address.

In order to overcome the set-up complexity of Mobile IP, Inayat *et al.* [206], [207] propose *Mobile IP with Address Translation (MAT)* which is based on a dual interface IP handoff technique to smooth the network mobility. MAT is an optimization to Mobile IP. It runs a MAT kernel in the Mobile Node (MN) rather than configures a Home Agent (HA). Each MN has two network interfaces for a Home Address (HoA) and a Mobile Address (MoA), respectively. The mapping between HoA and MoA is maintained by the IP Address Mapping Server (IMS). Because each VM has two network interfaces, the network at the destination site can be configured before migration, i.e., setting one interface of the migrated VM. When the migration finishes, the network connection can be fast recovered by directly switching it to the pre-configured interface. Based on the fast handoff feature of MAT, several mechanisms are designed based on it to lower the network downtime of live VM migration [208], [209].

An IP address has two basic functionalities: a locator for routing purpose (where the node is located) and an identifier for recognizing purpose (who the node is). In order to improve the mobility of a node, Locator/Identifier Separation Protocol (LISP) [210] separates these two functionalities into Route Locator (RLOC) and Endpoint Identifier (EID), respectively. The mapping information between RLOCs and EIDs are managed by a mapping system. In live VM migration, when a VM is migrated to a new location, the network connection continuity can be guaranteed by updating the mapping information of VM's EID and RLOC in the mapping system. Raad *et al.* [211] design several optimizations for LISP to lower the network downtime during VM migration. Cisco also shows many configuration examples by combining LISP with its products to implement VM mobility [212].

Xie *et al.* [213] implement seamless VM migration by utilizing Named Data Networking (NDN) [214]. In NDN, communication is conducted by location-independent names rather than IP. It is a receiver-driven communication paradigm. The migrated VM stops receiving new service requests when the migration starts, and recommences responding requests until the VM is resumed at the destination site. During the whole migration, all service requests will be redirected to other VMs. This method is only suitable for migrating the VMs which provide public services rather private VMs.

C. Layer-4 Solution

Snoeren and Balakrishnan [215] implement a *Migrate* option for TCP based on Dynamic DNS. It supports a communicating TCP peer to suspend the current connection and transparently reactivate it on another IP address. This is carried out by including the notification of IP address update into the SYN segments. Furthermore, they design a fine-grained, connection-level failover mechanism when a server exists a replica at another site [216]. It is based on a soft-state session synchronization protocol and a connection resumption mechanism. It supports the replica of a server to take over a connection in the middle of a data stream. This can be derived for the network handover of live VM migration.

Maltz and Bhagwat [217] implement *Transport Layer Mobility* (TLM) by utilizing *TCP Splice* technology. They find that multiple network interfaces can be simultaneously used by a node to get benefits from overlay networks. TLM supports a node to change its network attach point and control the used network interface.

Kalim *et al.* [218] redirect network connections based on an *isolation boundary* mechanism [219] which separates the transport endpoint identifiers from IP addresses. The isolation boundary mechanism creates a transport-independent flow (TI flow) which is different from TCP flow. Therefore, IP address change will not invalidate the connection. A transport-independent flow identifier (TIFID) is used to identify a TI flow. After a VM is migrated to a new location, its new IP address is sent with an SYN message to the client. The TIFID of the TI flow can avoid the SYN message being ignored.

With the development of cloud computing (such as the increase of the tenants accommodated in a data center, the increase of the number of data centers), the limited number of VLANs enabled by IEEE 802.1q falls short of the amount required by dispersed data centers. To this end, some novel network virtualization technologies are proposed to create a Layer-2 overlay network on a higher layer (such as Layer 3 or Layer 4), which also support VMs to retain their addresses as they are migrated within a data center and across data centers. They divide a network into a number of segments by adding outer headers to the existing network protocols. Packets are routed according to these outer headers, which decouples a virtual network from the underlying physical network infrastructure. There are three main protocols to implement this type of network virtualization: Virtual eXtensible Local Area Network (VXLAN) [220], Stateless Transport Tunneling (STT) [221] and Network Virtualization Using Generic Routing Encapsulation (NVGRE) [222]. VXLAN is UDP-based, STT is TCP-based and NVGRE is based on Generic Routing Encapsulation (GRE).

D. SDN-Based Solution

In traditional network, control plane and data plane are tightly coupled, which hinders network flexibility. To this end, Software-Defined Networking (SDN) [223] separates data plane from control plane. The network infrastructures only handle the data-forwarding issue, and the control logic is moved to a central controller. SDN has two main advantages. (1) It is easy to choose the fastest path to transfer data. (2) The network connections are easily redirected. SDN is flexible, programmable and manageable, which makes it perfect for the network connection redirection during live VM migration.

Mann *et al.* [224] implement a network fabric—*CrossRoads*, to implement seamless network connection during VM migration based on SDN/OpenFlow. *CrossRoads* introduces a central network controller to control all switches and routers in each data center. With this structure, each device (including VM, router, gateway) is assigned a pseudo MAC (PMAC) and a pseudo IP (PIP) to act as its location identifiers by getting ideas from PortLand [225]. The PMACs and PIPs for routers and gateways are fixed, while these for VMs change

according to their current location. The mapping information between the real MAC and IP addresses and the PMAC and PIP of a VM is stored in the central controller of a data center. The central controllers of different data centers can communicate with each other to forward packets between data centers to support VM migration over WAN.

Xiao *et al.* [226] fully utilize the flexibility of SDN to construct a topology-adaptive Data Center Network (DCN). Based on this structure, they try to minimize both migration cost and communication cost resulting from multiple migration. Communication cost denotes the communication overhead between the migrated VMs during migration. Optimizing both of these aspects under adaptive network topology is NP-hard. Therefore, they design a progressive-decompose-rounding (PDR) algorithm to solve this problem within a polynomial time.

Boughzala *et al.* [227] take advantage of OpenFlow to facilitate the inter-data-center operations, including live VM migration. It is implemented by defining two levels of rules: *global rules* and *specific rules*. Global rules define the network topology of a data center, and specific rules describe how operations are executed in hardware. Specific rules will be instantiated as OpenFlow rules. A similar method is also utilized by Liu *et al.* [228].

Samadi *et al.* [229] design a converged inter/intra-data-center architecture in the metro-scale distance. It is carried out by combining Optical Space Switches (OSS) with SDN. OSS acts as the Top-of-Rack (ToR) switch and SDN intelligently manages the network in a data center. In further, different data centers in a metro region are managed by a unified SDN control plane. Their experiment shows live VM migration among 3 data centers over 50km.

Shen *et al.* [178] keep the network connection during VM migration by creating a SDN overlay based on Open vSwitch, VXLAN tunnels and the Frenetic SDN controller. The network architecture is: Open vSwitch implements data plane, each virtual switch connects to a VM, a set of VXLAN tunnels are created to connect all switches, a gateway switch is built for each data center to implement directly tunnels with other switches in other data centers, and all switches are connected to a centralized SDN controller which is implemented with Frenetic. With this structure, the traffic of the migrated VM can be easily redirected to the destination data center after migration. To decrease the time needed for network reconnection, the controller injects a preparation rule to the destination switch to get the migrated VM's MAC address on the destination host as soon as possible. It also only updates the switches which have the migrated VM's MAC address in its cache table to avoid APR broadcast.

E. Summary of Network Mobility Mechanisms

The mechanisms available for keeping network connections of VM migration are summarized in TABLE VI. Every solution has both advantages and disadvantages. Layer-2 solutions can preserve the network configurations of the migrated VM after migration, which is beneficial to reduce migration downtime. However, they may result in a long routing delay.

TABLE VI
THE SUMMARY OF NETWORK MOBILITY MECHANISMS

Layer	Mechanism	Reference	Technique
2	LAN extension	[30], [182]	Cisco DCI
		[183]	Cisco OTV
	VPN	[37]	MPLS, VPLS
		[162], [163]	-
	Overlay networks	[184]	-
		[185], [186]	IPOP
		[188]	VAN
[191]–[193]		SPB,PBB,TRILL	
3	Tunneling	[111], [148], [152]	IP tunneling
		[38], [197]	Dynamic DNS
		[43]	F5 iSessions tunneling
	Mobile IP	[197]	IP tunneling
		[201]	HyperMIP
		[202]	Mobile IPv6
		[206], [207]	MAT
	[208], [209]		
	PMIPv6	[204], [205]	-
	LISP	[210]–[212]	-
NDN	[213]	-	
4	TCP <i>Migrate</i> option	[215]	Dynamic DNS
	Network failover	[216]	Session synchronization protocol Connection resumption mechanism
	TLM	[217]	TCP Splice
	Isolation boundary	[218]	TI flow
	Network virtualization	[220]–[222]	VXLAN,STT,NVGRE
-	SDN	[224]	PMAC, PIP
		[227], [228]	Two levels of rules
		[229]	OSS
		[226]	Topology-adaptive DCN
		[178]	Open vSwitch, VXLAN

Also, Layer-2 solutions have a high deployment effort. Layer-3 solutions are featured by slow route-changing speed (such as dynamic DNS) or residual dependency on the source site (such as Mobile IP). Layer-4 solutions are not bothered by those problems. However, they are accompanied by a high implementation complexity, such as changing communication protocol. In contrast, SDN is the most promising one due to its flexibility. Actually, which mechanism will be used during VM migration significantly depends on the underlying network infrastructures and topology of the source and the destination data centers. For example, if a data center is running based on SDN, it is unnecessary to use Mobile IP to keep network connection.

VII. MIGRATION PERFORMANCE ANALYSIS

VM migration performance will be impacted by a variety of factors. In this section, we review the literature focusing on exploring the relationships between migration performances and the related factors. We sum up them as follows:

- *Memory access pattern*: It indicates how the migrated VM accesses its memory space, such as memory dirtying rate, the size of WWS. This is the most critical factor for memory data migration performance. Especially, memory dirtying rate determines whether the iteration phase of pre-copy can find a proper termination point.
- *Migration bandwidth*: It is the bandwidth between the source server and the destination server. The migration in LAN environments has a much higher bandwidth than that in WAN environments.

- *Workload feature*: As discussed above, the applications running in the migrated VM influence on migration performances. For example, the applications can be I/O-intensive, network-intensive, or CPU-intensive. They will compete for the corresponding resources with the migration process.
- *Host performance*: The performance and available resources of the source and the destination servers are also important for migration performance, since the migration processes are normally running on them.
- *Co-located VMs*: This parameter refers to how many VMs are co-located on the source and the destination servers and how many hardware resources have been consumed by them. Virtualization is only to isolate resources but not performance, the VMs running on a same server will impact the performance of each other (including the migration process).
- *Migration pattern*: Different migration patterns will result in different migration performances. In addition, different optimization strategies also introduce different amount of benefits and overheads.
- *VM configuration*: This denotes the static parameters of a VM, such as memory size, used memory space, virtual disk size, virtual disk format, network buffer size, and so on.
- *Disk I/O pattern*: Similar with memory access pattern, it will influence on the performance of storage data migration because dirtied disk blocks will be retransmitted.
- *Migration sequence*: When a plenty of VMs will be migrated, sequential or parallel migration will lead to different performances.

- *VMM*: Although pre-copy migration strategy is widely used in VMMs, the implementations in different VMMs are slightly different, which is followed by different migration performances.
- *SLA*: SLA indirectly imposes restrictions on migration performances. For example, it sets the maximum downtime a VM can experience within a duration.

A. Practical Test

Some researchers evaluate migration performance by directly varying the factors listed above. Zhao and Figueiredo [230] test the performances of non-live migration. Both single and multiple migrations are evaluated with different configurations. They find that: (1) The migration time for a number of VMs is predictable based on the migration time of a single VM. (2) The interruption time to the services running in the migrated VM is longer than the total migration time, i.e., the VM needs some time to recover back to its original performance after migration. (3) Migrating multiple VMs in parallel improves the utilization of network bandwidth.

Hu *et al.* [231] build an automatic testing framework for evaluating migration strategies. With this framework, they evaluate and compare the migration performances (total migration time, downtime, total network traffic) of four mainstream virtualization platforms (VMware, Xen, Hyper-V, KVM) with different parameters. Both storage data migration and non-live migration are tested with their framework. However, their testing system utilizes a dedicated link for migration process, which is uncommon in real data centers.

Salfner *et al.* [232] explore how total migration time and downtime are influenced on by: CPU load, memory size, used memory size, and memory access pattern. They firstly try to find out the variables which do not significantly affect migration performances with single variable experiments (only one parameter is changing in each test). According to the results, configured memory size, used memory size and memory dirtying rate are the three main factors affecting memory data migration performances. They then test how these factors jointly influence on migration performances on Xen and VMware platforms. Li *et al.* [233] explore the influences of bandwidth limit, TCP buffer size, maximum downtime and VM properties on migration performance (total migration time and downtime) and energy consumption during VM migration on a KVM platform. Based on these experimental results, they further develop an analytical model to formulate the relationship between them.

Voorsluys *et al.* [18] aim at evaluating the performance interruption of live VM migration to the applications (especially Internet applications) running inside the migrated VM. Strunk and Dargie [234] test the energy cost resulting from live VM migration on KVM platform. They find that the energy consumption is proportional to the total migration time. Bezerra *et al.* [235] evaluate the overheads resulting from live VM migration from clients' perspective on both real and virtual environments. They find that a client can observe significant performance degradation during VM migration.

B. Performance Model

Some researchers reflect the relationships between migration performance and influence factors with mathematical models. Zhang *et al.* [236] create models to minimize the bandwidth consumption during pre-copy phase with a given maximum bandwidth to satisfy a targeted total migration time and a targeted downtime. The models are created on the basis of three different conditions: (1) the dirtying probability of all pages is deterministic; (2) the dirtying probability follows the bernoulli distribution [237]; (3) a reciprocal-based dirtying model which is achieved by the statistics of some typical workloads.

Liu *et al.* [21] build models for both migration performance and energy cost during migration. The models for migration performance are derived based on uniform memory dirtying rate and refined by using real-time memory dirtying rate which is achieved by the exponential weighted moving average (EWMA). The models for the energy consumption during VM migration in both wired and wireless network environments are designed by utilizing linear regression technique and Shannon's theorem, respectively. They further elaborate how to combine these models to guide migration decision to minimize the cost. Basu *et al.* [238] also create cost models for energy consumption and SLA violation during live VM migration. They formulate the problem of energy- and performance-efficient resource management during live VM migration as a reinforcement learning problem and propose an algorithm to learn the uncertainty and dynamic of workload as-it-goes to address three issues: when to start migrating a VM, which VM to migrate and where to migrate a VM to.

Xu *et al.* [17] show that live VM migration will introduce interferences to both the VMs on the source and the destination servers based on practical experiments. They classify these interferences into two categories: *migration interference* and *co-location interference*. Migration interference denotes the interruptions to the migrated VM and other VMs running on the source and the destination servers, and co-location interference is the service degradation of the VMs on the destination server after a new VM is migrated in. They design a *demand-supply* model for each of the two interferences. The resource requirements (such as CPU, memory, network bandwidth) from the management VM and other VMs on a host are *demand*, and the resources the physical host can provide are *supply*. Breitgand *et al.* [239] focus on the service performance degradation during migration. They relate this degradation to the residual network bandwidth for the services running in the migrated VM. They define migration cost as the portion of client's requests which are not served within the deadline stated in SLA. They build performance models for two conditions: fixed migration bandwidth and variable migration bandwidth. Based on these models, they design a bandwidth allocation algorithm to minimize total migration time and maintain an acceptable QoS as well.

Another method used to create performance models for VM migration is regression technique [240]. With this method, performance models are derived based on the statistics from extensive experiments. Galloway *et al.* [241] focus on the relationship between VM migration and service degradation. They

test the interruptions of VM migration on several type of VMs which are running different workloads. Wu and Zhao [242] try to understand the relationship between resource allocation and migration performance. The allocated resource mainly refers to the CPU cycles reserved in VMM for migration. Utilizing regression technique to analyzing migration performance is simple, but it suffers from a limited applicability. When the migration environment and parameters change, the models become useless. Rybina *et al.* [243] derive models for the total migration time by using simple and multiple linear regressions. They observe that the total number of retired CPU instructions, the total number of L3 cache misses and the number of dirty pages in the source server during migration have a significant influence on total migration time. They also find that considering more than five parameters has a little help for the strength of the models. Based on the experimental data in his previous study [234], Strunk [244] also derives an energy cost model for live VM migration by using linear regression.

Xia *et al.* [245] focus on the VM replacement problem when VM workloads or the resources of a switch have been changed in the NFV environments. They build a migration cost model for this scenario by taking the constraints of switch resources and the link delay between different switches into consideration, to solve the problem of which VM will be migrated to which node.

Sometimes multiple migration is inevitable, such as load balancing and server consolidation. Also, in a large data center, users send migration requests randomly, so concurrent VM migration is conducted at any time as well. Kikuchi and Matsumoto [246] build a performance model for concurrent VM migration in a data center to guide a better cloud management. They firstly collect a set of performance data from a practical experiment, and then use the PRISM probabilistic model checker [247] for model creation based on the collected data.

When a group of correlated VMs are migrated concurrently over WAN, a smaller service degradation can be achieved by handing them over to the target site at the approximate time to shorten the time of remotely communicating with each other. Liu and He [126] struggle for this goal by designing a bandwidth allocation strategy. They abstract this issue as a distributed constraint optimization problem (DCOP). A model for static workloads which have stable memory dirtying rate is firstly proposed. Then the model for dynamic workloads which are with a changing memory dirtying rate is gained by cutting the migration window into small pieces and regarding the workload as static within each time piece. Cerroni [45], [172] creates models for multiple migration under the cloud federation scenario regarding total migration time and downtime. Based on these models, he analyses the influences of multiple VM migration on the inter-data center network capacity in cloud federation. His finding can guide the provision of inter-data-center network capacity to achieve a given performance level.

C. Prediction Model

It is also valuable if the migration performance is predictable. For planned migration, we can make migration

decisions according to predicted migration performances. For unplanned migration, we can foresee the remaining migration time and possible influences. Migration performance prediction can be used to estimate the QoS of a data center as well. Akoush *et al.* [144] calculate the upper and lower migration performances by taking the migration termination conditions of Xen as an example. The results illustrate that the wide bound between the upper and the lower performances is not applicable to predict the real migration performance. Therefore, they design two simulation models to predict the performance (total migration time and downtime) of pre-copy migration pattern, AVG (average page dirty rate) and HIST (history based page dirty). The AVG model is to predict the migration performance of the VM which has a constant memory dirtying rate, while the HIST model is designed for the VM which is with similar memory behaviors between different runs (e.g., a MapReduce workload). They classify the parameters affecting the migration performance into *static* parameters (such as, memory size, VM resumption time) and *dynamic* parameters (such as, migration bandwidth, memory dirtying rate).

Nathan *et al.* [248] create performance prediction models for pre-copy with page-skipping optimization. They do so by firstly concluding and classifying the existing estimation models, and then empirically evaluating their correctness with experiments. They find that all of them are accompanied by high prediction error ratios. After analyzing the causes of the errors, they propose models by taking two factors into consideration: (1) the unique pages dirtied, i.e., the pages will be transferred in each iteration after the optimization of skip technique; (2) the number of skipped pages in each iteration. They monitor these two factors and then integrate them with their models. However, their models only consider the page-skipping optimization technique (KVM does not support this technique). Actually, some other techniques (such as, memory ballooning, dynamic bandwidth limiting) occupied by VMM also will influence on the migration time and then the prediction correctness.

Aldhalaan and Menascé [249] design several analytic models to predict migration performance (total network traffic, downtime, network utilization) under three conditions: (1) uniform memory dirtying rate, (2) hot pages are copied during the pre-copy phase, (3) hot pages are copied during the downtime phase. They find that the remaining data size α for terminating pre-copy phase is a critical parameter for downtime and network utilization. They further formulate the issue of choosing an appropriate value of α to both minimize the downtime and improve network utilization as a non-linear optimization problem.

Salfner *et al.* [250] build a model to predict the worst-case performances of live VM migration. In their models, workload feature and host behavior are considered. They further confirm that memory access pattern is the main factor impacting the total migration time and downtime. They verify their models on different virtualization platforms (VMware, Xen, and KVM) with both artificial and real workloads.

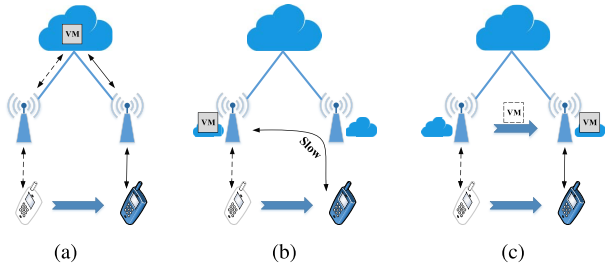


Fig. 10. The comparison between VM migration in cloud computing and MEC. (a) Cloud computing, (b) No VM migration in MEC, (c) VM migration in MEC. The double-headed lines denote the communication between a UE and its VM.

D. Summary of Performance Analysis Studies

The studies on VM migration performances are summarized in TABLE VII. The migration pattern in the table follows the same rules as TABLE V. From the table, we can see that: (1) The majority of performance studies focus on: single migration in LAN environments with pre-copy migration pattern. No performance model is designed for post-copy and hybrid-copy patterns. (2) Total migration time, downtime, and service interruption are the main concerns of migration performance, as discussed in Section II-D. (3) Additionally, from the considered factors in these models, we can observe that the mostly considered parameters for memory data migration are: workload feature, VM configuration, memory access pattern, migration bandwidth. (4) Different from the studies on designing migration strategies for memory and storage data, the migration performances of some commercial virtualization platforms (VMware, Hyper-V) are also evaluated or utilized to create or evaluate models.

VIII. USER MOBILITY-INDUCED VM MIGRATION

MEC has two basic differences from conventional cloud computing: (1) an edge cloud data center is close to users (sometimes only one-hop away); (2) users are with a high mobility. Conventional cloud computing denotes that users offload tasks to a remote central cloud data center (as shown in Fig. 10(a)). These differences introduce new problems for live VM migration in MEC besides solving the three challenges of the migration in WAN environments. For example, when a user moves out the coverage area of the current edge cloud data center, two options are available for how to tackle with the corresponding VM. The first one is that the VM is not migrated and the user remotely accesses it (as shown in Fig. 10(b)) [251]. The second one is that the VM is migrated to the edge cloud data center where the user is currently at (as shown in Fig. 10(c)). These options incur a trade-off between migration costs and gains to decide whether a VM will be migrated or not. In this section, we will review the works on live VM migration in MEC from two perspectives: *migration performance improvement* and *migration performance analysis*. At last, these studies are summarized and discussed with the metrics extracted from the literature.

1) *Migration Performance Improvement*: Ha *et al.* [252] create a VM in an edge cloud data center by generating a

overlay which is the compressed binary difference between a base VM image and the launch VM image (both memory and disk data). Based on this VM provisioning manner, they only migrate the overlay while migrating a VM with the assumption that the base image is present at the destination site [253]. In further, a pipeline of processing stages (delta-encoding, deduplication, compression) are designed to reduce the overlay size. The settings of these stages are adaptively configured by dynamically monitoring the migration performance to achieve the shortest total migration time.

Taleb *et al.* [254], [255] proposed a Follow-Me-Cloud (FMC) concept, which aims to ensure a UE to always be connected to and served by the optimal data center. To achieve this goal, they introduce two additional components to the conventional network architecture, namely FMC controller and data center/gateway (DC/GW) mapping entity. The DC/GW mapping entity is to update the network anchor point of a UE while it is moving from location 1 to location 2. The FMC controller is in charge of selecting the optimal data center according to a UE's current location and migrating the UE's service to this data center. Based on this architecture, several mechanisms are designed to guarantee the service continuity during migration, such as replacing data anchoring at the network layer by service anchoring, replacing IP addressing by service/data identification, etc.

Teka *et al.* [256] solve the network connection problem when a UE is moving between edge cloud data centers by using Multipath TCP (MPTCP) [257]. Each VM in an edge cloud data center is assigned two network interfaces. One is operated in the regular VM operation mode, and the other is activated only after the VM is migrated. Similarly, MPTCP is utilized by Qiu *et al.* [258] between two edge cloud data centers to improve the connection fault tolerance and reduce migration time.

To both improve VM migration speed and reduce the cost of improper VM migration due to the uncertainty of user mobility, Saurez *et al.* [259] choose the target node according to the workload running on the node and initialize VM migration according to the latency between a VM and its user. A VM is migrated in two steps: (1) when the latency is bigger than a threshold, all persistent data (such as disk data) generated by the VM is migrated to the target node, called *state migration*; (2) when the latency continues to increase and becomes bigger than another bigger threshold, the remaining data of the VM is migrated, called *computation migration*.

Secci *et al.* [260] link VM mobility to user mobility by leveraging one of the following three options: (1) moving the VM to an edge cloud data center which is closer to the user; (2) just switching the data-center routing locator to lower the network access latency; or (3) performing both the previous operations. These operations also must be transparently conducted with respect to the underlying network infrastructure. To these ends, they propose a cloud access overlay protocol architecture based on LISP protocol. A controller is introduced to adaptively determine the best entry DC and the best VM location on a per-user basis.

Ottenwälder *et al.* [261], [262] propose a placement and migration method for the VMs of a mobile Complex Event

TABLE VII
THE SUMMARY OF STUDIES ON MIGRATION PERFORMANCE ANALYSIS. THE MIGRATION
PATTERN IN [45] AND [172] IS FOR MEMORY DATA MIGRATION

Ma.	Ref.	G	Env.	VMM	Pattern	EPM	Factors
PT	[230]	S&M	LAN	VMware	Non-live	I, T	VMC, WF, MG, MS
	[232]	S	LAN	VMware Xen	Pre	T, D	WF, VMC, MAP, VMM
	[233]	S	LAN	KVM	Pre	T,D,EC	MB,TCPB,MD,VMC,SLA
	[231]	S	LAN	VMware Xen Hyper-V KVM	Pre Non-live	T, D, N	VMC, VMM, WF, MAP
	[18]	S&M	LAN	Xen	Pre	I	WF, SLA
	[234]	S	LAN	KVM	Pre	EC	VMC,MB
	[235]	S	LAN	Xen	Pre	I	WF
PM	[21]	S	LAN	Xen	Pre	T, D, N, EC	VMC, WF, MB
	[238]	S	LAN	-	Pre	EC,RE	WF
	[244]	S	LAN	KVM	Pre	EC	VMC,MB
	[239]	S	LAN	Xen	Pre	T, I	MB, MAP, SLA
	[17]	S	LAN	Xen	Pre	MI, CoI	VMC, WF, CVM, HP
	[236]	S	LAN	Xen	Pre	T, D	MAP
	[242]	S	LAN	Xen	Pre	T	WF, HP
	[243]	S	LAN	KVM	Pre	T	MB,VMC,MAP,HP
	[241]	S	LAN	Xen	Pre	I	WF
	[245]	S	WAN	-	-	I	WF,VMC,HP,ND
	[246]	M	LAN	Xen	Pre	T	VMC, MRD
	[126]	M	WAN	Xen	Pre-Pre	I	MAP, MB
	[45], [172]	M	WAN	-	Pre	T, D	MS, VMA, MB VMC, MAP
PrM	[144]	S	LAN	Xen	Pre	T, D	VMC, MB, MDR
	[249]	S	LAN	-	Pre	D, N, NU	VMC, MAP, MP
	[250]	S	LAN	VMware Xen KVM	Pre	T, D	VMC, MAP, MB, VMM, CVM
	[248]	S	LAN	Xen KVM	Pre	T, D, N	VMC, MB, MAP, VMM

The abbreviation list: *Ma.*: evaluation manner. *Ref.*: references. *G*: granularity (*S*: single migration. *M*: multiple migration). *Env.*: migration environment. *EPM*: evaluated performance metric. *Factors*: the factors considered in the test or the model. *PT*: practical test. *PM*: performance model. *PrM*: prediction model. *I*: service interruption. *T*: total migration time. *D*: downtime. *N*: total network traffic. *NU*: network utilization. *MI*: migration interruption. *CoI*: co-located interruption. *EC*: energy cost. *RE*: resource efficiency. *WF*: workload feature. *VMC*: VM configuration. *CVM*: co-located VMs. *MAP*: memory access pattern. *MG*: migration granularity. *MDR*: memory dirtying rate. *MB*: migration bandwidth. *MS*: migration sequence. *MRD*: migration request distribution. *HP*: host performance. *VMA*: VM amount. *VMM*: virtual machine manager. *SLA*: service level agreements. *TCPB*: TCP buffer size. *MD*: maximum downtime. *ND*: network delay.

Processing system which is running on infrastructures which contain both cloud and fog resources. They take both the gains and the costs of VM migration into consideration. By using the predicted future movement of users, they design a migration plan for each VM in advance from a systematic perspective to lower the overall network utilization and meet the user-defined latency requirements.

Sun and Ansari [263] propose to place a number of replicas for each VM virtual disk at several edge cloud data centers which are selected by a Latency Aware Replica placement (LEARN) algorithm. With this manner, only the newly dirtied disk blocks need to be migrated to the destination site during VM migration. However, it incurs a big storage space consumption and a big overhead on VM image management. Machen *et al.* [264] store VM images in layers (similar to [156] and [160]) and use cloning and incremental synchronization to recreate the missing layers of the migrated VM at the destination site.

2) *Migration Performance Analysis*: Some studies mainly focus on quantitatively analyzing the VM migration procedure in MEC, especially the costs and the benefits. They are critical for the design of a better migration strategy.

Gkatzikis and Koutsopoulos [265] list the parameters which may influence on the efficiency of migration mechanisms in MEC, such as workload uncertainty, unpredictability of multi-tenancy effects, unknown evolution of accompanying data volume, etc. They propose that migration strategies can be made at three different levels: cloud-wide migration policy, server-initiated migration policy and task-initiated migration policy.

Ksentini *et al.* [266] formulate the migration procedure in FMC as a Markov Decision Process (MDP). To solve the trade-off between migration costs and gains, a decision policy is proposed for whether to migrate a service when a UE is at a certain distance from the source DC. Wang *et al.* [267] also model the migration procedure as a Markov Decision Process (MDP). But they only consider the situation where UEs follow a one-dimensional asymmetric random walk mobility model. Under this situation, the optimal policy for VM migration is a threshold policy. Then they propose an algorithm for finding the optimal thresholds. In comparison with [266], they design an optimal threshold policy to find the optimal action of the MDP. Taleb and Ksentini [268] use Markovian models to analyze the performance of FMC. The evaluated metrics include the probability of a user to be always served by the optimal

TABLE VIII
THE SUMMARY OF USER MOBILITY-INDUCED MIGRATION TECHNOLOGIES

Perspective	Reference	Main optimization	Others	Benefit			Cost		
				T	N	I	Co	Ne	Sp
PI	[252]	VM overlay	DE,DD,DC	✓	✓		✓		
	[254], [255]	FMC	DC/GW			✓	✓		
	[256]	MPTCP	-			✓		✓	
	[258]	MPTCP	-	✓		✓		✓	
	[259]	Two-stage migration	-	✓		✓		✓	✓
	[260]	Cloud access overlay protocol	-			✓		✓	
	[261], [262]	Migration plan	-	✓	✓				
	[263]	Disk replicas	-	✓			✓	✓	✓
	[264]	Layered image structure	-	✓	✓				
Perspective	Reference	Main mechanism or model	Others	CIF			PM		
PA	[265]	Three-level migration strategies	IF	WL,MLT,DS,BW,DCC			T,I		
	[266]	MDP	DP	UMF,BW,DCU			MG,MC		
	[267]		TP	UMF			MG,MC		
	[268]		-	UMF			T,ADCU		
	[269]	Performance estimation schemes	MMS,MDP	UMF,DS			I,MC		
	[270]	General performance model	VMP	BW,T,DCU			MG,MC		

The abbreviation list: *PI*: performance improvement. *PA*: performance analysis. *Others*: other mechanisms proposed in the paper. *T*: total migration time. *N*: network traffic. *I*: service interruption. *Co*: computational overhead. *Ne*: network overhead. *Sp*: space overhead. *CIF*: considered influence factors. *PM*: performance metrics. *DE*: delta-encoding. *DD*: data deduplication. *DC*: data compression. *DC/GW*: data center/gateway (DC/GW) mapping entity. *IF*: influence factors. *DP*: decision policy. *TP*: threshold policy. *MMS*: migration management scheme. *VMP*: VM placement strategy. *WL*: workload type. *MLT*: multitendency effect. *DS*: data size. *BW*: network bandwidth. *DCC*: data center capacity. *DCU*: the distance between data center and users. *MG*: migration gain. *MC*: migration cost. *UMF*: user mobility feature. *ADCU*: average distance between a user to the optimal DC.

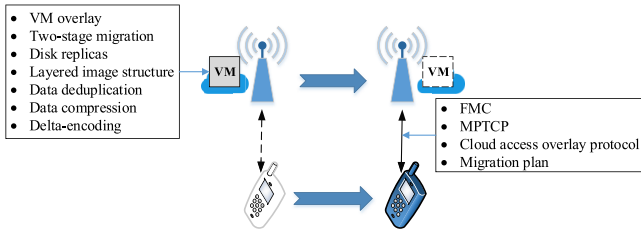


Fig. 11. The summary of migration optimization technologies in MEC.

DC, the average distance from the optimal DC and the cost of VM migration.

Nadembega *et al.* [269] propose two estimation schemes for live VM migration in MEC. (1) Data transfer throughput estimation scheme, i.e., the data flow size between a UE and its VM during VM migration. (2) VM handoff time estimation scheme. With the support of these two schemes, a VM migration management scheme is further proposed. Sun and Ansari [270] create models for the gains and costs of VM migration in MEC, respectively. Based on these two models, they design a VM placement strategy to maximize the profit of live VM migration which is the difference between migration gains and costs.

3) *Summary of User Mobility-Induced Migration Technologies*: The studies on user mobility-induced migration technologies are summarized in TABLE VIII. From the perspective of improving migration performance, the optimizations can be divided into two categories: (1) reducing the data transferred during VM migration and (2) keeping a smooth connection between the migrated VM and the moving user, as shown in Fig. 11. Regarding the first one, the proposed technologies include VM overlay [252], two-stage migration [259], disk replicas [263], layered image structure [264], data deduplication, compression, delta-encoding [252]. Those for the second issue contain

FMC [254], [255], MPTCP [256], cloud access overlay protocol [260], migration plan [261], [262].

As discussed before, the researches on migration performance mainly concentrate on analyzing the trade-off between migration gains and costs, which is critical for migration decision making and destination site selection. MDP [266]–[269] is widely used to formulate the procedure of live VM migration in MEC where both users and VMs may change their locations.

IX. OPEN RESEARCH ISSUES

Even though VM migration has been thoroughly studied, there still are some issues which remain unsolved or need further improvement. Meanwhile, the development of cloud computing introduces many new challenges for VM migration. For example, across-data-center management is required in cloud federation. We list some of these issues in this section.

Optimal and adaptive termination conditions for pre-copy: Currently, the iteration phase of pre-copy is terminated when some predefined conditions are met. However, VMs have different configurations and the applications running inside have different memory access patterns as well. It is hard to stop the iteration phase at the right time with these preset static conditions, which in turn leads to a big service degradation and downtime. Some more optimal termination approaches are needed. They should be adaptive to the memory access patterns of the migrated VM and can be tuned along with the migration progress.

Improving the robustness of post-copy and hybrid-copy: As discussed in Section IV-A, post-copy and hybrid-copy is more optimal regarding migration performance than pre-copy. In particular, post-copy can give clients the impression of instant migration accomplishment. Even though pre-copy is overwhelmingly employed by VMs at present, its performances are strongly related to the workloads running in the

migrated VM. In order to make the migration convergent, we have to compromise QoS through some strategies, such as, write throttling and CPU frequency tuning. Therefore, if some mechanisms are designed to improve the robustness of post-copy and hybrid-copy, we can benefit from other advantages of them (fast handover, less migration traffic, short migration time).

Quantitative analysis of WAN migration: From the Section VII, we can notice that the relationships between migration performance and influence factors for LAN migration attract almost all researchers' attentions. Nevertheless, those for WAN migration are rarely analyzed. With the development of cloud computing, migrating VMs across data centers is an inevitable trend. Quantitatively understanding the issues in VM migration over WAN is critical to design better migration mechanisms. For example, it is obvious that WAN migration performances may suffer from fast memory dirtying rate, slow migration bandwidth, and big virtual disk size. *How and how much do these factors individually and interactively influence on migration performances?* Also, as described in Section V-A, there are nine options for WAN migration by combining memory migration patterns with storage migration patterns. *Which migration pattern is better? Which factors are the critical ones for the migration performance of each pattern?* All these questions need to be further studied.

Multiple migration: When correlated VMs are migrated between the servers within a data center, they still are reachable for each other. However, if we migrate some of them to another data center, the slow network bandwidth between these two data centers will severely degrade the communication efficiency between them, even destroys the running services. There are already some achievements on multiple migration over WAN, but many issues still remain unsolved. For example, current studies mainly concentrate on the communication between the migrated VM and its clients, and rarely on the communications between VMs, especially in the NFV environments where the virtual network function instances (VNFI) in a service function chain are closely connected with each other.

Migration security: Same as the problems of multiple migration, the security issue will not bother the migration within a data center because it happens behind the firewall and the migration link is trusty. However, the Internet is unauthentic, so the migration between data centers face a high security risk. Oberheide *et al.* [271] classify the threats in VM migration into three classes: control plane, data plane, and migration module. (1) Control plane threat. An attacker may take over the right of a VMM and control the whole migration process. (2) Data plane treat. If the migration path is not secured and trusted, the data and states of the migrated VM may be snooped and tampered. (3) Migration module treat. The bugs in the migration program itself will leave some attack points to attackers. Currently, it is lack of comprehensive security strategies to guarantee the safety of VM migration over WAN.

X. CONCLUSION

VM migration offers many benefits to cloud providers and users. It lays the base for the majority of cloud management

tasks, such as load balancing, hardware maintenance, etc. The migration across data centers further improves the mobility of VMs and breaks the vendor lock-in issue. A plenty of researchers struggle for improving VM migration performances since this concept was proposed. In this paper, we begin with comprehensively introducing the basic knowledge on VM migration, including its advantages for cloud management and development, main challenges, and the performance criteria for evaluating a migration strategy. We propose a classification for migration schemes and describe the structure theory of this survey based on this classification.

We firstly depict the studies on non-live VM migration. Then we review the strategies and optimization mechanisms for live VM migration from the perspective of the three challenges it faces: memory data migration, storage data migration, and network connection continuity. We also sum up the studies which focus on understanding the relationship between migration performances and influence factors. These works are helpful for guiding the design of optimal migration strategies and optimization technologies. The researches on user mobility-induced VM migration are also reviewed. All the reviewed technologies in this paper are compared with the metrics extracted from the literature. At last, the open research issues on VM migration are discussed.

REFERENCES

- [1] N. Huber, M. von Quast, M. Hauck, and S. Kounev, "Evaluating and modeling virtualization performance overhead for cloud environments," in *Proc. CLOSER*, 2011, pp. 563–573.
- [2] C. Develder *et al.*, "Optical networks for grid and cloud computing applications," *Proc. IEEE*, vol. 100, no. 5, pp. 1149–1167, May 2012.
- [3] L. Wang *et al.*, "Scientific cloud computing: Early definition and experience," in *Proc. IEEE 10th Int. Conf. High Perform. Comput. Commun. (HPCC)*, Dalian, China, 2008, pp. 825–830.
- [4] T. Mather, S. Kumaraswamy, and S. Latif, *Cloud Security and Privacy: An Enterprise Perspective on Risks and Compliance*. Beijing, China: O'Reilly Media, 2009.
- [5] G. Boss, P. Malladi, D. Quan, L. Legregni, and H. Hall, "Cloud computing," White Paper, IBM, Armonk, NY, USA, vol. 321, pp. 224–231, 2007.
- [6] A. Weiss, "Computing in the clouds," *Networker*, vol. 11, no. 4, pp. 16–25, Dec. 2007.
- [7] L. Cheng, I. Tachmazidis, S. Kotoulas, and G. Antoniou, "Design and evaluation of small-large outer joins in cloud computing environments," *J. Parallel Distrib. Comput.*, vol. 110, pp. 2–15, Dec. 2017.
- [8] J. Wu, S. Guo, J. Li, and D. Zeng, "Big data meet green challenges: Greening big data," *IEEE Syst. J.*, vol. 10, no. 3, pp. 873–887, Sep. 2016.
- [9] S. Osman, D. Subhraveti, G. Su, and J. Nieh, "The design and implementation of Zap: A system for migrating computing environments," *ACM SIGOPS Oper. Syst. Rev.*, vol. 36, no. SI, pp. 361–376, 2002.
- [10] J. G. Hansen and A. K. Henriksen, "Nomadic operating systems," M.S. thesis, Dept. Comput. Sci., Univ. Copenhagen, Copenhagen, Denmark, 2002.
- [11] P. Padala, X. Zhu, Z. Wang, S. Singhal, and K. G. Shin, "Performance evaluation of virtualization technologies for server consolidation," HP Labs, Palo Alto, CA, USA, Rep. HPL-2007-59, 2007.
- [12] L. Cheng and T. Li, "Efficient data redistribution to speedup big data analytics in large systems," in *Proc. IEEE 23rd Int. Conf. High Perform. Comput. (HiPC)*, Hyderabad, India, 2016, pp. 91–100.
- [13] R. Bianchini and R. Rajamony, "Power and energy management for server systems," *Computer*, vol. 37, no. 11, pp. 68–76, Nov. 2004.
- [14] J. Wu, "Green wireless communications: From concept to reality [industry perspectives]," *IEEE Wireless Commun.*, vol. 19, no. 4, pp. 4–5, Aug. 2012.
- [15] J. W. Jiang, T. Lan, S. Ha, M. Chen, and M. Chiang, "Joint VM placement and routing for data center traffic engineering," in *Proc. IEEE INFOCOM*, Orlando, FL, USA, 2012, pp. 2876–2880.

- [16] F. Xu, F. Liu, H. Jin, and A. V. Vasilakos, "Managing performance overhead of virtual machines in cloud computing: A survey, state of the art, and future directions," *Proc. IEEE*, vol. 102, no. 1, pp. 11–31, Jan. 2014.
- [17] F. Xu *et al.*, "iAware: Making live migration of virtual machines interference-aware in the cloud," *IEEE Trans. Comput.*, vol. 63, no. 12, pp. 3012–3025, Dec. 2014.
- [18] W. Voorsluys, J. Broberg, S. Venugopal, and R. Buyya, "Cost of virtual machine live migration in clouds: A performance evaluation," in *Proc. IEEE Int. Conf. Cloud Comput.*, Beijing, China, 2009, pp. 254–265.
- [19] C. Clark *et al.*, "Live migration of virtual machines," in *Proc. 2nd Conf. Symp. Netw. Syst. Design Implement.*, vol. 2, 2005, pp. 273–286.
- [20] A. J. Mashtizadeh *et al.*, "XvMotion: Unified virtual machine migration over long distance," in *Proc. USENIX Annu. Tech. Conf.*, Philadelphia, PA, USA, 2014, pp. 97–108.
- [21] H. Liu, H. Jin, C.-Z. Xu, and X. Liao, "Performance and energy modeling for live migration of virtual machines," *Cluster Comput.*, vol. 16, no. 2, pp. 249–264, 2013.
- [22] V. Medina and J. M. García, "A survey of migration mechanisms of virtual machines," *ACM Comput. Surveys*, vol. 46, no. 3, p. 30, 2014.
- [23] R. W. Ahmad *et al.*, "Virtual machine migration in cloud data centers: A review, taxonomy, and open research issues," *J. Supercomput.*, vol. 71, no. 7, pp. 2473–2515, 2015.
- [24] P. Kokkinos, D. Kalogerias, A. Levin, and E. Varvarigos, "Survey: Live migration and disaster recovery over long-distance networks," *ACM Comput. Surveys*, vol. 49, no. 2, p. 26, 2016.
- [25] R. W. Ahmad *et al.*, "A survey on virtual machine migration and server consolidation frameworks for cloud data centers," *J. Netw. Comput. Appl.*, vol. 52, pp. 11–25, Jun. 2015.
- [26] P. G. J. Leelipushpam and J. Sharmila, "Live VM migration techniques in cloud environment—A survey," in *Proc. IEEE Conf. Inf. Commun. Technol. (ICT)*, 2013, pp. 408–413.
- [27] A. Strunk, "Costs of virtual machine live migration: A survey," in *Proc. IEEE 8th World Congr. Services (SERVICES)*, Honolulu, HI, USA, 2012, pp. 323–329.
- [28] D. Kapil, E. S. Pilli, and R. C. Joshi, "Live virtual machine migration techniques: Survey and research challenges," in *Proc. IEEE 3rd Int. Adv. Comput. Conf. (IACC)*, 2013, pp. 963–969.
- [29] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the Internet of Things," in *Proc. 1st Ed. MCC Workshop Mobile Cloud Comput.*, Helsinki, Finland, 2012, pp. 13–16.
- [30] *Virtual Machine Mobility With Vmware Vmotion and CISCO Data Center Interconnect Technologies*. Accessed: Mar. 2017. [Online]. Available: http://www.cisco.com/c/dam/en/us/solutions/collateral/data-center-virtualization/data-center-virtualization/white_paper_c11-557822.pdf
- [31] L. Youseff, M. Butrico, and D. Da Silva, "Toward a unified ontology of cloud computing," in *Proc. Grid Comput. Environ. Workshop (GCE)*, Austin, TX, USA, 2008, pp. 1–10.
- [32] J. Hu, J. Gu, G. Sun, and T. Zhao, "A scheduling strategy on load balancing of virtual machine resources in cloud computing environment," in *Proc. 3rd Int. Symp. Parallel Archit. Algorithms Program. (PAAP)*, Dalian, China, 2010, pp. 89–96.
- [33] Z. Chaczko, V. Mahadevan, S. Aslanzadeh, and C. Mcdermid, "Availability and load balancing in cloud computing," in *Proc. Int. Conf. Comput. Softw. Model.*, vol. 14, Singapore, 2011, pp. 134–140.
- [34] Z. Zhang *et al.*, "VMthunder: Fast provisioning of large-scale virtual machine clusters," *IEEE Trans. Parallel Distrib. Syst.*, vol. 25, no. 12, pp. 3328–3338, Dec. 2014.
- [35] R. Hu, G. Liu, J. Jiang, and L. Wang, "A new resources provisioning method based on QoS differentiation and VM resizing in IaaS," *Math. Problems Eng.*, vol. 2015, pp. 1–9, Jul. 2015.
- [36] C. Ge, Z. Sun, N. Wang, K. Xu, and J. Wu, "Energy management in cross-domain content delivery networks: A theoretical perspective," *IEEE Trans. Netw. Service Manag.*, vol. 11, no. 3, pp. 264–277, Sep. 2014.
- [37] T. Wood, K. K. Ramakrishnan, P. Shenoy, and J. Van der Merwe, "CloudNet: Dynamic pooling of cloud resources by live WAN migration of virtual machines," *ACM SIGPLAN Notices*, vol. 46, no. 7, pp. 121–132, 2011.
- [38] F. Travostino *et al.*, "Seamless live migration of virtual machines over the MAN/WAN," *Future Gener. Comput. Syst.*, vol. 22, no. 8, pp. 901–907, 2006.
- [39] K. Takahashi, K. Sasada, and T. Hirofuchi, "A fast virtual machine storage migration technique using data deduplication," in *Proc. Cloud Comput.*, 2012, pp. 57–64.
- [40] B. Sotomayor, R. S. Montero, I. M. Llorente, and I. Foster, "Virtual infrastructure management in private and hybrid clouds," *IEEE Internet Comput.*, vol. 13, no. 5, pp. 14–22, Sep./Oct. 2009.
- [41] Y. Lu, X. Xu, and J. Xu, "Development of a hybrid manufacturing cloud," *J. Manuf. Syst.*, vol. 33, no. 4, pp. 551–566, 2014.
- [42] *Practical Guide to Hybrid Cloud Computing*. [Online]. Available: <http://www.cloud-council.org/deliverables/CSCC-Practical-Guide-to-Hybrid-Cloud-Computing.pdf>
- [43] A. Murphy, "Enabling long distance live migration with F5 and VMware vMotion," F5 Netw., Inc., Seattle, WA, USA, White Paper, 2011.
- [44] A. Celesti, F. Tusa, M. Villari, and A. Puliafito, "How to enhance cloud architectures to enable cross-federation," in *Proc. IEEE 3rd Int. Conf. Cloud Comput. (CLOUD)*, Miami, FL, USA, 2010, pp. 337–345.
- [45] W. Cerroni, "Multiple virtual machine live migration in federated cloud systems," in *Proc. IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, Toronto, ON, Canada, 2014, pp. 25–30.
- [46] A. Celesti, F. Tusa, M. Villari, and A. Puliafito, "Improving virtual machine migration in federated cloud environments," in *Proc. 2nd Int. Conf. Evol. Internet (INTERNET)*, Valencia, Spain, 2010, pp. 61–67.
- [47] R. Buyya, J. Broberg, and A. M. Goscinski, *Cloud Computing: Principles and Paradigms*. Somerset, U.K.: Wiley, 2010, vol. 87.
- [48] B. Rochwerger *et al.*, "The reservoir model and architecture for open federated cloud computing," *IBM J. Res. Develop.*, vol. 53, no. 4, pp. 535–545, 2009.
- [49] EGI. Accessed: Jan. 2017. [Online]. Available: <https://www.egi.eu>
- [50] B. Satzger, W. Hummer, C. Inzinger, P. Leitner, and S. Dustdar, "Winds of change: From vendor lock-in to the meta cloud," *IEEE Internet Comput.*, vol. 17, no. 1, pp. 69–73, Jan./Feb. 2013.
- [51] J. Opara-Martins, R. Sahandi, and F. Tian, "Critical review of vendor lock-in and its impact on adoption of cloud computing," in *Proc. Int. Conf. Inf. Society (i-Soc.)*, London, U.K., 2014, pp. 92–97.
- [52] M. ETSI, "Mobile edge computing (MEC); framework and reference architecture," *ETSI, DGS MEC*, vol. 3, 2016.
- [53] M. Satyanarayanan *et al.*, "Cloudlets: At the leading edge of mobile-cloud convergence," in *Proc. 6th Int. Conf. Mobile Comput. Appl. Services (MobiCASE)*, Austin, TX, USA, 2014, pp. 1–9.
- [54] F. Han, S. Zhao, L. Zhang, and J. Wu, "Survey of strategies for switching off base stations in heterogeneous networks for greener 5G systems," *IEEE Access*, vol. 4, pp. 4959–4973, 2016.
- [55] N. Grozev and R. Buyya, "Performance modelling and simulation of three-tier applications in cloud and multi-cloud environments," *Comput. J.*, vol. 58, no. 1, pp. 1–22, Jan. 2015.
- [56] C. Weinhardt *et al.*, "Cloud computing—A classification, business models, and research directions," *Bus. Inf. Syst. Eng.*, vol. 1, no. 5, pp. 391–399, 2009.
- [57] R. Dua, A. R. Raja, and D. Kakadia, "Virtualization vs containerization to support PaaS," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Boston, MA, USA, 2014, pp. 610–614.
- [58] M. J. Scheepers, "Virtualization and containerization of application infrastructure: A comparison," in *Proc. 21st Twente Student Conf. IT*, 2014, pp. 1–7.
- [59] A. Mirkin, A. Kuznetsov, and K. Kolyshkin, "Containers checkpointing and live migration," in *Proc. Linux Symp.*, vol. 2, 2008, pp. 85–90.
- [60] CRIU. [Online]. Available: https://criu.org/Main_Page
- [61] J. G. Hansen, "Virtual machine mobility with self-migration," Ph.D. dissertation, Dept. Comput. Sci., Univ. Copenhagen, Copenhagen, Denmark, 2009.
- [62] A. Mashtizadeh, E. Celebi, T. Garfinkel, and M. Cai, "The design and evolution of live storage migration in VMware ESX," in *Proc. Usenix Atc*, vol. 11, Portland, OR, USA, 2011, pp. 1–14.
- [63] X. Xu, H. Jin, S. Wu, and Y. Wang, "Rethink the storage of virtual machine images in clouds," *Future Gener. Comput. Syst.*, vol. 50, pp. 75–86, Sep. 2015.
- [64] M. Kozuch and M. Satyanarayanan, "Internet suspend/resume," in *Proc. 4th IEEE Workshop Mobile Comput. Syst. Appl.*, Callicoon, NY, USA, 2002, pp. 40–46.
- [65] B. Pawlowski *et al.*, "NFS version 3: Design and implementation," in *Proc. USENIX Summer*, Boston, MA, USA, 1994, pp. 137–152.
- [66] A. Whitaker, R. S. Cox, M. Shaw, and S. D. Gribble, "Constructing services with interposable virtual hardware," in *Proc. NSDI*, 2004, pp. 169–182.
- [67] C. P. Sapuntzakis *et al.*, "Optimizing the migration of virtual computers," *ACM SIGOPS Oper. Syst. Rev.*, vol. 36, no. SI, pp. 377–390, 2002.

- [68] M. Nelson, B.-H. Lim, and G. Hutchins, "Fast transparent migration for virtual machines," in *Proc. USENIX Annu. Tech. Conf. Gen. Track*, Anaheim, CA, USA, 2005, pp. 391–394.
- [69] A. Kivity, Y. Kamay, D. Laor, U. Lublin, and A. Liguori, "KVM: The linux virtual machine monitor," in *Proc. Linux Symp.*, vol. 1, 2007, pp. 225–230.
- [70] M. R. Hines, U. Deshpande, and K. Gopalan, "Post-copy live migration of virtual machines," *ACM SIGOPS Oper. Syst. Rev.*, vol. 43, no. 3, pp. 14–26, 2009.
- [71] T. Hirofuchi, H. Nakada, S. Itoh, and S. Sekiguchi, "Enabling instantaneous relocation of virtual machines with a lightweight VMM extension," in *Proc. 10th IEEE/ACM Int. Conf. Cluster Cloud Grid Comput.*, Melbourne, VIC, Australia, 2010, pp. 73–83.
- [72] L. Hu, J. Zhao, G. Xu, Y. Ding, and J. Chu, "HMDC: Live virtual machine migration based on hybrid memory copy and delta compression," *Appl. Math.*, vol. 7, no. 2L, pp. 639–646, 2013.
- [73] J. Kim, D. Chae, J. Kim, and J. Kim, "Guide-copy: Fast and silent migration of virtual machine for datacenters," in *Proc. Int. Conf. High Perform. Comput. Netw. Stor. Anal.*, Denver, CO, USA, 2013, p. 66.
- [74] P. R. Wilson, S. F. Kaplan, and Y. Smaragdakis, "The case for compressed caching in virtual memory systems," in *Proc. USENIX Annu. Tech. Conf. Gen. Track*, Monterey, CA, USA, 1999, pp. 101–116.
- [75] M. Ekman and P. Stenstrom, "A robust main-memory compression scheme," *ACM SIGARCH Comput. Archit. News*, vol. 33, no. 2, pp. 74–85, 2005.
- [76] M. Nelson and J.-L. Gailly, *The Data Compression Book*, vol. 2. New York, NY, USA: M&T Books, 1996.
- [77] H. Jin, L. Deng, S. Wu, X. Shi, and X. Pan, "Live virtual machine migration with adaptive, memory compression," in *Proc. IEEE Int. Conf. Cluster Comput. Workshops (CLUSTER)*, New Orleans, LA, USA, 2009, pp. 1–10.
- [78] H. Jin *et al.*, "MECOM: Live migration of virtual machines by adaptively compressing memory pages," *Future Gener. Comput. Syst.*, vol. 38, pp. 23–35, Sep. 2014.
- [79] M. Oberhumer, (2005). *LZO Real-Time Data Compression Library, User Manual for LZO Version 0.28*. Accessed: Feb. 1997. [Online]. Available: <http://www.info-sys.tuwien.ac.at/Staff/lux/marco/lzo.html>
- [80] S. Hacking and B. Hudzia, "Improving the live migration process of large enterprise applications," in *Proc. 3rd Int. Workshop Virtual. Technol. Distrib. Comput.*, Barcelona, Spain, 2009, pp. 51–58.
- [81] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," in *Proc. FAST*, vol. 3. San Francisco, CA, USA, 2003, pp. 115–130.
- [82] P. Svård, B. Hudzia, J. Tordsson, and E. Elmroth, "Evaluation of delta compression techniques for efficient live migration of large virtual machines," *ACM SIGPLAN Notices*, vol. 46, no. 7, pp. 111–120, 2011.
- [83] M. D. Hill, "Aspects of cache memory and instruction buffer performance," DTIC, Univ. California at Berkeley, Berkeley, CA, USA, Rep. UCB/CSD 87/381, 1987.
- [84] D. Pountain, "Run-length encoding," *Byte*, vol. 12, no. 6, pp. 317–319, 1987.
- [85] B. Geroft, Z. Vass, and Y. Ishikawa, "Utilizing memory content similarity for improving the performance of replicated virtual machines," in *Proc. 4th IEEE Int. Conf. Utility Cloud Comput. (UCC)*, 2011, pp. 73–80.
- [86] X. Zhang, Z. Huo, J. Ma, and D. Meng, "Exploiting data deduplication to accelerate live virtual machine migration," in *Proc. IEEE Int. Conf. Cluster Comput. (CLUSTER)*, 2010, pp. 88–96.
- [87] D. Gupta *et al.*, "Difference engine: Harnessing memory redundancy in virtual machines," *Commun. ACM*, vol. 53, no. 10, pp. 85–93, 2010.
- [88] D. Eastlake, III, and P. Jones, "U.S. secure hash algorithm 1 (SHA1)," Internet Eng. Task Force, Fremont, CA, USA, Rep. 3174, 2001.
- [89] P. Hsieh, *Hash Functions*. Accessed: Dec. 2016. [Online]. Available: <http://www.azillionmonkeys.com/qed/hash.html>
- [90] P. Riteau, C. Morin, and T. Priol, "Shrinker: Efficient wide-area live virtual machine migration using distributed content-based addressing," Ph.D. dissertation, Dept. Distrib. High Perform. Comput., INRIA, 2010.
- [91] P. Riteau, C. Morin, and T. Priol, "Shrinker: Improving live migration of virtual clusters over WANs with distributed data deduplication and content-based addressing," in *Proc. Eur. Conf. Parallel Process.*, Bordeaux, France, 2011, pp. 431–442.
- [92] I. Stoica, R. Morris, D. Karger, M. F. Kaashoek, and H. Balakrishnan, "Chord: A scalable peer-to-peer lookup service for Internet applications," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 31, no. 4, pp. 149–160, 2001.
- [93] Z. Zhang, L. Xiao, M. Zhu, and L. Ruan, "Mvmotion: A metadata based virtual machine migration in cloud," *Cluster Comput.*, vol. 17, no. 2, pp. 441–452, 2014.
- [94] C. A. Waldspurger, "Memory resource management in VMware ESX server," *ACM SIGOPS Oper. Syst. Rev.*, vol. 36, no. SI, pp. 181–194, 2002.
- [95] G. Mil6s, D. G. Murray, S. Hand, and M. A. Fetterman, "Satori: Enlightened page sharing," in *Proc. Conf. USENIX Annu. Tech. Conf.*, San Diego, CA, USA, 2009, p. 1.
- [96] J. F. Kloster, J. Kristensen, A. Mejlholm, and G. Behrmann, "On the feasibility of memory sharing: Content-based page sharing in the Xen virtual machine monitor," M.S. thesis, Dept. Comput. Sci., Aalborg Univ., Aalborg, Denmark, 2006.
- [97] T. Wood, "Improving data center resource management, deployment, and availability with virtualization," Ph.D. dissertation, Dept. Comput. Sci., Univ. Massachusetts at Amherst, Amherst, MA, USA, 2011.
- [98] C. Jo, E. Gustafsson, J. Son, and B. Egger, "Efficient live migration of virtual machines using shared storage," *ACM SIGPLAN Notices*, vol. 48, no. 7, pp. 41–50, 2013.
- [99] E. Park, B. Egger, and J. Lee, "Fast and space-efficient virtual machine checkpointing," *ACM SIGPLAN Notices*, vol. 46, no. 7, pp. 75–86, 2011.
- [100] M. Li, M. Zheng, and X. Hu, "Template-based memory deduplication method for inter-data center live migration of virtual machines," in *Proc. IEEE Int. Conf. Cloud Eng. (IC2E)*, Boston, MA, USA, 2014, pp. 127–134.
- [101] M. Zheng and X. Hu, "Template-based migration between data centers using distributed hash tables," in *Proc. 12th Int. Conf. Fuzzy Syst. Knowl. Disc. (FSKD)*, 2015, pp. 2443–2447.
- [102] P. Barham *et al.*, "Xen and the art of virtualization," *ACM SIGOPS Oper. Syst. Rev.*, vol. 37, no. 5, pp. 164–177, 2003.
- [103] (Apr. 2015). *Programming Intel Quickassist Technology Hardware Accelerators for Optimal Performance*. [Online]. Available: https://01.org/sites/default/files/page/332125_002_0.pdf
- [104] J. Zhang, L. Li, and D. Wang, "Optimizing VNF live migration via para-virtualization driver and quickassist technology," in *Proc. IEEE Int. Conf. Commun. (ICC)*, Paris, France, 2017, pp. 1–6.
- [105] A. Koto, H. Yamada, K. Ohmura, and K. Kono, "Towards unobtrusive VM live migration for cloud computing platforms," in *Proc. Asia-Pac. Workshop Syst.*, Seoul, South Korea, 2012, p. 7.
- [106] *InfiniBand Architecture Specification: Release 1.0*, InfiniBand Trade Assoc., Portland, OR, USA, 2000.
- [107] N. J. Boden *et al.*, "Myrinet: A gigabit-per-second local area network," *IEEE Micro*, vol. 15, no. 1, pp. 29–36, Feb. 1995.
- [108] W. Huang, Q. Gao, J. Liu, and D. K. Panda, "High performance virtual machine migration with RDMA over modern interconnects," in *Proc. IEEE Int. Conf. Cluster Comput.*, Austin, TX, USA, 2007, pp. 11–20.
- [109] J. Liu, J. Wu, S. P. Kini, P. Wyckoff, and D. K. Panda, "High performance RDMA-based MPI implementation over infiniband," in *Proc. 17th Annu. Int. Conf. Supercomput.*, San Francisco, CA, USA, 2003, pp. 295–304.
- [110] K. Z. Ibrahim, S. A. Hofmeyr, C. Iancu, and E. Roman, "Optimized pre-copy live migration for memory intensive applications," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Analysis*, Seattle, WA, USA, 2011, pp. 1–11.
- [111] H. Liu, H. Jin, X. Liao, C. Yu, and C.-Z. Xu, "Live virtual machine migration via asynchronous replication and state synchronization," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 12, pp. 1986–1999, Dec. 2011.
- [112] H. Liu, H. Jin, X. Liao, L. Hu, and C. Yu, "Live migration of virtual machine based on full system trace and replay," in *Proc. 18th ACM Int. Symp. High Perform. Distrib. Comput.*, 2009, pp. 101–110.
- [113] G. W. Dunlap, S. T. King, S. T. Cinar, M. A. Basrai, and P. M. Chen, "ReVirt: Enabling intrusion analysis through virtual-machine logging and replay," *ACM SIGOPS Oper. Syst. Rev.*, vol. 36, no. SI, pp. 211–224, 2002.
- [114] B. Cully *et al.*, "Remus: High availability via asynchronous virtual machine replication," in *Proc. 5th USENIX Symp. Netw. Syst. Design Implement.*, San Francisco, CA, USA, 2008, pp. 161–174.
- [115] P. Svård, J. Tordsson, B. Hudzia, and E. Elmroth, "High performance live migration through dynamic page transfer reordering and compression," in *Proc. IEEE 3rd Int. Conf. Cloud Comput. Technol. Sci. (CloudCom)*, Athens, Greece, 2011, pp. 542–548.
- [116] F. Checconi, T. Cucinotta, and M. Stein, "Real-time issues in live migration of virtual machines," in *Proc. Eur. Conf. Parallel Process.*, Delft, The Netherlands, 2009, pp. 454–466.

- [117] H. Jin *et al.*, "Optimizing the live migration of virtual machine by CPU scheduling," *J. Netw. Comput. Appl.*, vol. 34, no. 4, pp. 1088–1096, 2011.
- [118] Z. Liu, W. Qu, W. Liu, and K. Li, "Xen live migration with slowdown scheduling algorithm," in *Proc. Int. Conf. Parallel Distrib. Comput. Appl. Technol. (PDCAT)*, Wuhan, China, 2010, pp. 215–221.
- [119] M. Atif and P. Strazdins, "Optimizing live migration of virtual machines in SMP clusters for HPC applications," in *Proc. 6th IFIP Int. Conf. Netw. Parallel Comput. (NPC)*, Gold Coast, QLD, Australia, 2009, pp. 51–58.
- [120] U. Deshpande, X. Wang, and K. Gopalan, "Live gang migration of virtual machines," in *Proc. 20th Int. Symp. High Perform. Distrib. Comput.*, San Jose, CA, USA, 2011, pp. 135–146.
- [121] U. Deshpande, U. Kulkarni, and K. Gopalan, "Inter-rack live migration of multiple virtual machines," in *Proc. 6th Int. Workshop Virtualization Technol. Distrib. Comput. Date*, Delft, The Netherlands, 2012, pp. 19–26.
- [122] U. Deshpande, B. Schlinder, E. Adler, and K. Gopalan, "Gang migration of virtual machines using cluster-wide deduplication," in *Proc. 13th IEEE/ACM Int. Symp. Cluster Cloud Grid Comput. (CCGrid)*, Delft, The Netherlands, 2013, pp. 394–401.
- [123] M. F. Bari, M. F. Zhani, Q. Zhang, R. Ahmed, and R. Boutaba, "CQNCr: Optimal VM migration planning in cloud data centers," in *Proc. Netw. Conf. IFIP*, Trondheim, Norway, 2014, pp. 1–9.
- [124] H. Wang, Y. Li, Y. Zhang, and D. Jin, "Virtual machine migration planning in software-defined networks," in *Proc. IEEE Conf. Comput. Commun. (INFOCOM)*, Hong Kong, 2015, pp. 487–495.
- [125] T. K. Sarker and M. Tang, "Performance-driven live migration of multiple virtual machines in datacenters," in *Proc. IEEE Int. Conf. Granular Comput. (GrC)*, Beijing, China, 2013, pp. 253–258.
- [126] H. Liu and B. He, "VMbuddies: Coordinating live migration of multi-tier applications in cloud environments," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 4, pp. 1192–1205, Apr. 2015.
- [127] T. S. Kang, M. Tsugawa, J. Fortes, and T. Hirofuchi, "Reducing the migration times of multiple VMS on WANs using a feedback controller," in *Proc. IEEE 27th Int. Parallel Distrib. Process. Symp. Workshops Ph.D. Forum (IPDPSW)*, Cambridge, MA, USA, 2013, pp. 1480–1489.
- [128] T. S. Kang, M. O. Tsugawa, A. M. Matsunaga, T. Hirofuchi, and J. A. B. Fortes, "Design and implementation of middleware for cloud disaster recovery via virtual machine migration management," in *Proc. IEEE/ACM 7th Int. Conf. Utility Cloud Comput.*, London, U.K., 2014, pp. 166–175.
- [129] V. Jacobson, "Congestion avoidance and control," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 18, no. 4, pp. 314–329, 1988.
- [130] K. Ye, X. Jiang, D. Huang, J. Chen, and B. Wang, "Live migration of multiple virtual machines with resource reservation in cloud computing environments," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, Washington, DC, USA, 2011, pp. 267–274.
- [131] P. Liu *et al.*, "Heterogeneous live migration of virtual machines," in *Proc. Int. Workshop Virtualization Technol. (IWVT)*, Beijing, China, 2008.
- [132] S. Nathan, P. Kulkarni, and U. Bellur, "Resource availability based performance benchmarking of virtual machine migrations," in *Proc. 4th ACM/SPEC Int. Conf. Perform. Eng.*, Prague, Czech Republic, 2013, pp. 387–398.
- [133] B. R. Raghunath and B. Annappa, "Virtual machine migration triggering using application workload prediction," *Procedia Comput. Sci.*, vol. 54, pp. 167–176, Aug. 2015.
- [134] A. Baruchi, E. T. Midorikawa, and M. A. S. Netto, "Improving virtual machine live migration via application-level workload analysis," in *Proc. 10th Int. Conf. Netw. Service Manag. (CNSM)*, Rio de Janeiro, Brazil, 2014, pp. 163–168.
- [135] V. Mann *et al.*, "Remedy: Network-aware steady state VM management for data centers," in *Proc. NETWORKING*, Prague, Czech Republic, 2012, pp. 190–204.
- [136] J. Xia, D. Pang, Z. Cai, M. Xu, and G. Hu, "Reasonably migrating virtual machine in NFV-enabled networks," in *Proc. IEEE Int. Conf. Comput. Inf. Technol. (CIT)*, Nadi, Fiji, 2016, pp. 361–366.
- [137] M. R. Hines and K. Gopalan, "Post-copy based live virtual machine migration using adaptive pre-paging and dynamic self-ballooning," in *Proc. ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environ.*, Washington, DC, USA, 2009, pp. 51–60.
- [138] S. Sahni and V. Varma, "A hybrid approach to live migration of virtual machines," in *Proc. IEEE Int. Conf. Cloud Comput. Emerg. Markets (CEEM)*, Bengaluru, India, 2012, pp. 1–5.
- [139] T. Hirofuchi, H. Nakada, S. Itoh, and S. Sekiguchi, "Reactive consolidation of virtual machines enabled by postcopy live migration," in *Proc. 5th Int. Workshop Virtualization Technol. Distrib. Comput.*, San Jose, CA, USA, 2011, pp. 11–18.
- [140] A. Shribman and B. Hudzia, "Pre-copy and post-copy VM live migration for memory intensive applications," in *Proc. Eur. Conf. Parallel Process.*, 2012, pp. 539–547.
- [141] U. Deshpande and K. Keahey, "Traffic-sensitive live migration of virtual machines," *Future Gener. Comput. Syst.*, vol. 72, pp. 118–128, 2016.
- [142] U. Deshpande *et al.*, "Agile live migration of virtual machines," in *Proc. IEEE Int. Parallel Distrib. Process. Symp.*, Chicago, IL, USA, 2016, pp. 1061–1070.
- [143] V. Shrivastava *et al.*, "Application-aware virtual machine migration in data centers," in *Proc. IEEE INFOCOM*, Shanghai, China, 2011, pp. 66–70.
- [144] S. Akoush, R. Sohan, A. C. Rice, A. W. Moore, and A. Hopper, "Predicting the performance of virtual machine migration," in *Proc. IEEE Int. Symp. Modeling Anal. Simulat. Comput. Telecommun. Syst. (MASCOTS)*, 2010, pp. 37–46.
- [145] T. Treutner and H. Hlavacs, "Service level management for iterative pre-copy live migration," in *Proc. 8th Int. Conf. Netw. Service Manag.*, Las Vegas, NV, USA, 2012, pp. 252–256.
- [146] VMware Inc. Accessed: Dec. 2016. [Online]. Available: <http://www.vmware.com/>
- [147] N. S. Lemak, "Data mover," U.S. Patent 4 296 465, Oct. 20, 1981.
- [148] K. K. Ramakrishnan, P. Shenoy, and J. Van der Merwe, "Live data center migration across WANs: A robust cooperative context aware approach," in *Proc. SIGCOMM Workshop Internet Netw. Manag.*, Kyoto, Japan, 2007, pp. 262–267.
- [149] C. Ruemmler and J. Wilkes, *UNIX Disk Access Patterns*. Palo Alto, CA, USA: Hewlett-Packard Lab., 1992.
- [150] DRBD. Accessed: Mar. 2017. [Online]. Available: <https://docs.linbit.com/>
- [151] *Hast—Highly Available Storage*. Accessed: Mar. 2017. [Online]. Available: <https://wiki.freebsd.org/HAST>
- [152] R. Bradford, E. Kotsovinos, A. Feldmann, and H. Schiöberg, "Live wide-area migration of virtual machines including local persistent state," in *Proc. 3rd Int. Conf. Virtual Execution Environ.*, San Diego, CA, USA, 2007, pp. 169–179.
- [153] K. R. Jayaram *et al.*, "An empirical analysis of similarity in virtual machine images," in *Proc. Middleware Ind. Track Workshop*, Lisbon, Portugal, 2011, Art. no. 6.
- [154] M. O. Rabin, *Fingerprinting by Random Polynomials*. Cambridge, MA, USA: Center Res. Comput. Techn., Aiken Comput. Lab., Univ., 1981.
- [155] K. Jin and E. L. Miller, "The effectiveness of deduplication on virtual machine disk images," in *Proc. SYSTOR Israeli Exp. Syst. Conf.*, Haifa, Israel, 2009, p. 7.
- [156] F. Zhang, X. Fu, and R. Yahyapour, "Layermover: Storage migration of virtual machine across data centers based on three-layer image structure," in *Proc. IEEE 24th Int. Symp. Modeling Anal. Simulat. Comput. Telecommun. Syst. (MASCOTS)*, London, U.K., 2016, pp. 400–405.
- [157] S. Al-Kiswani, D. Subhraveti, P. Sarkar, and M. Ripeanu, "VMFlock: Virtual machine co-migration for the cloud," in *Proc. 20th Int. Symp. High Perform. Distrib. Comput.*, San Jose, CA, USA, 2011, pp. 159–170.
- [158] S. K. Bose, S. Brock, R. Skeoch, N. Shaikh, and S. Rao, "Optimizing live migration of virtual machines across wide area networks using integrated replication and scheduling," in *Proc. IEEE Int. Syst. Conf. (SysCon)*, Montreal, QC, Canada, 2011, pp. 97–102.
- [159] S. K. Bose, S. Brock, R. Skeoch, and S. Rao, "Cloudspider: Combining replication with scheduling for optimizing live migration of virtual machines across wide area networks," in *Proc. 11th IEEE/ACM Int. Symp. Cluster Cloud Grid Comput.*, Newport Beach, CA, USA, 2011, pp. 13–22.
- [160] F. Zhang, X. Fu, and R. Yahyapour, "CBase: A new paradigm for fast virtual machine migration across data centers," in *Proc. 17th IEEE/ACM Int. Symp. Cluster Cloud Grid Comput.*, Madrid, Spain, 2017, pp. 284–293.
- [161] S. Sud *et al.*, "Dynamic migration of computation through virtualization of the mobile platform," *Mobile Netw. Appl.*, vol. 17, no. 2, pp. 206–215, 2012.
- [162] T. Hirofuchi, H. Ogawa, H. Nakada, S. Itoh, and S. Sekiguchi, "A live storage migration mechanism over WAN for relocatable virtual machine services on clouds," in *Proc. 9th IEEE/ACM Int. Symp. Cluster Comput. Grid*, Shanghai, China, 2009, pp. 460–465.

- [163] T. Hirofuchi, H. Nakada, H. Ogawa, S. Itoh, and S. Sekiguchi, "A live storage migration mechanism over WAN and its performance evaluation," in *Proc. 3rd Int. Workshop Virtual. Technol. Distrib. Comput.*, Barcelona, Spain, 2009, pp. 67–74.
- [164] P. T. Breuer, A. M. Lopez, and A. G. Ares, "The network block device," *Linux J.*, vol. 2000, no. 73, p. 40, 2000.
- [165] C. Tang, "FVD: A high-performance virtual machine image format for cloud," in *Proc. USENIX Annu. Tech. Conf.*, Portland, OR, USA, 2011, p. 2.
- [166] J. Zheng, T. S. E. Ng, and K. Sripanidkulchai, "Workload-aware live storage migration for clouds," *ACM SIGPLAN Notices*, vol. 46, no. 7, pp. 133–144, 2011.
- [167] B. Nicolae and F. Cappello, "A hybrid local storage transfer scheme for live migration of I/O intensive workloads," in *Proc. 21st Int. Symp. High-Perform. Parallel Distrib. Comput.*, Delft, The Netherlands, 2012, pp. 85–96.
- [168] Q. Zhang, L. Cheng, and R. Boutaba, "Cloud computing: State-of-the-art and research challenges," *J. Internet Services Appl.*, vol. 1, no. 1, pp. 7–18, 2010.
- [169] J. Bi, Z. Zhu, R. Tian, and Q. Wang, "Dynamic provisioning modeling for virtualized multi-tier applications in cloud data center," in *Proc. IEEE 3rd Int. Conf. Cloud Comput. (CLOUD)*, Miami, FL, USA, 2010, pp. 370–377.
- [170] J. Zheng, T. S. E. Ng, K. Sripanidkulchai, and Z. Liu, "Pacer: A progress management system for live virtual machine migration in cloud computing," *IEEE Trans. Netw. Service Manag.*, vol. 10, no. 4, pp. 369–382, Dec. 2013.
- [171] J. Zheng, T. S. E. Ng, K. Sripanidkulchai, and Z. Liu, "COMMA: Coordinating the migration of multi-tier applications," *ACM SIGPLAN Notices*, vol. 49, no. 7, pp. 153–164, 2014.
- [172] W. Cerroni, "Network performance of multiple virtual machine live migration in cloud federations," *J. Internet Services Appl.*, vol. 6, no. 1, p. 6, 2015.
- [173] K. Tsakalozos, V. Verroios, M. Roussopoulos, and A. Delis, "Time-constrained live VM migration in share-nothing IaaS-clouds," in *Proc. IEEE 7th Int. Conf. Cloud Comput. (CLOUD)*, 2014, pp. 56–63.
- [174] K. Tsakalozos, V. Verroios, M. Roussopoulos, and A. Delis, "Live VM migration under time-constraints in share-nothing IaaS-clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 8, pp. 2285–2298, Aug. 2017.
- [175] Y. Luo *et al.*, "Live and incremental whole-system migration of virtual machines using block-bitmap," in *Proc. IEEE Int. Conf. Cluster Comput.*, Tsukuba, Japan, 2008, pp. 99–106.
- [176] R. Zhou, F. Liu, C. Li, and T. Li, "Optimizing virtual machine live storage migration in heterogeneous storage environment," *ACM SIGPLAN Notices*, vol. 48, no. 7, pp. 73–84, 2013.
- [177] T. Lu *et al.*, "Successor: Proactive cache warm-up of destination hosts in virtual machine migration contexts," in *Proc. 35th Annu. IEEE Int. Conf. Comput. Commun. (IEEE INFOCOM)*, San Francisco, CA, USA, 2016, pp. 1–9.
- [178] Z. Shen *et al.*, "Follow the sun through the clouds: Application migration for geographically shifting workloads," in *Proc. 7th ACM Symp. Cloud Comput.*, 2016, pp. 141–154.
- [179] Q. Jia, Z. Shen, W. Song, R. Van Renesse, and H. Weatherspoon, "SuperCloud: Opportunities and challenges," *ACM SIGOPS Oper. Syst. Rev.*, vol. 49, no. 1, pp. 137–141, 2015.
- [180] M. Arif, A. K. Kiani, and J. Qadir, "Machine learning based optimized live virtual machine migration over WAN links," *Telecommun. Syst.*, vol. 64, no. 2, pp. 245–257, 2017.
- [181] S. Kumar and K. Schwan, "Netchannel: A VMM-level mechanism for continuous, transparent device access during VM migration," in *Proc. 4th ACM SIGPLAN/SIGOPS Int. Conf. Virtual Execution Environ.*, 2008, pp. 31–40.
- [182] *Data Center Interconnect: Layer 2 Extension Between Remote Data Centers*. Accessed: Mar. 2017. [Online]. Available: http://www.cisco.com/c/en/us/products/collateral/data-center-virtualization/data-center-interconnect/white_paper_c11_493718.html
- [183] *Cisco Overlay Transport Virtualization Technology Introduction and Deployment Considerations*. Accessed: Mar. 2017. [Online]. Available: http://www.cisco.com/c/en/us/td/docs/solutions/Enterprise/Data_Center/DCI/whitepaper/DCI3_OTV_Intro/DCI_1.html
- [184] X. Jiang and D. Xu, "Violin: Virtual internetworking on overlay infrastructure," in *Parallel and Distributed Processing and Applications*, vol. 3358. Hong Kong: Springer, Dec. 2004, pp. 937–946.
- [185] A. Ganguly, A. Agrawal, P. O. Boykin, and R. Figueiredo, "WOW: Self-organizing wide area overlay networks of virtual workstations," in *Proc. 15th IEEE Int. Symp. High Perform. Distrib. Comput.*, Paris, France, 2006, pp. 30–42.
- [186] A. Ganguly, A. Agrawal, P. O. Boykin, and R. Figueiredo, "IP over P2P: Enabling self-configuring virtual IP networks for grid computing," in *Proc. 20th Int. Parallel Distrib. Process. Symp. (IPDPS)*, 2006, p. 10.
- [187] P. O. Boykin *et al.* *Brunet Software Library*. [Online]. Available: <http://brunet.ee.ucla.edu/brunet>
- [188] K. Nagin *et al.*, "Inter-cloud mobility of virtual machines," in *Proc. 4th Annu. Int. Conf. Syst. Stor.*, Haifa, Israel, 2011, p. 3.
- [189] D. Hadas, S. Guenender, and B. Rochwerger, "Virtual network services for federated cloud computing," IBM Res. Divis., HRL, Malibu, CA, USA, Rep. H-0269, 2009.
- [190] B. Rochwerger *et al.*, "Reservoir—When one cloud is not enough," *Computer*, vol. 44, no. 3, pp. 44–51, 2011.
- [191] *Provider Backbone Bridges*. [Online]. Available: <http://www.ieee802.org/1/pages/802.1ah.html>
- [192] *Shortest Path Bridging*. [Online]. Available: <http://www.ieee802.org/1/pages/802.1aq.html>
- [193] (Jul. 2011). *Routing Bridges (Rbridges): Base Protocol Specification*. [Online]. Available: <https://tools.ietf.org/html/rfc6325>
- [194] R. Perlman and D. Eastlake, "Introduction to trill," *Internet Protocol J.*, vol. 14, no. 3, pp. 2–20, 2011.
- [195] C. Perkins, "IP encapsulation within IP," 1996.
- [196] B. Wellington, "Secure domain name system (DNS) dynamic update," Internet Eng. Task Force, Fremont, CA, USA, Rep. 3007, 2000.
- [197] E. Silvera, G. Sharaby, D. Lorenz, and I. Shapira, "IP mobility to support live migration of virtual machines across subnets," in *Proc. SYSTOR Israeli Exp. Syst. Conf.*, Haifa, Israel, 2009, p. 13.
- [198] *F5 Networks*. [Online]. Available: <https://f5.com/>
- [199] C. Perkins, "IP mobility support for IPv4," 2002.
- [200] D. Johnson, C. Perkins, and J. Arkko, "Mobility support in IPv6," Internet Eng. Task Force, Fremont, CA, USA, Rep. 3775, 2004.
- [201] Q. Li, J. Huai, J. Li, T. Wo, and M. Wen, "HyperMIP: Hypervisor controlled mobile IP for virtual machine live migration across networks," in *Proc. 11th IEEE High Assurance Syst. Eng. Symp. (HASE)*, Nanjing, China, 2008, pp. 80–88.
- [202] E. Harney, S. Goasguen, J. Martin, M. Murphy, and M. Westall, "The efficacy of live virtual machine migrations over the Internet," in *Proc. 2nd Int. Workshop Virtual. Technol. Distrib. Comput.*, Reno, NV, USA, 2007, p. 8.
- [203] S. Gundavelli, K. Leung, V. Devarapalli, K. Chowdhury, and B. Patil, "Proxy mobile IPv6," Internet Eng. Task Force, Fremont, CA, USA, Rep. 5213, 2008.
- [204] S. Kassahun, A. Demessie, and D. Ilie, "A PMIPv6 approach to maintain network connectivity during VM live migration over the Internet," in *Proc. IEEE 3rd Int. Conf. Cloud Netw. (CloudNet)*, Luxembourg City, Luxembourg, 2014, pp. 64–69.
- [205] J. Lei and X. Fu, "Evaluating the benefits of introducing PMIPv6 for localized mobility management," in *Proc. Int. Wireless Commun. Mobile Comput. Conf. (IWCMC)*, 2008, pp. 74–80.
- [206] R. Inayat *et al.*, "MAT: An end-to-end mobile communication architecture with seamless IP handoff support for the next generation Internet," in *Web and Communication Technologies and Internet-Related Social Issues—HSI 2003*. Heidelberg, Germany: Springer, 2003, pp. 465–475.
- [207] R. Inayat, R. Aibara, K. Nishimura, T. Fujita, and K. Maeda, "An end-to-end network architecture for supporting mobility in wide area wireless networks," *IEICE Trans. Commun.*, vol. 87, no. 6, pp. 1584–1593, 2004.
- [208] T. Kondo, R. Aibara, K. Suga, and K. Maeda, "A mobility management system for the global live migration of virtual machine across multiple sites," in *Proc. IEEE 38th Int. Comput. Softw. Appl. Conf. Workshops (COMPSACW)*, Västerås, Sweden, 2014, pp. 73–77.
- [209] H. Watanabe, T. Ohigashi, T. Kondo, K. Nishimura, and R. Aibara, "A performance improvement method for the global live migration of virtual machine with IP mobility," in *Proc. 5th Int. Conf. Mobile Comput. Ubiquitous Netw.*, Seattle, WA, USA, 2010, pp. 194–199.
- [210] D. Farinacci, D. Lewis, D. Meyer, and V. Fuller, "The locator/ID separation protocol (LISP)," Internet Eng. Task Force, Fremont, CA, USA, RFC 6830, Jan. 2013.
- [211] P. Raad *et al.*, "Achieving sub-second downtimes in large-scale virtual machine migrations with LISP," *IEEE Trans. Netw. Service Manag.*, vol. 11, no. 2, pp. 133–143, Jun. 2014.
- [212] *Locator ID Separation Protocol (LISP) VM Mobility Solution*. Accessed: Mar. 2017. [Online]. Available: http://www.cisco.com/c/dam/en/us/products/collateral/ios-nx-os-software/locator-id-separation-protocol-lisp/lisp-vm_mobility_wp.pdf

- [213] R. Xie, Y. Wen, X. Jia, and H. Xie, "Supporting seamless virtual machine migration via named data networking in cloud data center," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 12, pp. 3485–3497, Dec. 2015.
- [214] L. Zhang *et al.*, "Named data networking," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 3, pp. 66–73, 2014.
- [215] A. C. Snoeren and H. Balakrishnan, "An end-to-end approach to host mobility," in *Proc. 6th Annu. Int. Conf. Mobile Comput. Netw.*, Boston, MA, USA, 2000, pp. 155–166.
- [216] A. C. Snoeren, D. G. Andersen, and H. Balakrishnan, "Fine-grained failover using connection migration," in *USITS*, vol. 1. San Francisco, CA, USA, 2001, p. 19.
- [217] D. A. Maltz and P. Bhagwat, "MSOCKS: An architecture for transport layer mobility," in *Proc. IEEE 17th Annu. Joint Conf. IEEE Comput. Commun. Soc. (INFOCOM)*, vol. 3. San Francisco, CA, USA, 1998, pp. 1037–1045.
- [218] U. Kalim, M. K. Gardner, E. J. Brown, and W.-C. Feng, "Seamless migration of virtual machines across networks," in *Proc. 22nd Int. Conf. Comput. Commun. Netw. (ICCCN)*, Nassau, Bahamas, 2013, pp. 1–7.
- [219] E. J. Brown, M. K. Gardner, U. Kalim, and W.-C. Feng, "Restoring end-to-end resilience in the presence of middleboxes," in *Proc. IEEE 20th Int. Conf. Comput. Commun. Netw. (ICCCN)*, 2011, pp. 1–7.
- [220] M. Mahalingam *et al.*, "Virtual extensible local area network (VXLAN): A framework for overlaying virtualized layer 2 networks over layer 3 networks," Internet Eng. Task Force, Fremont, CA, USA, Rep. 7348, 2014.
- [221] J. Gross and B. Davie, "A stateless transport tunneling protocol for network virtualization (STT)," 2016.
- [222] M. Sridharan *et al.*, "NVGRE: Network virtualization using generic routing encapsulation," IETF Draft, Fremont, CA, USA, 2011.
- [223] D. Kreutz *et al.*, "Software-defined networking: A comprehensive survey," *Proc. IEEE*, vol. 103, no. 1, pp. 14–76, Jan. 2015.
- [224] V. Mann, A. Vishnoi, K. Kannan, and S. Kalyanaraman, "CrossRoads: Seamless VM mobility across data centers through software defined networking," in *Proc. IEEE Netw. Oper. Manag. Symp. (NOMS)*, 2012, pp. 88–96.
- [225] R. N. Mysore *et al.*, "Portland: A scalable fault-tolerant layer 2 data center network fabric," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 39, no. 4, pp. 39–50, 2009.
- [226] S. Xiao *et al.*, "Traffic-aware virtual machine migration in topology-adaptive DCN," in *Proc. IEEE 24th Int. Conf. Netw. Protocols (ICNP)*, Singapore, 2016, pp. 1–10.
- [227] B. Boughzala, R. B. Ali, M. Lemay, Y. Lemieux, and O. Cherkaoui, "OpenFlow supporting inter-domain virtual machine migration," in *Proc. IEEE 8th Int. Conf. Wireless Opt. Commun. Netw. (WOCN)*, Paris, France, 2011, pp. 1–7.
- [228] J. Liu, Y. Li, and D. Jin, "SDN-based live VM migration across datacenters," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 583–584, 2015.
- [229] P. Samadi, J. Xu, and K. Bergman, "Virtual machine migration over optical circuit switching network in a converged inter/intra data center architecture," in *Proc. Opt. Fiber Commun. Conf. Exhibit. (OFC)*, Los Angeles, CA, USA, 2015, pp. 1–3.
- [230] M. Zhao and R. J. Figueiredo, "Experimental study of virtual machine migration in support of reservation of cluster resources," in *Proc. IEEE 2nd Int. Workshop Virtual. Technol. Distrib. Comput. (VTDC)*, Reno, NV, USA, 2007, pp. 1–8.
- [231] W. Hu *et al.*, "A quantitative study of virtual machine live migration," in *Proc. ACM Cloud Auton. Comput. Conf.*, Miami, FL, USA, 2013, p. 11.
- [232] F. Salfner, P. Tröger, and A. Polze, "Downtime analysis of virtual machine live migration," in *Proc. 4th Int. Conf. Dependability (DEPEND) IARIA*, 2011, pp. 100–105.
- [233] J. Li *et al.*, "iMIG: Toward an adaptive live migration method for KVM virtual machines," *Comput. J.*, vol. 58, no. 6, pp. 1227–1242, 2014.
- [234] A. Strunk and W. Dargie, "Does live migration of virtual machines cost energy?" in *Proc. IEEE 27th Int. Conf. Adv. Inf. Netw. Appl. (AINA)*, Barcelona, Spain, 2013, pp. 514–521.
- [235] P. Bezerra, G. Martins, R. Gomes, F. Cavalcante, and A. Costa, "Evaluating live virtual machine migration overhead on client's application perspective," in *Proc. Int. Conf. Inf. Netw. (ICOIN)*, Da Nang, Vietnam, 2017, pp. 503–508.
- [236] J. Zhang, F. Ren, and C. Lin, "Delay guaranteed live migration of virtual machines," in *Proc. IEEE INFOCOM*, Toronto, ON, Canada, 2014, pp. 574–582.
- [237] Y. Chen, *Introduction to Probability Theory*, Lecture Notes on Information Theory, Duisburg-Essen Univ., Duisburg, Germany, 2010.
- [238] D. Basu, X. Wang, Y. Hong, H. Chen, and S. Bressan, "Learn-as-you-go with MEGH: Efficient live migration of virtual machines," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Atlanta, GA, USA, 2017, pp. 2608–2609.
- [239] D. Breitgand, G. Kutiel, and D. Raz, "Cost-aware live migration of services in the cloud," in *Proc. SYSTOR*, Boston, MA, USA, 2010, p. 3.
- [240] D. C. Montgomery, E. A. Peck, and G. G. Vining, *Introduction to Linear Regression Analysis*. Hoboken, NJ, USA: Wiley, 2015.
- [241] M. Galloway, G. Loewen, and S. Vrbsky, "Performance metrics of virtual machine live migration," in *Proc. IEEE 8th Int. Conf. Cloud Comput. (CLOUD)*, New York, NY, USA, 2015, pp. 637–644.
- [242] Y. Wu and M. Zhao, "Performance modeling of virtual machine live migration," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, Washington, DC, USA, 2011, pp. 492–499.
- [243] K. Rybina, W. Dargie, S. Umashankar, and A. Schill, "Modelling the live migration time of virtual machines," in *Proc. OTM Confederated Int. Conf. Move Meaningful Internet Syst.*, 2015, pp. 575–593.
- [244] A. Strunk, "A lightweight model for estimating energy cost of live migration of virtual machines," in *Proc. IEEE 6th Int. Conf. Cloud Comput. (CLOUD)*, Santa Clara, CA, USA, 2013, pp. 510–517.
- [245] J. Xia, Z. Cai, and M. Xu, "Optimized virtual network functions migration for NFV," in *Proc. IEEE 22nd Int. Conf. Parallel Distrib. Syst. (ICPADS)*, Wuhan, China, 2016, pp. 340–346.
- [246] S. Kikuchi and Y. Matsumoto, "Performance modeling of concurrent live migration operations in cloud computing systems using PRISM probabilistic model checker," in *Proc. IEEE Int. Conf. Cloud Comput. (CLOUD)*, Washington, DC, USA, 2011, pp. 49–56.
- [247] *Prism Model Checker*. Accessed: Mar. 2017. [Online]. Available: <http://www.prismmodelchecker.org/>
- [248] S. Nathan, U. Bellur, and P. Kulkarni, "Towards a comprehensive performance model of virtual machine live migration," in *Proc. 6th ACM Symp. Cloud Comput.*, 2015, pp. 288–301.
- [249] A. Aldhalaan and D. A. Menascé, "Analytic performance modeling and optimization of live VM migration," in *Proc. Eur. Workshop Perform. Eng.*, Venice, Italy, 2013, pp. 28–42.
- [250] F. Salfner, P. Tröger, and M. Richly, "Dependable estimation of downtime for virtual machine live migration," *Int. J. Adv. Syst. Meas.*, vol. 5, nos. 1–2, pp. 70–88, Jun. 2012.
- [251] Q. Luo, W. Fang, J. Wu, and Q. Chen, "Reliable broadband wireless communication for high speed trains using baseband cloud," *EURASIP J. Wireless Commun. Netw.*, vol. 2012, no. 1, p. 285, 2012.
- [252] K. Ha, P. Pillai, W. Richter, Y. Abe, and M. Satyanarayanan, "Just-in-time provisioning for cyber foraging," in *Proc. 11th Annu. Int. Conf. Mobile Syst. Appl. Services*, 2013, pp. 153–166.
- [253] K. Ha *et al.*, "Adaptive VM handoff across cloudlets," School Comput. Sci., Carnegie Mellon Univ., Pittsburgh, PA, USA, Rep. CMU-CS-15-113, 2015.
- [254] T. Taleb and A. Ksentini, "Follow me cloud: Interworking federated clouds and distributed mobile networks," *IEEE Netw.*, vol. 27, no. 5, pp. 12–19, Sep./Oct. 2013.
- [255] T. Taleb, P. Hasselmeyer, and F. G. Mir, "Follow-me cloud: An OpenFlow-based implementation," in *Proc. Int. Conf. IEEE Cyber Phys. Soc. Comput. Green Comput. Commun. (GreenCom) IEEE Internet Things (iThings/CPSCoM)*, Beijing, China, 2013, pp. 240–245.
- [256] F. Teka, C.-H. Lung, and S. A. Ajila, "Nearby live virtual machine migration using cloudlets and multipath TCP," *J. Cloud Comput.*, vol. 5, no. 1, p. 12, 2016.
- [257] *MPTCP*. Accessed: Sep. 2017. [Online]. Available: <http://multipath-tcp.org/>
- [258] Y. Qiu, C.-H. Lung, S. Ajila, and P. Srivastava, "LXC container migration in cloudlets under multipath TCP," in *Proc. IEEE 41st Annu. Comput. Softw. Appl. Conf. (COMPSAC)*, vol. 2. Turin, Italy, 2017, pp. 31–36.
- [259] E. Saurez, K. Hong, D. Lillethun, U. Ramachandran, and B. Ottenwälder, "Incremental deployment and migration of geo-distributed situation awareness applications in the fog," in *Proc. 10th ACM Int. Conf. Distrib. Event Based Syst.*, Irvine, CA, USA, 2016, pp. 258–269.
- [260] S. Secci, P. Raad, and P. Gallard, "Linking virtual machine mobility to user mobility," *IEEE Trans. Netw. Service Manag.*, vol. 13, no. 4, pp. 927–940, Dec. 2016.
- [261] B. Ottenwälder, B. Koldehofe, K. Rothermel, and U. Ramachandran, "MigCEP: Operator migration for mobility driven distributed complex event processing," in *Proc. 7th ACM Int. Conf. Distrib. Event Based Syst.*, 2013, pp. 183–194.

- [262] B. Ottenwälder *et al.*, “MCEP: A mobility-aware complex event processing system,” *ACM Trans. Internet Technol.*, vol. 14, no. 1, p. 6, 2014.
- [263] X. Sun and N. Ansari, “Avaptive avatar handoff in the cloudlet network,” *IEEE Trans. Cloud Comput.*, to be published, doi: [10.1109/TCC.2017.2701794](https://doi.org/10.1109/TCC.2017.2701794).
- [264] A. Machen, S. Wang, K. K. Leung, B. J. Ko, and T. Salonidis, “Live service migration in mobile edge clouds,” *arXiv preprint arXiv:1706.04118*, 2017.
- [265] L. Gkatzikis and I. Koutsopoulos, “Migrate or not? Exploiting dynamic task migration in mobile cloud computing systems,” *IEEE Wireless Commun.*, vol. 20, no. 3, pp. 24–32, Jun. 2013.
- [266] A. Ksentini, T. Taleb, and M. Chen, “A Markov decision process-based service migration procedure for follow me cloud,” in *Proc. IEEE Int. Conf. Commun. (ICC)*, Sydney, NSW, Australia, 2014, pp. 1350–1354.
- [267] S. Wang *et al.*, “Mobility-induced service migration in mobile micro-clouds,” in *Proc. IEEE Mil. Commun. Conf. (MILCOM)*, Baltimore, MD, USA, 2014, pp. 835–840.
- [268] T. Taleb and A. Ksentini, “An analytical model for follow me cloud,” in *Proc. IEEE Glob. Commun. Conf. (GLOBECOM)*, Atlanta, GA, USA, 2013, pp. 1291–1296.
- [269] A. Nadembega, A. S. Hafid, and R. Brisebois, “Mobility prediction model-based service migration procedure for follow me cloud to support QoS and QoE,” in *Proc. IEEE Int. Conf. Commun. (ICC)*, Kuala Lumpur, Malaysia, 2016, pp. 1–6.
- [270] X. Sun and N. Ansari, “PRIMAL: Profit maximization avatar placement for mobile edge computing,” in *Proc. IEEE Int. Conf. Commun. (ICC)*, Kuala Lumpur, Malaysia, 2016, pp. 1–6.
- [271] J. Oberheide, E. Cooke, and F. Jahanian, “Empirical exploitation of live virtual machine migration,” in *Proc. BlackHat DC Conv.*, 2008.



Guangming Liu received the B.S. and M.S. degrees from the National University of Defense Technology, Changsha, China, in 1980 and 1986, respectively, where he is currently a Professor with the College of Computer and the Director of the National Supercomputer Center in Tianjin, Tianjin, China. His research interests include high performance computing, massive storage technology, and cloud computing.



Xiaoming Fu received the Ph.D. degree in computer science from Tsinghua University, China, in 2000. He is a Full Professor with the Georg-August University of Göttingen. He was a Research Staff with Technical University Berlin. He joined the Georg-August University of Göttingen, Germany, in 2002, where he has been the Head of the Computer Networks Group since 2007. His research interests are architectures, protocols, and applications for networked systems, including information dissemination, mobile networking, cloud computing, and social networks.



Fei Zhang received the master's degree in computer science and technology from the National University of Defense Technology, China, in 2013. He is currently pursuing the Ph.D. degree with the Georg-August University of Göttingen and Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen, Germany. In 2012, he was with National Supercomputer Center, Tianjin, China, as a Researcher for one year. He has published papers in several referred international conferences. His research interests include cloud computing,

virtualization technology, data storage, and big data.



Ramin Yahyapour has been a Full Professor with the Georg-August University of Göttingen since 2011. He is also the Managing Director of the Gesellschaft für wissenschaftliche Datenverarbeitung mbH Göttingen, a joint compute and IT competence center of the university and the Max Planck Society. He is also the CIO of the Georg-August University of Göttingen and the University Medical Center Göttingen. He was a Professor with TU Dortmund University, where he was the Director of the IT & Media Center, and CIO. His research interest lies in the area of efficient resource management in its application to service-oriented infrastructures, cloud computing, and data management. He is especially interested in data and computing services for eScience.